Blind 75 — Beginner-Friendly Learning Sheet

A complete practical roadmap for mastering Blind 75 from scratch.

Written like a mentor explaining DSA to a junior developer who already knows basic syntax but freezes on coding interview problems.

---

1. Introduction

What is DSA?

DSA stands for Data Structures and Algorithms.

A data structure is a way to organize data inside the memory of a computer. Examples: array, hash map, tree, graph. Each one has its own strengths. An array is great for fast index access. A hash map is great for fast lookup by key. A tree is great for hierarchical data.

An algorithm is a step-by-step procedure to solve a problem. Examples: sorting numbers, finding the shortest path, searching in a list.

In short: a data structure is a container, and an algorithm is the recipe. Together they decide how fast and how cheaply your code runs.

Why Companies Ask DSA in Interviews

Companies do not ask DSA because every job requires you to invert a binary tree. They ask it because DSA is the cheapest way to test three things at once:

- Problem solving: can you take a vague problem and break it down?
- Communication: can you explain your thinking out loud, clearly?
- Engineering judgment: can you choose the right tool, estimate cost, and reason about edge cases?

A good DSA answer shows your thought process, not just memorized code. That is exactly what teams want at work.

How Blind 75 Helps

The Blind 75 is a curated list of 75 LeetCode problems. It is not random. It is built so that if you understand all 75 deeply, you will recognize the underlying pattern in around 80 to 90 percent of interview questions.

The list covers every important pattern: arrays, hashing, two pointers, sliding window, stack, binary search, linked lists, trees, tries, heap, backtracking, graphs, dynamic programming, greedy, intervals, and bit manipulation.

Once you know the pattern, you stop seeing 1000 different problems. You start seeing 16 patterns showing up in different costumes.

How to Study This Sheet

- Read the topic introduction first. Do not jump to problems before understanding the data structure.
- For each problem: read the brute force first, even if you already know the optimized solution. The brute force teaches you why the optimized version was invented.
- Always do a dry run on paper. If you cannot trace the code by hand, you do not understand it yet.
- After solving, close the sheet and re-implement from scratch the next day. Repetition without looking is the only thing that builds intuition.
- Do not chase quantity. Five deeply understood problems beat fifty memorized ones.
- When stuck, set a 25-minute timer. If you have not made progress, look at the hint, then close it and try again.

Rule of thumb: you have learned a problem only when you can solve it tomorrow without looking, and explain it to a friend in plain English.

How This Document Is Organized

Each topic has two parts. First, a topic introduction: what the data structure is, why it exists, what it is used for in real software systems, and its time and space costs. Second, every Blind 75 problem under that topic, broken down into nine sections:

- Problem Summary — the question in plain English.
- Real-Life Scenario — where this shows up in real software.
- Pattern Recognition — clues that tell you which technique to use.
- Brute Force Approach — the first idea, with its cost.
- Optimized Approach — the better idea, with the intuition behind it.
- Dry Run — a small example traced step by step.
- Code Implementation — clean Java and Python.
- Common Mistakes — pitfalls that catch beginners.
- Key Takeaways — what to lock into long-term memory.

2. Arrays and Hashing

Topic Introduction

What it is: An array stores elements in a continuous block of memory, accessed by index. A hash map (also called dictionary or hash table) stores key-to-value pairs and lets you look up a value by its key in roughly one step.

Why it exists: Arrays exist because we need ordered, indexable storage. Hash maps exist because scanning an array to find something is slow; hashing turns "look through everything" into "jump straight to it."

What problem it solves: Arrays solve the problem of storing a list of items you can iterate. Hash maps solve the problem of "have I seen this before?" or "what value goes with this key?" in constant time on average.

Typical time complexity: Array access by index is O(1). Array search is O(n). Hash map insert, lookup, delete are O(1) average and O(n) worst case.

Typical space complexity: O(n) for both, where n is the number of elements stored.

Real-life analogy: An array is like numbered lockers in a school: you know locker 17, you walk straight to it. A hash map is like a name-tag system: you say "John" and a magical label-reader points to the right locker without checking each one.

Real-world software engineering use cases:

- Hash maps power database indexing — looking up a row by primary key is essentially a hash map lookup.
- Caching layers like Redis and Memcached are giant distributed hash maps.
- Compilers use hash maps to track variable names (symbol tables).
- De-duplication: removing duplicate entries from logs or events uses a hash set.
- Counting frequencies (word counts, request counts per IP) is a hash map use case.

1. Two Sum

Problem Summary

Given an array of integers and a target number, return the indices of the two numbers that add up to the target. Each input has exactly one solution and you cannot use the same element twice.

Real-Life Scenario

A point-of-sale system needs to know if any two items in the cart add up exactly to a discount threshold. Instead of comparing every pair of items, a hash map lets you check it as you scan through the cart once.

Pattern Recognition

Clues that this is the right pattern:

- You see "find a pair that sums to X" or "find two values that combine to a known total."
- You need indices, not just yes or no — that hints at storing positions in a hash map.
- Brute force is obviously two nested loops, which is the trigger to think "can I trade space for time?"

Brute Force Approach

Try every pair (i, j) with i < j. For each pair, check if nums[i] + nums[j] equals target. If yes, return [i, j]. It works because we are literally checking every possible pair, but it is slow because we recheck many pairs.

Time complexity: O(n^2) — two nested loops over the array.

Space complexity: O(1) — only a couple of integer variables.

Optimized Approach

Walk through the array once. For each number num at index i, ask: "have I already seen the value (target - num) earlier?" If yes, we are done — those two indices form the pair.

To answer that question instantly, store each number you have seen so far in a hash map as { value -> index }. Lookup in a hash map is O(1) on average, so the total work becomes O(n).

The trick is: instead of looking for the partner ahead of you, remember every value behind you and let the partner find you when its turn comes.

Time complexity: O(n) — one pass through the array.

Space complexity: O(n) — the hash map can hold up to n entries.

Dry Run

- Input: nums = [2, 7, 11, 15], target = 9. seen = {} (empty map).
- i=0, num=2. complement = 9 - 2 = 7. Is 7 in seen? No. Put 2 -> 0. seen = {2: 0}.
- i=1, num=7. complement = 9 - 7 = 2. Is 2 in seen? Yes, at index 0. Return [0, 1].
- Done.

Code Implementation

Java:

```JAVA
import java.util.HashMap;
import java.util.Map;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        // Map from value to its index
        Map<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            // Have we already seen the partner we need?
            if (seen.containsKey(complement)) {
                return new int[] { seen.get(complement), i };
            }
            // Otherwise, remember this number for later
            seen.put(nums[i], i);
        }
        // Problem says one solution exists, so this is unreachable
        return new int[0];
    }
}
```

Python:

```python
class Solution:
    def twoSum(self, nums, target):
        seen = {}  # value -> index
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i
        return []
```

Common Mistakes

- Storing all numbers in the map first, then looping again. That double pass works, but it can match a number with itself if target is 2 * num and the value appears once.
- Returning the values instead of the indices. Read the problem twice — they want indices.
- Forgetting that the input is not sorted, so you cannot use two pointers without sorting and losing original indices.

Key Takeaways

- Hash map is the go-to weapon when you need O(1) "have I seen this?" lookup.
- Trade space for time: an extra O(n) of memory turned an $O(n^2)$ loop into O(n).
- Single-pass with a map is a pattern you will reuse in many problems.

---

2. Valid Anagram

Problem Summary

Given two strings s and t, return true if t is an anagram of s. Anagram means same letters with the same counts, just rearranged.

Real-Life Scenario

A word puzzle game checks whether the letters a player typed match the target word, ignoring order. The same idea is used in plagiarism detectors that check if two text snippets are reorderings of each other.

Pattern Recognition

Clues that this is the right pattern:

- You see "rearrangement," "permutation of letters," or "same characters, different order."
- Order does not matter, only the multiset of characters matters.
- When character counts are the test, a frequency map (or array of size 26 for lowercase letters) is the natural tool.

Brute Force Approach

Sort both strings and compare. If they are equal after sorting, they are anagrams. It works because anagrams produce the same sorted string. The cost is the sort.

Time complexity: O(n log n) — driven by sorting.

Space complexity: O(n) for the sorted copies (or O(log n) extra in some sort implementations).

Optimized Approach

Count how many times each character appears in s, then subtract counts as you walk through t. If any count goes negative, they cannot be anagrams. If lengths differ, return false immediately.

For lowercase English letters you only need an array of size 26, which is O(1) extra space.

You make one pass to add and another to subtract, then a final check, so you only touch each character a constant number of times.

Time complexity: O(n) — single pass.

Space complexity: O(1) for fixed alphabet, O(k) where k is alphabet size in general.

Dry Run

- Input: s = "anagram", t = "nagaram". Both length 7.
- count = [0]*26. After scanning s: a=3, n=1, g=1, r=1, m=1.
- Scan t and decrement: n -> 0, a -> 2, g -> 0, a -> 1, r -> 0, a -> 0, m -> 0.
- No count went negative. All counts end at 0. Return true.

Code Implementation

Java:

```java
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }
        for (int c : count) {
            if (c != 0) return false;
        }
        return true;
    }
}
```

Python:

```python
class Solution:
    def isAnagram(self, s, t):
        if len(s) != len(t):
            return False
        count = [0] * 26
        for cs, ct in zip(s, t):
            count[ord(cs) - ord('a')] += 1
            count[ord(ct) - ord('a')] -= 1
        return all(c == 0 for c in count)
```

Common Mistakes

- Forgetting the length check. Different lengths can never be anagrams.
- Using a hash map when a 26-size array works — slower in practice and unnecessary.
- Assuming Unicode is the same as ASCII. For Unicode, use a hash map keyed by character.

Key Takeaways

- Frequency counting is the canonical approach for "same characters" problems.
- Fixed alphabet + frequency array = O(1) extra space.
- Always check obvious early-exit conditions like length.

---

3. Contains Duplicate

Problem Summary

Given an array of integers, return true if any value appears at least twice; return false if every element is distinct.

Real-Life Scenario

A sign-up service must check if an email is already in use before creating a new account. The same idea applies to checking duplicate transaction IDs in a payments system.

Pattern Recognition

Clues that this is the right pattern:

- You see "duplicate," "appears more than once," or "all unique."
- Anytime "have I seen this exact value before?" is the core question, a hash set is the fit.

Brute Force Approach

Compare every pair. If any two indices have equal values, return true. It works but rechecks the same pairs repeatedly.

Time complexity: $O(n^2)$.

Space complexity: O(1).

Optimized Approach

Walk through the array once. For each element, ask the hash set "have I added this before?" If yes, return true. Otherwise add it. If you finish the loop, return false.

A hash set is just a hash map without values — it tells you membership in O(1) average time.

Time complexity: O(n).

Space complexity: O(n) for the set.

Dry Run

- Input: [1, 2, 3, 1]. seen = {}.
- 1 not in seen, add it. seen = {1}.
- 2 not in seen, add it. seen = {1, 2}.
- 3 not in seen, add it. seen = {1, 2, 3}.
- 1 IS in seen. Return true.

Code Implementation

Java:

```JAVA
import java.util.HashSet;
import java.util.Set;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int num : nums) {
            if (!seen.add(num)) {
                // add() returns false if the value was already there
                return true;
            }
        }
        return false;
    }
}
```

Python:

```PYTHON
class Solution:
    def containsDuplicate(self, nums):
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False
```

Common Mistakes

- Sorting then scanning is O(n log n) and modifies the input — works, but slower and not always allowed.
- Using a list instead of a set makes the lookup O(n), which silently turns the solution back into $O(n^2)$.

Key Takeaways

- Hash set is the cleanest tool for membership checks.
- set.add() returning false (Java) or "in" before insert (Python) are both clean idioms.

---

4. Group Anagrams

Problem Summary

Given an array of strings, group together all strings that are anagrams of each other. Return the groups in any order.

Real-Life Scenario

A search engine groups query variants ("listen" and "silent") to a single canonical bucket so that synonyms hit the same set of indexed documents.

Pattern Recognition

Clues that this is the right pattern:

- Multiple inputs need to be grouped by some computable signature.
- When two inputs share the signature, they belong to the same bucket — that is a hash map keyed by signature.

Brute Force Approach

For every string, compare it to every other string using the anagram check. If they match, put them in the same group. This re-runs the anagram check for every pair.

Time complexity: $O(n^2 \cdot k)$ where k is the average string length.

Space complexity: $O(n \cdot k)$ for the result.

Optimized Approach

For each string, build a signature that is identical for all anagrams. Two natural signatures: the sorted string (e.g. "eat" -> "aet") or the 26-character count string ("a1e1t1...").

Use a hash map { signature -> list of strings }. Walk through every input once; append each string to its signature's list.

At the end, return the values of the map.

Time complexity: $O(n \cdot k \log k)$ using sorted signature, or $O(n \cdot k)$ using count signature.

Space complexity: $O(n \cdot k)$ for the groups.

Dry Run

- Input: ["eat", "tea", "tan", "ate", "nat", "bat"].
- "eat" -> sorted "aet". groups = {aet: [eat]}.
- "tea" -> "aet". groups = {aet: [eat, tea]}.
- "tan" -> "ant". groups = {aet: [eat, tea], ant: [tan]}.
- "ate" -> "aet". groups = {aet: [eat, tea, ate], ant: [tan]}.
- "nat" -> "ant". groups = {aet: [eat, tea, ate], ant: [tan, nat]}.
- "bat" -> "abt". groups = {..., abt: [bat]}.
- Return the lists: [[eat, tea, ate], [tan, nat], [bat]].

Code Implementation

Java:

```JAVA
import java.util.*;

class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> groups = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
```

```
            groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(groups.values());
    }
}
```

Python:

```
PYTHON
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs):
        groups = defaultdict(list)
        for s in strs:
            key = ''.join(sorted(s))
            groups[key].append(s)
        return list(groups.values())
```

Common Mistakes

- Using the original string as the key — that only groups identical strings, not anagrams.
- Forgetting to convert the map values back to a list before returning.

Key Takeaways

- When a problem says "group by some property," a hash map keyed by that property is almost always the answer.
- The signature you choose decides the time complexity. Pick the cheapest signature that still uniquely identifies the group.

---

5. Top K Frequent Elements

Problem Summary

Given an integer array and an integer k, return the k most frequent elements. You can return them in any order.

Real-Life Scenario

A trending hashtags service must report the top 10 hashtags from the last hour out of millions of posts. The same shape solves "top searched products" or "top-streamed songs today."

Pattern Recognition

Clues that this is the right pattern:

- You need the top-K of something — that is the canonical heap or bucket-sort cue.
- Frequency comes up, which means a hash map of counts is step one.

Brute Force Approach

Count frequencies with a hash map, sort the entries by frequency descending, and take the first k. It works but you do more sorting than you need.

Time complexity: O(n log n) for the sort.

Space complexity: O(n).

Optimized Approach

Step 1: count frequencies using a hash map.

Step 2 (bucket sort approach): create an array of empty lists, where bucket[f] holds all numbers with frequency f. Since frequency is at most n, the array has size n + 1.

Step 3: walk the bucket array from the highest index downward and collect numbers until you have k of them.

This avoids sorting entirely and gives true O(n) time.

Time complexity: O(n).

Space complexity: O(n).

Dry Run

- Input: nums = [1, 1, 1, 2, 2, 3], k = 2.
- Counts: {1: 3, 2: 2, 3: 1}.
- Buckets (index = frequency): bucket[3] = [1], bucket[2] = [2], bucket[1] = [3].
- Walk buckets from index 6 down to 1. Pick from bucket[3]: result = [1]. Pick from bucket[2]: result = [1, 2]. We have k=2, stop.
- Return [1, 2].

Code Implementation

Java:

```JAVA
import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int num : nums) {
            count.merge(num, 1, Integer::sum);
        }
        // bucket[i] = list of numbers that appear i times
        List<Integer>[] bucket = new List[nums.length + 1];
        for (Map.Entry<Integer, Integer> e : count.entrySet()) {
            int freq = e.getValue();
            if (bucket[freq] == null) bucket[freq] = new ArrayList<>();
            bucket[freq].add(e.getKey());
        }
        int[] result = new int[k];
        int idx = 0;
        for (int f = bucket.length - 1; f >= 1 && idx < k; f--) {
            if (bucket[f] == null) continue;
            for (int num : bucket[f]) {
                if (idx == k) break;
                result[idx++] = num;
            }
        }
```

```
        return result;
    }
}
```

Python:

```
PYTHON
from collections import Counter

class Solution:
    def topKFrequent(self, nums, k):
        count = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in count.items():
            buckets[freq].append(num)
        result = []
        for f in range(len(buckets) - 1, 0, -1):
            for num in buckets[f]:
                result.append(num)
                if len(result) == k:
                    return result
        return result
```

Common Mistakes

- Using a min-heap of size k is also O(n log k) and is fine, but bucket sort is strictly faster here.
- Sizing the bucket array as nums.length instead of nums.length + 1 — index out of bounds when one element has frequency n.

Key Takeaways

- "Top K" by frequency has two clean solutions: a min-heap of size k (O(n log k)) or bucket sort (O(n)).
- When the range of a key is bounded (here: frequency is bounded by n), bucket sort beats general sorting.

---

6. Product of Array Except Self

Problem Summary

Given an integer array nums, return an array answer where answer[i] is the product of all elements of nums except nums[i]. Solve it without using division and in O(n).

Real-Life Scenario

In a shopping cart, you want to show the discount that applies if a particular item is removed — the value depends on the product of every other item in the cart.

Pattern Recognition

Clues that this is the right pattern:

- You see "for each index, compute something based on everything except that index."

- Division is forbidden or unsafe (zeros), so you need a prefix and suffix construction.

Brute Force Approach

For each index i, multiply all other elements in a separate inner loop. Works directly but is slow.

Time complexity: $O(n^2)$.

Space complexity: O(1) extra.

Optimized Approach

Idea: answer[i] = (product of everything left of i) * (product of everything right of i).

First pass left to right: build answer[] where answer[i] holds the product of elements strictly to the left of i. answer[0] = 1.

Second pass right to left: keep a running variable rightProduct, starting at 1. For each i from n-1 down to 0, multiply answer[i] *= rightProduct, then update rightProduct *= nums[i].

This uses only the output array as workspace, so no extra O(n) buffer is needed.

Time complexity: O(n).

Space complexity: O(1) extra (the output array does not count toward extra space).

Dry Run

- Input: [1, 2, 3, 4].
- Left pass: answer[0]=1, answer[1]=1, answer[2]=1*2=2, answer[3]=2*3=6. answer = [1, 1, 2, 6].
- Right pass: rightProduct=1.
- i=3: answer[3] = 6 * 1 = 6. rightProduct = 1 * 4 = 4.
- i=2: answer[2] = 2 * 4 = 8. rightProduct = 4 * 3 = 12.
- i=1: answer[1] = 1 * 12 = 12. rightProduct = 12 * 2 = 24.
- i=0: answer[0] = 1 * 24 = 24. rightProduct = 24 * 1 = 24.
- Final: [24, 12, 8, 6].

Code Implementation

Java:

```JAVA
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] answer = new int[n];
        // Left products
        answer[0] = 1;
        for (int i = 1; i < n; i++) {
            answer[i] = answer[i - 1] * nums[i - 1];
        }
        // Multiply by right products on the fly
        int right = 1;
        for (int i = n - 1; i >= 0; i--) {
            answer[i] *= right;
            right *= nums[i];
        }
        return answer;
    }
```

```
}
```

Python:

```
PYTHON
class Solution:
    def productExceptSelf(self, nums):
        n = len(nums)
        answer = [1] * n
        for i in range(1, n):
            answer[i] = answer[i - 1] * nums[i - 1]
        right = 1
        for i in range(n - 1, -1, -1):
            answer[i] *= right
            right *= nums[i]
        return answer
```

Common Mistakes

- Using division — fails when the array contains a zero.
- Forgetting to initialize answer[0] = 1 in the left pass.
- Using two extra arrays for left and right products — works, but uses O(n) extra space when O(1) is achievable.

Key Takeaways

- Prefix-product and suffix-product is a reusable trick. The same skeleton works for prefix sums.
- Reuse the output array to avoid extra space. This kind of trick shows up in many array problems.

---

7. Valid Sudoku

Problem Summary

Given a 9x9 Sudoku board (partially filled), determine if the current state is valid. A board is valid if each row, each column, and each of the nine 3x3 sub-boxes contains the digits 1-9 with no repetition. Empty cells are represented by ".".

Real-Life Scenario

A puzzle game backend validates a player's in-progress board after every move so it can show a red highlight as soon as a rule is violated.

Pattern Recognition

Clues that this is the right pattern:

- You need to check three independent uniqueness constraints (row, col, sub-box) on a grid.
- Whenever you must check "no duplicates in this group," a hash set per group is the fit.

Brute Force Approach

For every row, every column, every sub-box, scan all cells in that group and check duplicates with a fresh set. Many passes over the same board.

Time complexity: O(81) which is constant, but with 27 groups and 9 cells each. Practically slow with three separate passes.

Space complexity: O(1).

Optimized Approach

Maintain three sets for every row, every column, and every sub-box, all updated in a single pass over the board.

Use one set keyed by string like "row3-5" meaning "digit 5 in row 3," and similarly for columns and boxes. The sub-box index is row/3 * 3 + col/3.

For each filled cell, check all three keys. If any key already exists in the set, the board is invalid. Otherwise insert all three.

Time complexity: O(1) — board is always 9x9.

Space complexity: O(1) — at most 81 entries per set.

Dry Run

- Walk through cells. At cell (0, 0) = "5": insert "row0-5", "col0-5", "box0-5". All new. Continue.
- At cell (0, 1) = "3": insert "row0-3", "col1-3", "box0-3". All new. Continue.
- If later we hit (0, 5) = "5", we attempt to insert "row0-5" again. It already exists. Return false.

Code Implementation

Java:

```JAVA
import java.util.HashSet;
import java.util.Set;

class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set<String> seen = new HashSet<>();
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                char ch = board[r][c];
                if (ch == '.') continue;
                int box = (r / 3) * 3 + (c / 3);
                if (!seen.add("R" + r + ch) ||
                    !seen.add("C" + c + ch) ||
                    !seen.add("B" + box + ch)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Python:

```PYTHON
class Solution:
```

```
    def isValidSudoku(self, board):
        seen = set()
        for r in range(9):
            for c in range(9):
                ch = board[r][c]
                if ch == '.':
                    continue
                box = (r // 3) * 3 + (c // 3)
                keys = (f"R{r}-{ch}", f"C{c}-{ch}", f"B{box}-{ch}")
                for k in keys:
                    if k in seen:
                        return False
                    seen.add(k)
        return True
```

Common Mistakes

- Mixing up the sub-box index formula. It is (r/3)*3 + (c/3), using integer division.
- Skipping the empty-cell guard (".") and inserting "." into the set, which falsely flags duplicates.

Key Takeaways

- Tag your keys with a prefix to keep multiple group types in one set.
- Single-pass with a unified set is cleaner than three separate passes.

---

8. Encode and Decode Strings

Problem Summary

Design an algorithm to encode a list of strings into a single string, and decode the single string back into the original list. The strings can contain any character, including delimiters.

Real-Life Scenario

A network protocol must serialize multiple messages into one byte stream and let the receiver split them back. This is essentially how length-prefixed framing works in TCP-based protocols.

Pattern Recognition

Clues that this is the right pattern:

- You need to pack variable-length items into a flat string and recover them.
- You cannot rely on a delimiter that might appear inside the data — that hints at a length prefix.

Brute Force Approach

Use a delimiter like "#" between strings. Fails if any string contains "#". Even rare characters can appear in real data, so this is unreliable.

Time complexity: O(N) but incorrect.

Space complexity: O(N).

Optimized Approach

Encode each string as "<length>#<string>". For example, "abc" becomes "3#abc" and "" becomes "0#".

To decode, repeatedly read digits until "#", parse the length L, take the next L characters as the string, and continue.

Because we trust the length prefix instead of the content, the actual string can contain any character including "#" and digits.

Time complexity: O(N) where N is total characters across all strings.

Space complexity: O(N).

Dry Run

- Encode ["leet", "code"] -> "4#leet4#code".
- Decode "4#leet4#code": pointer = 0.
- Read digits until "#": "4". length = 4. Move past "#". Take next 4 chars: "leet". Add to result. Pointer at 6.
- Read digits until "#": "4". length = 4. Move past "#". Take next 4 chars: "code". Add to result. Pointer at 12.
- End of string. Return ["leet", "code"].

Code Implementation

Java:

```JAVA
import java.util.*;

public class Codec {
    public String encode(List<String> strs) {
        StringBuilder sb = new StringBuilder();
        for (String s : strs) {
            sb.append(s.length()).append('#').append(s);
        }
        return sb.toString();
    }

    public List<String> decode(String s) {
        List<String> result = new ArrayList<>();
        int i = 0;
        while (i < s.length()) {
            int j = i;
            while (s.charAt(j) != '#') j++;
            int len = Integer.parseInt(s.substring(i, j));
            result.add(s.substring(j + 1, j + 1 + len));
            i = j + 1 + len;
        }
        return result;
    }
}
```

Python:

```PYTHON
class Codec:
    def encode(self, strs):
        return ''.join(f"{len(s)}#{s}" for s in strs)
```

```
    def decode(self, s):
        result, i = [], 0
        while i < len(s):
            j = i
            while s[j] != '#':
                j += 1
            length = int(s[i:j])
            result.append(s[j + 1: j + 1 + length])
            i = j + 1 + length
        return result
```

Common Mistakes

- Using a single character delimiter and assuming the data will never contain it.
- Forgetting to handle empty strings (length 0 still produces "0#").

Key Takeaways

- Length-prefixing is the universal way to frame variable-length records. It is used in network protocols, file formats, and binary serialization.
- Never trust the data to avoid your delimiter.

---

9. Longest Consecutive Sequence

Problem Summary

Given an unsorted array of integers, return the length of the longest run of consecutive integers (like 4, 5, 6, 7). Solve it in O(n).

Real-Life Scenario

A streaming watch-time tool finds the longest run of consecutive days a user logged in, given an unsorted set of login dates.

Pattern Recognition

Clues that this is the right pattern:

- You see "longest consecutive run" or "longest streak."
- Sorting would solve it in O(n log n), but the problem demands O(n) — that points to hashing.

Brute Force Approach

Sort the array, then scan and count consecutive runs. Works, but O(n log n).

Time complexity: O(n log n).

Space complexity: O(1) extra (or O(n) depending on sort).

Optimized Approach

Put every number in a hash set for O(1) lookup.

For each number num, check if (num - 1) is in the set. If yes, num is not the start of a run, skip. If no, num IS the start. Count how long the run goes by checking num+1, num+2, ... in the set.

Since each number is the start of at most one run and is only counted once during a run, the total work is O(n).

Time complexity: O(n).

Space complexity: O(n) for the set.

Dry Run

- Input: [100, 4, 200, 1, 3, 2]. set = {100, 4, 200, 1, 3, 2}.
- num=100: is 99 in set? No. Start of run. Check 101? No. Length = 1. best = 1.
- num=4: is 3 in set? Yes. Skip.
- num=200: is 199 in set? No. Start of run. Length 1. best stays 1.
- num=1: is 0 in set? No. Start of run. Check 2 (yes), 3 (yes), 4 (yes), 5 (no). Length = 4. best = 4.
- num=3, num=2: each has predecessor in the set, skip.
- Return 4.

Code Implementation

Java:

```JAVA
import java.util.HashSet;
import java.util.Set;

class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int n : nums) set.add(n);

        int best = 0;
        for (int n : set) {
            // Only start counting from the smallest number of a run
            if (!set.contains(n - 1)) {
                int current = n;
                int length = 1;
                while (set.contains(current + 1)) {
                    current++;
                    length++;
                }
                best = Math.max(best, length);
            }
        }
        return best;
    }
}
```

Python:

```PYTHON
class Solution:
    def longestConsecutive(self, nums):
        s = set(nums)
        best = 0
        for n in s:
```

```
            if n - 1 not in s:
                current, length = n, 1
                while current + 1 in s:
                    current += 1
                    length += 1
                best = max(best, length)
        return best
```

Common Mistakes

- Iterating over the array (not the set) without skipping duplicates — still correct but wastes work.
- Forgetting the "predecessor not in set" check, which turns the algorithm back into $O(n^2)$.

Key Takeaways

- Only start counting from a run's smallest element. That single guard is what keeps the algorithm linear.
- Hash set converts ordering questions into O(1) presence checks.

---

3. Two Pointers

Topic Introduction

What it is: Two pointers means using two indices that move through an array (or string) under some rule. They might start at the two ends and move inward, or both start at the beginning and move at different speeds.

Why it exists: When a problem involves a sorted array or palindrome-like comparison, scanning two values at once is often enough to solve it without nested loops.

What problem it solves: Pair-finding in a sorted array, palindrome checks, in-place transformations, and "trim from both ends" problems.

Typical time complexity: Usually O(n) — each pointer crosses the array once.

Typical space complexity: O(1) — pointers are constant extra memory.

Real-life analogy: Two friends checking if a string is a palindrome by reading from each end and walking toward the middle, comparing the letters they see.

Real-world software engineering use cases:

- Database join algorithms on sorted indexes use two-pointer merging.
- Stream merge: combining two sorted log files into one sorted output.
- In-place buffer compaction in low-level memory systems.
- Subsequence matching in text editors and diff tools.

10. Valid Palindrome

Problem Summary

Given a string, return true if it reads the same forward and backward after converting to lowercase and ignoring all non-alphanumeric characters.

Real-Life Scenario

A spam filter normalizes user submissions and checks for symmetric patterns that bots often produce. The same logic is used in DNA matching and reversible-string detection.

Pattern Recognition

Clues that this is the right pattern:

- You see "palindrome," "reads the same forwards and backwards," or "mirror."
- The natural fit is a left pointer at 0 and a right pointer at length-1, walking inward.

Brute Force Approach

Filter out non-alphanumeric characters into a new string, lowercase it, then compare to its reverse. Works fine, just uses extra memory.

Time complexity: O(n).

Space complexity: O(n) for the cleaned copy.

Optimized Approach

Use two pointers, left at 0 and right at the last index.

Skip non-alphanumeric characters by advancing left or right past them.

Compare the lowercased characters at left and right. If they differ, return false.

Otherwise move both pointers one step closer to the center, repeating until they cross.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- Input: "A man, a plan, a canal: Panama".
- left=0 ('A'), right=29 ('a'). Lowercased a == a. Move both.
- Skip non-alphanumeric on either side as needed (commas, colons, spaces).
- Continue until pointers cross. All comparisons match. Return true.

Code Implementation

Java:

```JAVA
class Solution {
    public boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
            if (Character.toLowerCase(s.charAt(left))
                != Character.toLowerCase(s.charAt(right))) {
                return false;
            }
            left++;
```

```
            right--;
        }
        return true;
    }
}
```

Python:

```
PYTHON
class Solution:
    def isPalindrome(self, s):
        left, right = 0, len(s) - 1
        while left < right:
            while left < right and not s[left].isalnum():
                left += 1
            while left < right and not s[right].isalnum():
                right -= 1
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

Common Mistakes

- Forgetting the inner while-loops to skip junk characters.
- Forgetting to lowercase before comparing.
- Missing the "left < right" guard inside the skip loops, which can move the pointer past the other one.

Key Takeaways

- Two pointers from opposite ends is the canonical palindrome technique.
- Always check pointer order before dereferencing.

---

11. 3Sum

Problem Summary

Given an integer array nums, return all unique triplets [a, b, c] such that a + b + c = 0. The solution set must not contain duplicate triplets.

Real-Life Scenario

A finance system finds combinations of three transactions that net to zero — useful for detecting wash trades or self-cancelling activity.

Pattern Recognition

Clues that this is the right pattern:

- You see "three numbers sum to a target."

- Two nested loops becomes an O(n²) solution if you fix one number and apply two-pointer on the rest. The "sort + two pointers" combo is the standard recipe.

Brute Force Approach

Three nested loops over every triplet. To avoid duplicates, store each found triplet in a set after sorting it.

Time complexity: O(n³).

Space complexity: O(n) for the de-dup set.

Optimized Approach

Sort the array first. Sorting lets duplicates sit next to each other and lets two-pointer search work.

For each index i, fix nums[i] as the first number. Then use two pointers, left = i+1 and right = n-1, to find pairs whose sum equals -nums[i].

If the sum is too small, move left right; if too large, move right left. If equal, record the triplet, then move BOTH pointers and skip duplicates of the values you just used.

Also skip duplicates of nums[i] in the outer loop. Stop the outer loop once nums[i] > 0 because the smallest possible triplet sum becomes positive.

Time complexity: O(n²) — outer loop n times, inner two-pointer scan n.

Space complexity: O(1) extra (or O(log n) for the sort, ignoring the output).

Dry Run

- Input: [-1, 0, 1, 2, -1, -4]. Sort -> [-4, -1, -1, 0, 1, 2].
- i=0, nums[i]=-4. Need pair summing to 4. left=1, right=5. -1+2=1, too small, left++. -1+2=1, left++. 0+2=2, left++. 1+2=3, left++. left>=right. No triplet.
- i=1, nums[i]=-1. Need pair summing to 1. left=2, right=5. -1+2=1, match -> triplet [-1,-1,2]. Move both, skip duplicates. left=3, right=4. 0+1=1, match -> triplet [-1,0,1]. Move both. left=right, stop.
- i=2, nums[i]=-1, same as previous, skip.
- i=3, nums[i]=0. left=4, right=5. 1+2=3, too big, right--. left>=right. Stop.
- i=4, nums[i]=1, but 1 > 0, stop the outer loop early.
- Result: [[-1,-1,2], [-1,0,1]].

Code Implementation

Java:

```JAVA
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < nums.length - 2; i++) {
            if (nums[i] > 0) break;
            if (i > 0 && nums[i] == nums[i - 1]) continue; // skip dup
            int left = i + 1, right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
```

```
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    left++;
                    right--;
                    while (left < right && nums[left] == nums[left - 1]) left++;
                    while (left < right && nums[right] == nums[right + 1]) right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

Python:

```
PYTHON
class Solution:
    def threeSum(self, nums):
        nums.sort()
        result = []
        n = len(nums)
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            left, right = i + 1, n - 1
            while left < right:
                s = nums[i] + nums[left] + nums[right]
                if s == 0:
                    result.append([nums[i], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
                elif s < 0:
                    left += 1
                else:
                    right -= 1
        return result
```

Common Mistakes

- Forgetting to skip duplicate values for i, left, and right — leads to repeated triplets.
- Not sorting first — without sorting, two-pointer logic breaks down.
- Comparing against a target other than 0 — fine, just remember to update the comparison.

Key Takeaways

- "Sort + two pointers" is the standard recipe for k-sum problems with k = 3.
- For 4-sum, wrap the same idea in another outer loop, giving O(n^3).
- Always handle duplicates explicitly when sorting was used to enable the algorithm.

---

12. Container With Most Water

Problem Summary

You are given an array of non-negative integers representing the heights of vertical lines on a coordinate plane. Find two lines that together with the x-axis form a container that holds the most water. Return the maximum amount of water it can hold.

Real-Life Scenario

A stadium event planner picks two seating sections to install a temporary banner stretched between them. The "water" is the visible banner area, bounded by the shorter of the two sections.

Pattern Recognition

Clues that this is the right pattern:

- You see "max area between two lines/values" with one input array.
- It is an optimization problem on a sorted-by-index but unsorted-by-value structure — two pointers from the ends is a strong candidate.

Brute Force Approach

Try every pair (i, j) and compute area = min(height[i], height[j]) * (j - i). Track the maximum.

Time complexity: O(n^2).

Space complexity: O(1).

Optimized Approach

Place pointers left = 0 and right = n - 1. The current width is right - left, which is the maximum possible width.

The area is bounded by the shorter line. To possibly improve area, you must replace the shorter line with a hopefully taller one. So move the shorter side's pointer inward.

Why this is correct: moving the taller side inward can never improve the answer for the current pair — width shrinks and the shorter side still caps the height.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- Input: [1, 8, 6, 2, 5, 4, 8, 3, 7]. left=0 (1), right=8 (7). Area = min(1,7)*8 = 8. left side shorter, move left.
- left=1 (8), right=8 (7). Area = min(8,7)*7 = 49. right shorter, move right.
- left=1 (8), right=7 (3). Area = min(8,3)*6 = 18. right shorter, move right.
- left=1 (8), right=6 (8). Area = min(8,8)*5 = 40. equal, move either; move right.
- ...continue moving inward, never beating 49.
- Return 49.

Code Implementation

Java:

```java
class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1;
        int best = 0;
        while (left < right) {
            int h = Math.min(height[left], height[right]);
            best = Math.max(best, h * (right - left));
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return best;
    }
}
```

Python:

```python
class Solution:
    def maxArea(self, height):
        left, right = 0, len(height) - 1
        best = 0
        while left < right:
            h = min(height[left], height[right])
            best = max(best, h * (right - left))
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return best
```

Common Mistakes

- Always moving the same pointer (e.g., always left) — misses the optimal pair.
- Confusing this with the "trapping rain water" problem, which has a different structure.

Key Takeaways

- Two pointers from opposite ends, moving the weaker side, is the right move when the answer is bounded by a min/max of two endpoints.
- The proof of correctness — "moving the taller side cannot help" — is the kind of insight interviewers love.

---

4. Sliding Window

Topic Introduction

What it is: Sliding window is a technique where you maintain a contiguous range [left, right] over an array or string, and grow or shrink it depending on a condition.

Why it exists: Many problems ask about subarrays or substrings with some property (max sum of size k, longest with no repeats, etc.). Recomputing from scratch for every window is wasteful — the window technique reuses the previous window's state.

What problem it solves: Subarray or substring problems with a "best length / count / sum / max / min" goal under a constraint, where elements enter and leave the window in order.

Typical time complexity: Usually O(n) — each index enters and leaves the window at most once.

Typical space complexity: O(k) for whatever auxiliary structure (set, map, counter) tracks the window contents.

Real-life analogy: Imagine a flashlight beam moving along a long shelf. It illuminates a fixed or variable range of items. As you slide it forward, you only update the items that just entered or just left the beam — you never re-examine the items in the middle.

Real-world software engineering use cases:

- Live analytics dashboards computing rolling averages over the last N events.
- Network traffic shaping using rate limits over a moving time window.
- Real-time anomaly detection over the last few minutes of metrics.
- Streaming top-K queries over recent events.

13. Best Time to Buy and Sell Stock

Problem Summary

Given an array of stock prices where prices[i] is the price on day i, find the maximum profit from a single buy and a single sell. You must buy before you sell. If no profit is possible, return 0.

Real-Life Scenario

A simple trading bot finds the single best buy/sell pair given historical daily prices.

Pattern Recognition

Clues that this is the right pattern:

- You see "max profit from one buy and one sell."
- It is essentially "find the largest j - i such that arr[j] > arr[i] and report arr[j] - arr[i]."
- Single-pass with a running minimum is the lightweight version of sliding window for this problem.

Brute Force Approach

For every pair (i, j) with i < j, compute prices[j] - prices[i]. Track the maximum.

Time complexity: $O(n^2)$.

Space complexity: O(1).

Optimized Approach

Walk the array once. Keep the lowest price seen so far in a variable minPrice.

For each price, the best profit if you sold today is price - minPrice. Update the global maximum profit if it is bigger.

Then update minPrice = min(minPrice, price) for tomorrow.

Return max profit, which stays at 0 if prices only decrease.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- Input: [7, 1, 5, 3, 6, 4]. minPrice = +inf, maxProfit = 0.
- price=7: profit if sold = -inf. maxProfit stays 0. minPrice = 7.
- price=1: profit = 1-7 = -6. maxProfit stays 0. minPrice = 1.
- price=5: profit = 5-1 = 4. maxProfit = 4. minPrice still 1.
- price=3: profit = 3-1 = 2. maxProfit stays 4.
- price=6: profit = 6-1 = 5. maxProfit = 5.
- price=4: profit = 4-1 = 3. maxProfit stays 5.
- Return 5.

Code Implementation

Java:

```JAVA
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }
        return maxProfit;
    }
}
```

Python:

```PYTHON
class Solution:
    def maxProfit(self, prices):
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            if price < min_price:
                min_price = price
            else:
                max_profit = max(max_profit, price - min_price)
```

```
        return max_profit
```

Common Mistakes

- Returning a negative profit when prices only decrease. The problem requires 0 in that case.
- Buying and selling on the same day. The constraint is buy before sell.

Key Takeaways

- "Best so far" with a running minimum is a one-pass O(n) classic.
- Many DP problems have this lightweight one-variable form when only the previous best matters.

---

14. Longest Substring Without Repeating Characters

Problem Summary

Given a string s, find the length of the longest substring that contains no repeating characters.

Real-Life Scenario

A typing-game scoring system finds the longest streak of distinct letters a player typed without re-using any letter.

Pattern Recognition

Clues that this is the right pattern:

- You see "longest substring with [some constraint]."
- The classic variable-size sliding window: grow on the right, shrink on the left when the constraint breaks.

Brute Force Approach

For every starting index i, expand j and check if s[i..j] has all unique characters using a set. Stop when a duplicate appears, record length, move i.

Time complexity: $O(n^2)$ or worse.

Space complexity: O(k) for the set.

Optimized Approach

Use two pointers, left and right, both at 0. Keep a hash set of characters in the current window.

Move right one step at a time. If s[right] is already in the set, shrink from the left: remove s[left] from the set and increment left, until s[right] can join the set.

Add s[right] and update best length = max(best, right - left + 1).

Each character enters and leaves the set at most once, so total work is O(n).

Time complexity: O(n).

Space complexity: O(k) where k is the alphabet size.

Dry Run

- Input: "abcabcbb". left=0, set={}, best=0.
- right=0 ('a'): add. set={a}. best=1.

- right=1 ('b'): add. set={a,b}. best=2.
- right=2 ('c'): add. set={a,b,c}. best=3.
- right=3 ('a'): 'a' is in set. Shrink: remove s[0]='a', left=1, set={b,c}. Now add 'a'. set={b,c,a}. window size 3. best stays 3.
- right=4 ('b'): 'b' in set. Shrink: remove s[1]='b', left=2, set={c,a}. Add 'b'. set={c,a,b}. window size 3. best 3.
- right=5 ('c'): 'c' in set. Shrink: remove s[2]='c', left=3, set={a,b}. Add 'c'. set={a,b,c}. size 3. best 3.
- right=6 ('b'): 'b' in set. Shrink until removed: remove s[3]='a' (set={b,c}), remove s[4]='b' (set={c}), left=5. Add 'b'. set={c,b}. size 2.
- right=7 ('b'): 'b' in set. Shrink: remove s[5]='c' (set={b}), remove s[6]='b' (set={}), left=7. Add 'b'. size 1.
- Return 3.

Code Implementation

Java:

```JAVA
import java.util.HashSet;
import java.util.Set;

class Solution {
    public int lengthOfLongestSubstring(String s) {
        Set<Character> window = new HashSet<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            while (window.contains(c)) {
                window.remove(s.charAt(left));
                left++;
            }
            window.add(c);
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

Python:

```PYTHON
class Solution:
    def lengthOfLongestSubstring(self, s):
        window = set()
        left = 0
        best = 0
        for right, c in enumerate(s):
            while c in window:
                window.remove(s[left])
                left += 1
            window.add(c)
            best = max(best, right - left + 1)
        return best
```

Common Mistakes

- Forgetting to remove characters from the set as left advances.
- Computing length as right - left instead of right - left + 1.
- Using a hash map of last-seen indices and forgetting to clamp left to max(left, lastIndex + 1).

Key Takeaways

- Variable-size sliding window: grow right, shrink left when invariant breaks.
- When the window invariant is "no duplicates," a hash set is the right state.

---

15. Longest Repeating Character Replacement

Problem Summary

Given a string s and an integer k, you can replace any character at most k times. Return the length of the longest substring containing the same letter you can produce after at most k replacements.

Real-Life Scenario

A spell-checker tries to find the longest run that could be "corrected" to a uniform pattern with at most k edits — useful for fuzzy matching.

Pattern Recognition

Clues that this is the right pattern:

- Sliding window with a budget on the number of differences from the dominant element.
- Classic move: track the count of the most frequent character in the window.

Brute Force Approach

Try every substring; for each, count its character frequencies and check if (length - max frequency) <= k. Slow.

Time complexity: $O(n^2 \cdot 26)$.

Space complexity: O(1).

Optimized Approach

Slide a window. Inside the window, let maxFreq be the highest character frequency. The number of replacements needed is (windowLength - maxFreq).

If that number exceeds k, shrink the window from the left. Otherwise the window is valid; record its length.

Tip: maxFreq does not need to be recomputed perfectly when shrinking. The largest maxFreq the algorithm has ever seen is enough — the window can only stay the same length or grow, never shrink to produce a better answer with a smaller maxFreq.

Time complexity: O(n).

Space complexity: O(1) — array of size 26.

Dry Run

- Input: s = "AABABBA", k = 1. count = [0]*26. left=0, maxFreq=0, best=0.
- right=0 ('A'): count[A]=1, maxFreq=1. window length 1, replacements = 0. valid. best=1.
- right=1 ('A'): count[A]=2, maxFreq=2. length 2, replacements 0. best=2.
- right=2 ('B'): count[B]=1, maxFreq still 2. length 3, replacements 1. valid. best=3.
- right=3 ('A'): count[A]=3, maxFreq=3. length 4, replacements 1. valid. best=4.
- right=4 ('B'): count[B]=2, maxFreq still 3. length 5, replacements 2. > k. Shrink: remove s[0]='A', count[A]=2, left=1. Length 4, replacements = 4 - 3 = 1 (using stale maxFreq, OK). best still 4.
- right=5 ('B'): count[B]=3, maxFreq=3. length 5, replacements 2. > k. Shrink: remove s[1]='A', count[A]=1, left=2. length 4, replacements 1. best 4.
- right=6 ('A'): count[A]=2, maxFreq still 3. length 5, replacements 2. > k. Shrink: remove s[2]='B', count[B]=2, left=3. length 4, replacements 1. best 4.
- Return 4.

Code Implementation

Java:

```JAVA
class Solution {
    public int characterReplacement(String s, int k) {
        int[] count = new int[26];
        int left = 0, maxFreq = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            count[s.charAt(right) - 'A']++;
            maxFreq = Math.max(maxFreq, count[s.charAt(right) - 'A']);
            while ((right - left + 1) - maxFreq > k) {
                count[s.charAt(left) - 'A']--;
                left++;
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

Python:

```PYTHON
class Solution:
    def characterReplacement(self, s, k):
        count = [0] * 26
        left = 0
        max_freq = 0
        best = 0
        for right, ch in enumerate(s):
            count[ord(ch) - ord('A')] += 1
            max_freq = max(max_freq, count[ord(ch) - ord('A')])
            while (right - left + 1) - max_freq > k:
                count[ord(s[left]) - ord('A')] -= 1
                left += 1
            best = max(best, right - left + 1)
        return best
```

Common Mistakes

- Recomputing maxFreq on every shrink — works but is O(26n). Not necessary for correctness.
- Using "if" instead of "while" for the shrink — fine here because the invariant breaks by exactly one each time, but unsafe in similar problems.

Key Takeaways

- Sliding window with a "budget" on edits: count the dominant element, compare with window size.
- maxFreq does not need to decrease for the answer to be correct, because the answer only updates when the window grows.

---

16. Minimum Window Substring

Problem Summary

Given strings s and t, find the smallest substring of s that contains every character of t (with multiplicity). Return the empty string if no such window exists.

Real-Life Scenario

A log search system finds the smallest contiguous chunk of a log that contains every required keyword in some order — useful for showing the most compact "evidence" for a query.

Pattern Recognition

Clues that this is the right pattern:

- Sliding window with a "must-contain" constraint counted by character.
- Classic "shrink while still valid, record best" pattern.

Brute Force Approach

Check every substring of s and see if it contains all characters of t. $O(n^2)$ substrings, each check $O(n + m)$.

Time complexity: $O(n^2 \cdot (n + m))$.

Space complexity: $O(m)$.

Optimized Approach

Build need[c] = count of each char in t. Maintain have[c] for the current window.

Track a counter "matched" = number of distinct characters whose count in the window meets or exceeds the need. The window is valid when matched == required (where required is the number of distinct chars in t).

Expand right pointer. When the window is valid, try to shrink from the left to make it smaller while still valid; update best window. Continue until right reaches the end.

Time complexity: $O(n + m)$.

Space complexity: $O(k)$ for the maps.

Dry Run

- Input: s = "ADOBECODEBANC", t = "ABC". need = {A:1, B:1, C:1}, required = 3.

- Expand right until window contains all three. At s[0..5] = "ADOBEC", have all three. matched = 3. record best.
- Shrink left: drop 'A' -> matched=2. Stop shrinking, expand right.
- Continue until s[5..10] = "CODEBA": now contains A,B,C again. shrink to "DEBANC" then "EBANC" then "BANC". "BANC" still valid, length 4 < previous 6. update best.
- Continue. No smaller window. Return "BANC".

Code Implementation

Java:

```
JAVA
import java.util.HashMap;
import java.util.Map;

class Solution {
    public String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";
        Map<Character, Integer> need = new HashMap<>();
        for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

        int required = need.size();
        Map<Character, Integer> have = new HashMap<>();
        int matched = 0;
        int left = 0, bestLen = Integer.MAX_VALUE, bestLeft = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            have.merge(c, 1, Integer::sum);
            if (need.containsKey(c) && have.get(c).intValue() == need.get(c).intValue())
{
                matched++;
            }
            while (matched == required) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestLeft = left;
                }
                char lc = s.charAt(left);
                have.merge(lc, -1, Integer::sum);
                if (need.containsKey(lc) && have.get(lc).intValue() <
need.get(lc).intValue()) {
                    matched--;
                }
                left++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestLeft, bestLeft +
bestLen);
    }
}
```

Python:

```python
from collections import Counter

class Solution:
    def minWindow(self, s, t):
        if len(s) < len(t):
            return ""
        need = Counter(t)
        required = len(need)
        have = {}
        matched = 0
        left = 0
        best_len = float('inf')
        best_left = 0
        for right, c in enumerate(s):
            have[c] = have.get(c, 0) + 1
            if c in need and have[c] == need[c]:
                matched += 1
            while matched == required:
                if right - left + 1 < best_len:
                    best_len = right - left + 1
                    best_left = left
                lc = s[left]
                have[lc] -= 1
                if lc in need and have[lc] < need[lc]:
                    matched -= 1
                left += 1
        return "" if best_len == float('inf') else s[best_left: best_left + best_len]
```

Common Mistakes

- Updating "matched" by checking >= instead of ==. You only increment matched the moment have[c] reaches need[c], not every time it grows.
- Forgetting to handle the case where t is longer than s — return empty string upfront.

Key Takeaways

- "matched == required" is the cleanest way to know the window is valid without re-summing maps each step.
- This pattern (need + have + matched) generalizes to many "contains all" sliding window problems.

---

5. Stack

Topic Introduction

What it is: A stack is a Last-In-First-Out (LIFO) container. The only legal operations are push (add to the top) and pop (remove from the top). You can also peek at the top without removing it.

Why it exists: Many problems are naturally last-in-first-out: matching parentheses, undoing actions, recursive function calls, parsing expressions. A stack reflects that natural order.

What problem it solves: Bracket matching, expression evaluation, monotonic-stack problems (next greater element), recursion-to-iteration conversion, and undo histories.

Typical time complexity: Push, pop, peek are all O(1).

Typical space complexity: O(n) where n is the number of items currently on the stack.

Real-life analogy: A stack of plates in a cafeteria. The last plate placed is the first one taken. The plates in the middle are not directly accessible.

Real-world software engineering use cases:

- Programming languages use a call stack to track function calls.
- Browsers use a stack for the back button history.
- Editors use stacks to implement undo.
- Compilers and interpreters parse expressions using stacks.

17. Valid Parentheses

Problem Summary

Given a string containing only "()[]{}", return true if every opener has a matching closer in the correct order. For example, "([])" is valid; "([)]" is not.

Real-Life Scenario

A code editor highlights mismatched brackets in real time as you type. The exact same logic checks well-formed JSON or XML.

Pattern Recognition

Clues that this is the right pattern:

- You see "matching pairs" or "open and close in correct order."
- LIFO behavior: the most recent opener must match the next closer.

Brute Force Approach

Repeatedly remove inner matching pairs ("()", "[]", "{}") from the string until nothing changes. If the string is empty, it was valid. Slow due to string rebuilding.

Time complexity: $O(n^2)$.

Space complexity: O(n).

Optimized Approach

Walk the string. For every opener push it onto the stack. For every closer, check if the top of the stack is the matching opener — if not, return false; if yes, pop.

At the end the string is valid only if the stack is empty.

Time complexity: O(n).

Space complexity: O(n) in the worst case (all openers).

Dry Run

- Input: "([])". stack = [].

- '(': push. stack = ['('].
- '[': push. stack = ['(', '['].
- ']': top is '['. Match. Pop. stack = ['('].
- ')': top is '('. Match. Pop. stack = [].
- End. Stack empty. Return true.

Code Implementation

Java:

```JAVA
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')' && top != '(') ||
                    (c == ']' && top != '[') ||
                    (c == '}' && top != '{')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

Python:

```PYTHON
class Solution:
    def isValid(self, s):
        stack = []
        pair = {')': '(', ']': '[', '}': '{'}
        for c in s:
            if c in '([{':
                stack.append(c)
            else:
                if not stack or stack[-1] != pair[c]:
                    return False
                stack.pop()
        return not stack
```

Common Mistakes

- Forgetting the empty-stack check before popping — causes a crash on inputs like ")".
- Forgetting to verify the stack is empty at the end — strings like "((((" would be falsely accepted.

Key Takeaways

- Whenever pairs must close in reverse order of opening, a stack is the perfect data structure.
- A common follow-up is to extend the same idea to operator precedence parsing.

---

6. Binary Search

Topic Introduction

What it is: Binary search finds an item in a sorted array by repeatedly cutting the search range in half. Each step compares the middle element to the target and discards the half that cannot contain the answer.

Why it exists: Linear scan is O(n). When the data is sorted (or any monotonic property is being searched), halving the range is exponentially faster.

What problem it solves: Searching a sorted array, finding boundaries (first/last occurrence), or finding the smallest value that satisfies a monotonic predicate.

Typical time complexity: O(log n) per search.

Typical space complexity: O(1) iterative; O(log n) recursive due to call stack.

Real-life analogy: A guess-the-number game. You ask "is it more than 50?" If yes, you cut the range to 51-100. The next guess halves it again. After about log2(N) questions you find the number.

Real-world software engineering use cases:

- Database indexes (B-trees) use binary-search-like operations to look up rows.
- Search bars use binary search on sorted prefix arrays for autocomplete.
- Version control bisect commands use binary search to find the commit that introduced a bug.
- Game engines use binary search inside spatial data structures.

18. Find Minimum in Rotated Sorted Array

Problem Summary

A sorted array of unique numbers has been rotated some unknown number of times (e.g., [4,5,6,7,0,1,2]). Find the minimum element in O(log n).

Real-Life Scenario

A clock-aligned data feed exports a wrapped-around buffer where the oldest entry is somewhere in the middle. Finding the smallest timestamp finds the start of the data.

Pattern Recognition

Clues that this is the right pattern:

- Sorted but rotated array — half the array is still sorted.
- Modified binary search where you decide which half to discard based on which side is sorted.

Brute Force Approach

Linear scan to find the minimum. O(n).

Time complexity: O(n).

Space complexity: O(1).

Optimized Approach

left = 0, right = n - 1. While left < right, compute mid.

If nums[mid] > nums[right], the minimum is in the right half: left = mid + 1.

Otherwise (nums[mid] <= nums[right]), the right half is sorted but the minimum could be at mid itself: right = mid.

Loop ends with left == right, pointing to the minimum.

Time complexity: O(log n).

Space complexity: O(1).

Dry Run

- Input: [4,5,6,7,0,1,2]. left=0, right=6.
- mid=3, nums[mid]=7. 7 > nums[6]=2. Minimum on right. left=4.
- left=4, right=6. mid=5, nums[mid]=1. 1 <= nums[6]=2. right=5.
- left=4, right=5. mid=4, nums[mid]=0. 0 <= nums[5]=1. right=4.
- left==right==4. Return nums[4]=0.

Code Implementation

Java:

```JAVA
class Solution {
    public int findMin(int[] nums) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] > nums[right]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return nums[left];
    }
}
```

Python:

```PYTHON
class Solution:
    def findMin(self, nums):
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1
            else:
```

```
                right = mid
        return nums[left]
```

Common Mistakes

- Comparing nums[mid] to nums[left] instead of nums[right] — leads to wrong half being discarded near sorted segments.
- Using "left <= right" with right = mid — possible infinite loop. The pattern "left < right" with right = mid is safer.

Key Takeaways

- Binary search adapts to monotonic structure even after rotation; identify which half is sorted and discard accordingly.
- Use "left < right" with "right = mid" pattern for boundary-style searches.

---

19. Search in Rotated Sorted Array

Problem Summary

Same setup as the previous problem, but instead of finding the minimum you must return the index of a given target, or -1 if it is not present. Must run in O(log n).

Real-Life Scenario

A circular cache (rotated chronological order) is searched for a specific key without re-sorting it.

Pattern Recognition

Clues that this is the right pattern:

- Rotated sorted array + target search — the canonical binary-search variant.

Brute Force Approach

Linear scan, return the index or -1.

Time complexity: O(n).

Space complexity: O(1).

Optimized Approach

Standard binary search with one extra check per step: identify which half (left or right of mid) is sorted.

If nums[left] <= nums[mid], the left half is sorted. Check if the target lies inside [nums[left], nums[mid]). If yes, search left half; otherwise right.

If the right half is sorted instead, mirror the logic: if target lies in (nums[mid], nums[right]], search right; otherwise left.

Time complexity: O(log n).

Space complexity: O(1).

Dry Run

- Input: nums = [4,5,6,7,0,1,2], target = 0. left=0, right=6.

- mid=3, nums[mid]=7. nums[0]=4 <= 7, so left half is sorted. Is 0 in [4, 7)? No. Search right. left=4.
- left=4, right=6. mid=5, nums[mid]=1. nums[4]=0 <= 1, left half sorted. Is 0 in [0, 1)? Yes. Search left. right=4.
- left=4, right=4. mid=4, nums[mid]=0. Match. Return 4.

Code Implementation

Java:

```java
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            if (nums[left] <= nums[mid]) {
                // Left half is sorted
                if (target >= nums[left] && target < nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            } else {
                // Right half is sorted
                if (target > nums[mid] && target <= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }
        return -1;
    }
}
```

Python:

```python
class Solution:
    def search(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]:
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
```

```
                else:
                    right = mid - 1
        return -1
```

Common Mistakes

- Wrong inequality on the boundary checks — write them out carefully and trace one example.
- Using "<" instead of "<=" when comparing nums[left] and nums[mid] — at array of size 2 the left half is technically sorted by itself.

Key Takeaways

- Recognize "one half is always sorted" — that is the trick that keeps binary search valid in rotated arrays.
- Always include the equality boundary; off-by-one errors are the #1 binary search bug.

---

7. Linked List

Topic Introduction

What it is: A linked list is a chain of nodes, where each node holds a value and a pointer to the next node. The list is accessed only via its head pointer; you cannot jump to the middle.

Why it exists: Inserting or deleting in the middle of an array is O(n) because everything must shift. Linked lists make those operations O(1) once you have a pointer to the right place.

What problem it solves: Constant-time insert/delete in dynamic sequences, building queues and stacks, representing graphs as adjacency lists, implementing LRU caches.

Typical time complexity: Access by index is O(n). Insert/delete given a node is O(1). Search is O(n).

Typical space complexity: O(n) for n nodes; each node holds an extra pointer compared to an array element.

Real-life analogy: A treasure hunt where each clue tells you where the next clue is. You can only follow the chain — you cannot teleport to clue #5.

Real-world software engineering use cases:

- Operating system memory managers use linked lists to track free blocks.
- Music players use linked lists to represent the current playlist with next/previous.
- LRU caches maintain a doubly linked list of recently used items.
- Many graph algorithms store edges as linked lists per vertex.

20. Reverse Linked List

Problem Summary

Given the head of a singly linked list, reverse the list and return the new head.

Real-Life Scenario

Displaying a chronological feed in reverse-chronological order without copying it to an array first.

Pattern Recognition

Clues that this is the right pattern:

- You see "reverse linked list" or "reverse a portion of a linked list."
- Pointer manipulation with two or three pointers (prev, current, next).

Brute Force Approach

Push all values onto a stack, then create a new list popping the stack. Works but uses O(n) extra memory.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Walk the list. Keep three pointers: prev (initially null), curr (initially head), next.

At each step, save next = curr.next, flip curr.next = prev, then move prev = curr and curr = next.

When curr becomes null, prev is the new head.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- Input: 1 -> 2 -> 3 -> null.
- prev=null, curr=1.
- Iter 1: next=2. 1.next=null. prev=1, curr=2.
- Iter 2: next=3. 2.next=1. prev=2, curr=3.
- Iter 3: next=null. 3.next=2. prev=3, curr=null.
- Return prev=3. List is 3 -> 2 -> 1 -> null.

Code Implementation

Java:

```JAVA
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
}
```

Python:

```python
class Solution:
    def reverseList(self, head):
        prev, curr = None, head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        return prev
```

Common Mistakes

- Forgetting to save next before overwriting curr.next — you lose the rest of the list.
- Returning curr instead of prev. After the loop, curr is null.

Key Takeaways

- The three-pointer dance (prev, curr, next) is the foundation of most linked list manipulation.
- Practice this until you can write it without thinking; many harder problems build on it.

---

21. Merge Two Sorted Lists

Problem Summary

Given the heads of two sorted linked lists, merge them into one sorted list and return its head. The new list should reuse the nodes of the original lists.

Real-Life Scenario

Merging two sorted result streams from two database shards into one sorted output stream.

Pattern Recognition

Clues that this is the right pattern:

- Two sorted inputs that must be combined into one sorted output.
- Two pointers walking forward, choosing the smaller value at each step.

Brute Force Approach

Collect all values into an array, sort, then build a new list. Loses the O(n) advantage of already-sorted input.

Time complexity: O(n log n).

Space complexity: O(n).

Optimized Approach

Use a dummy node before the merged list to simplify the head logic. Maintain a tail pointer that always points to the last node of the merged list.

While both lists have nodes, attach the one with the smaller value to tail.next, advance that list, and advance tail.

When one list is exhausted, attach the other list as the rest. Return dummy.next.

Time complexity: O(n + m).

Space complexity: O(1).

Dry Run

- Input: l1 = 1 -> 2 -> 4, l2 = 1 -> 3 -> 4.
- dummy -> ?, tail=dummy. l1=1, l2=1. Equal, attach l1. tail.next=l1(1), tail=1, l1=2.
- l1=2, l2=1. l2 smaller. Attach l2. tail=1(l2), l2=3.
- l1=2, l2=3. l1 smaller. Attach l1. tail=2, l1=4.
- l1=4, l2=3. l2 smaller. Attach l2. tail=3, l2=4.
- l1=4, l2=4. Attach l1. tail=4, l1=null.
- Attach remainder of l2 (which is 4). Done. Return dummy.next.
- Result: 1 -> 1 -> 2 -> 3 -> 4 -> 4.

Code Implementation

Java:

```JAVA
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                tail.next = l1;
                l1 = l1.next;
            } else {
                tail.next = l2;
                l2 = l2.next;
            }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

Python:

```PYTHON
class Solution:
    def mergeTwoLists(self, l1, l2):
        dummy = ListNode(0)
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                tail.next = l1
                l1 = l1.next
            else:
```

```
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 if l1 else l2
        return dummy.next
```

Common Mistakes

- Forgetting the dummy node, leading to special-case handling for the first attachment.
- Not attaching the leftover list at the end.

Key Takeaways

- A dummy/sentinel head is a standard trick to simplify linked list construction.
- The merge step is the same merge used inside merge sort.

---

22. Linked List Cycle

Problem Summary

Given the head of a linked list, return true if there is a cycle (some node's next points back to a previous node). Use O(1) extra space.

Real-Life Scenario

A workflow engine detects circular dependencies between tasks: if you walk the dependency chain forever without reaching a leaf, something points back.

Pattern Recognition

Clues that this is the right pattern:

- You see "detect a cycle" with O(1) memory restriction.
- Two pointers moving at different speeds (Floyd's tortoise and hare).

Brute Force Approach

Walk the list, store each visited node in a hash set. If you see one twice, there is a cycle. Uses O(n) extra space.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Use two pointers: slow advances one step at a time, fast advances two steps at a time.

If the list ends (fast or fast.next becomes null), no cycle.

If they meet, there is a cycle. Why? In a cycle, fast gains one step on slow each iteration, so it must catch up eventually.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- No cycle: 1->2->3->4->null.
- slow=1, fast=1. Move: slow=2, fast=3. Move: slow=3, fast=null. Stop. Return false.
- Cycle: 1->2->3->4->2 (back to 2).
- slow=1, fast=1. Move: slow=2, fast=3. Move: slow=3, fast=2 (after going 4->2). Move: slow=4, fast=4. Match. Return true.

Code Implementation

Java:

```java
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

Python:

```python
class Solution:
    def hasCycle(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False
```

Common Mistakes

- Comparing values instead of references — false positives if duplicate values exist.
- Forgetting to check both fast and fast.next before advancing two steps.

Key Takeaways

- Floyd's cycle detection (slow/fast pointers) is the canonical O(1)-space cycle check.
- The same idea extends to finding the start of the cycle.

---

23. Reorder List

Problem Summary

Given the head of a singly linked list L0 -> L1 -> ... -> Ln-1 -> Ln, reorder it to L0 -> Ln -> L1 -> Ln-1 -> L2 -> Ln-2 -> ... in place.

Real-Life Scenario

A printing service interleaves the start and end of a long document for booklet binding (where the first and last pages must sit on the same physical sheet).

Pattern Recognition

Clues that this is the right pattern:

- Combination of three classic techniques: find middle (slow/fast), reverse second half, then merge.

Brute Force Approach

Copy values to an array, then weave them with two pointers from both ends, rebuilding nodes by re-pointing.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Step 1: find the middle using slow/fast pointers. After the loop, slow is the start of the second half.

Step 2: reverse the second half in place.

Step 3: merge the two halves by alternating, using the standard pointer dance. End the first half's tail with null.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- Input: 1 -> 2 -> 3 -> 4 -> 5.
- Step 1: middle. slow=3 (after the loop using fast.next != null && fast.next.next != null).
- Step 2: reverse 4 -> 5 to get 5 -> 4. Now first half = 1 -> 2 -> 3 -> null, second half = 5 -> 4 -> null.
- Step 3: merge. 1 -> 5 -> 2 -> 4 -> 3 -> null.

Code Implementation

Java:

```JAVA
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;

        // 1. Find middle
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // 2. Reverse second half
        ListNode prev = null, curr = slow.next;
        slow.next = null; // cut
```

```
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        // 3. Merge
        ListNode first = head, second = prev;
        while (second != null) {
            ListNode t1 = first.next, t2 = second.next;
            first.next = second;
            second.next = t1;
            first = t1;
            second = t2;
        }
    }
}
```

Python:

```
PYTHON
class Solution:
    def reorderList(self, head):
        if not head or not head.next:
            return
        # 1. Find middle
        slow = fast = head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next
        # 2. Reverse second half
        prev, curr = None, slow.next
        slow.next = None
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        # 3. Merge
        first, second = head, prev
        while second:
            t1, t2 = first.next, second.next
            first.next = second
            second.next = t1
            first, second = t1, t2
```

Common Mistakes

- Forgetting to cut the first half (slow.next = null) — the merged list ends up with a cycle.
- Mishandling odd vs even length: the slow/fast loop using "fast.next != null && fast.next.next != null" places slow at the middle for odd, and at end-of-first-half for even.

Key Takeaways

- Compose simpler list operations (find middle, reverse, merge) to solve harder list problems.
- Always cut connections you no longer want; a leftover pointer is how cycles sneak in.

---

24. Remove Nth Node From End of List

Problem Summary

Given the head of a linked list, remove the nth node from the end and return the new head.

Real-Life Scenario

A "delete the last N entries" feature in an audit log without first counting the total length.

Pattern Recognition

Clues that this is the right pattern:

- Two-pointer with a fixed gap of n. When the lead pointer hits the end, the trailing one is at the node to remove.

Brute Force Approach

First pass to count length L, second pass to walk to position L - n - 1 and unlink.

Time complexity: O(n).

Space complexity: O(1).

Optimized Approach

Use a dummy node to simplify removing the head.

Move a "fast" pointer n+1 steps ahead from dummy. Then move slow and fast together until fast is null.

At that moment, slow.next is the node to remove. Bypass it: slow.next = slow.next.next.

Return dummy.next.

Time complexity: O(n) — one pass.

Space complexity: O(1).

Dry Run

- Input: 1->2->3->4->5, n=2.
- dummy -> 1->2->3->4->5. fast at dummy. Advance fast 3 times (n+1): fast=3.
- Move slow and fast until fast is null: slow=dummy->1, fast=3->4. slow=1->2, fast=4->5. slow=2->3, fast=5->null.
- Remove slow.next which is 4: 3.next = 5.
- Return dummy.next = 1->2->3->5.

Code Implementation

Java:

```
JAVA
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
```

```
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode slow = dummy, fast = dummy;
        for (int i = 0; i <= n; i++) {
            fast = fast.next;
        }
        while (fast != null) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

Python:

```
PYTHON
class Solution:
    def removeNthFromEnd(self, head, n):
        dummy = ListNode(0, head)
        slow = fast = dummy
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            slow = slow.next
            fast = fast.next
        slow.next = slow.next.next
        return dummy.next
```

Common Mistakes

- Off-by-one when advancing fast — must be n+1 steps from dummy so slow lands on the node BEFORE the target.
- Not using a dummy node — removing the head becomes a special case.

Key Takeaways

- A dummy node simplifies edge cases at the head.
- Two-pointer with a gap is a reusable trick anytime you need "k from the end" in one pass.

---

25. Merge K Sorted Lists

Problem Summary

Given an array of k sorted linked lists, merge them into one sorted linked list and return it.

Real-Life Scenario

A search aggregator merges sorted result streams from k different sources into one ranked list.

Pattern Recognition

Clues that this is the right pattern:

- Generalization of "merge two sorted lists." A min-heap or divide-and-conquer makes it efficient.

Brute Force Approach

Collect all node values into one big array, sort it, build a new list. O(N log N) where N is total nodes.

Time complexity: O(N log N).

Space complexity: O(N).

Optimized Approach

Approach 1 (min-heap): push the head of each non-null list into a min-heap keyed by value. Pop the smallest, append to the merged tail, push its .next if any. Each pop/push is O(log k), and there are N nodes total: O(N log k).

Approach 2 (divide and conquer): pair up lists and merge them two at a time using mergeTwoLists, halving the count each round. Same total cost O(N log k), no heap needed.

Time complexity: O(N log k).

Space complexity: O(k) for the heap, or O(log k) for the recursion in divide-and-conquer.

Dry Run

- Input: lists = [[1,4,5], [1,3,4], [2,6]].
- Heap initially with heads: (1,a), (1,b), (2,c).
- Pop (1,a), append. Push next of a: (4,a). Heap: (1,b), (2,c), (4,a).
- Pop (1,b), append. Push (3,b). Heap: (2,c), (4,a), (3,b).
- Pop (2,c). Push (6,c). ... Continue until heap is empty.
- Final list: 1->1->2->3->4->4->5->6.

Code Implementation

Java:

```JAVA
import java.util.PriorityQueue;

class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode l : lists) {
            if (l != null) heap.offer(l);
        }
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (!heap.isEmpty()) {
            ListNode node = heap.poll();
            tail.next = node;
            tail = node;
            if (node.next != null) heap.offer(node.next);
        }
        return dummy.next;
    }
}
```

Python:

```python
import heapq

class Solution:
    def mergeKLists(self, lists):
        heap = []
        # Python heapq compares tuples; second element breaks ties
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node))
        dummy = ListNode(0)
        tail = dummy
        counter = len(lists)
        while heap:
            val, _, node = heapq.heappop(heap)
            tail.next = node
            tail = node
            if node.next:
                heapq.heappush(heap, (node.next.val, counter, node.next))
                counter += 1
        return dummy.next
```

Common Mistakes

- In Python, forgetting that heapq cannot directly compare ListNode objects with equal values — include a tiebreaker like an index.
- Forgetting to push the next node after popping — list disappears after the head is consumed.

Key Takeaways

- A k-way merge is the model problem for "min-heap of size k" pattern.
- Divide-and-conquer pairwise merge has the same complexity and is a clean alternative.

---

8. Trees (Binary Trees & BSTs)

Topic Introduction

What it is: A tree is a hierarchical data structure where each node has a value and links to child nodes. A binary tree allows up to two children per node (left and right). A binary search tree (BST) adds the rule: left subtree has only smaller values, right subtree has only larger values.

Why it exists: Trees model hierarchies naturally — file systems, organization charts, parsed expressions, decision logic. BSTs add fast ordered search: O(log n) on average.

What problem it solves: Hierarchical data, ordered storage with efficient search/insert, recursive partitioning of a problem space.

Typical time complexity: Tree traversals are O(n). BST search/insert/delete is O(log n) average and O(n) worst case (unbalanced).

Typical space complexity: O(n) for the tree itself plus O(h) for recursion, where h is the height.

Real-life analogy: A family tree. Each person has up to two children. To find someone, you don't need to look at every cousin — you follow the lineage.

Real-world software engineering use cases:

- File systems are tree structures (directories and files).
- HTML/XML documents are parsed into DOM trees.
- Database indexes (B-trees) are tree variants tuned for disk access.
- Decision trees power machine-learning classifiers.
- Compilers represent code as abstract syntax trees.

26. Invert Binary Tree

Problem Summary

Given the root of a binary tree, swap every left and right child throughout the tree and return the root.

Real-Life Scenario

A UI rendering library produces a mirrored layout for right-to-left languages by inverting the layout tree.

Pattern Recognition

Clues that this is the right pattern:

- Tree problem with a uniform operation at every node — natural fit for recursion.

Brute Force Approach

There is no really brute force here — recursion is essentially the only sensible approach. We'll list iterative-with-queue as the alternative.

Time complexity: O(n).

Space complexity: O(n) for the queue or recursion stack.

Optimized Approach

Recursive: at each node, swap its left and right children, then recurse into both subtrees.

Iterative: BFS with a queue. Pop a node, swap its children, enqueue both children.

Time complexity: O(n).

Space complexity: O(h) for recursion (h is height).

Dry Run

- Tree: 4(left=2, right=7), 2(left=1, right=3), 7(left=6, right=9).
- Recurse on 4. Swap children -> 4(left=7, right=2).
- Recurse on 7. Swap -> 7(left=9, right=6). Recurse on 9 (leaf), 6 (leaf): nothing.
- Recurse on 2. Swap -> 2(left=3, right=1). Recurse on 3 (leaf), 1 (leaf): nothing.
- Done. Final shape mirrored.

Code Implementation

Java:

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}

class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode tmp = root.left;
        root.left = invertTree(root.right);
        root.right = invertTree(tmp);
        return root;
    }
}
```

Python:

```python
class Solution:
    def invertTree(self, root):
        if not root:
            return None
        root.left, root.right = self.invertTree(root.right), self.invertTree(root.left)
        return root
```

Common Mistakes

- Overwriting root.left before saving its old value, then losing the original left subtree.
- Forgetting the null check at the top.

Key Takeaways

- Recursion mirrors tree structure: think about what to do at one node, then trust recursion to handle the children.

---

27. Maximum Depth of Binary Tree

Problem Summary

Given the root of a binary tree, return its maximum depth — the number of nodes on the longest path from root to a leaf.

Real-Life Scenario

Computing the deepest reply chain in a comment thread to know the layout indent depth.

Pattern Recognition

Clues that this is the right pattern:

- Recursive aggregation: each node's answer is 1 plus the max of its children's answers.

Brute Force Approach

BFS counting levels. Push root, then process the queue level by level. Same complexity but explicit level tracking.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Recursive: depth(null) = 0; depth(node) = 1 + max(depth(node.left), depth(node.right)).

This naturally computes the longest path because at each node you keep the deeper subtree.

Time complexity: O(n).

Space complexity: O(h) for the call stack.

Dry Run

- Tree: 3(9, 20(15, 7)).
- depth(3) = 1 + max(depth(9), depth(20)).
- depth(9) = 1 + max(0, 0) = 1.
- depth(20) = 1 + max(depth(15), depth(7)) = 1 + max(1, 1) = 2.
- depth(3) = 1 + max(1, 2) = 3.

Code Implementation

Java:

```java
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

Python:

```python
class Solution:
    def maxDepth(self, root):
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

Common Mistakes

- Returning 1 for null instead of 0 — off-by-one in depth.
- Returning sum instead of max — gives total nodes, not depth.

Key Takeaways

- Pure recursion: define base case (null = 0), define recursive step (1 + max of children).

---

28. Same Tree

Problem Summary

Given the roots of two binary trees, return true if they are structurally identical and the corresponding nodes have the same value.

Real-Life Scenario

A diff tool that reports whether two parsed configuration files have the same logical structure and contents.

Pattern Recognition

Clues that this is the right pattern:

- Compare two trees recursively node by node.

Brute Force Approach

No real brute force — direct recursion is already optimal.

Time complexity: O(n).

Space complexity: O(h).

Optimized Approach

Both null -> equal.

One null and the other not -> not equal.

Different values -> not equal.

Otherwise recurse on both pairs of children with AND.

Time complexity: O(n).

Space complexity: O(h).

Dry Run

- p = (1, 2, 3), q = (1, 2, 3). p.val == q.val. Recurse on (2,2): values match, both leaves. Recurse on (3,3): match. Return true overall.

Code Implementation

Java:

```
JAVA
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        if (p.val != q.val) return false;
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

Python:

```
PYTHON
```

```
class Solution:
    def isSameTree(self, p, q):
        if not p and not q:
            return True
        if not p or not q:
            return False
        return p.val == q.val and self.isSameTree(p.left, q.left) and
self.isSameTree(p.right, q.right)
```

Common Mistakes

- Forgetting one of the null cases — comparing null.val crashes.

Key Takeaways

- Two-tree recursion mirrors single-tree recursion; just walk both at once.

---

29. Subtree of Another Tree

Problem Summary

Given roots of two binary trees, root and subRoot, return true if there is a subtree of root with the exact same structure and values as subRoot.

Real-Life Scenario

Detecting whether a small UI component tree appears as a sub-tree within a larger app component tree (used in template-based rendering checks).

Pattern Recognition

Clues that this is the right pattern:

- Reuse "Same Tree" as a helper, called recursively on every node of the bigger tree.

Brute Force Approach

For every node in root, run the same-tree check against subRoot. If any matches, return true.

Time complexity: O(m · n) where m and n are the sizes.

Space complexity: O(h).

Optimized Approach

The brute force is acceptable. A faster approach uses serialization with unique markers and substring matching (O(m + n) using KMP). For an interview, the brute force is usually fine.

Time complexity: O(m · n) brute, O(m + n) with serialization.

Space complexity: O(h) brute, O(m + n) with serialization.

Dry Run

- root = 3 (4(1, 2), 5), subRoot = 4(1, 2).
- isSame(root=3, subRoot)? Values differ. Recurse left.
- isSame(4, subRoot)? Same values; same children. Match. Return true.

Code Implementation

Java:

```java
class Solution {
    public boolean isSubtree(TreeNode root, TreeNode subRoot) {
        if (root == null) return false;
        if (isSameTree(root, subRoot)) return true;
        return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
    }
    private boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        return p.val == q.val
            && isSameTree(p.left, q.left)
            && isSameTree(p.right, q.right);
    }
}
```

Python:

```python
class Solution:
    def isSubtree(self, root, subRoot):
        if not root:
            return False
        if self.isSame(root, subRoot):
            return True
        return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)

    def isSame(self, a, b):
        if not a and not b: return True
        if not a or not b: return False
        return a.val == b.val and self.isSame(a.left, b.left) and self.isSame(a.right,
b.right)
```

Common Mistakes

- Confusing "subtree" with "submatching nodes" — for a true subtree, structure AND values must match exactly.

Key Takeaways

- Reusing a primitive helper (isSameTree) inside a wrapper (isSubtree) is a common composition.

---

30. Lowest Common Ancestor of a BST

Problem Summary

Given a BST and two values p and q, return the lowest common ancestor (LCA): the deepest node that has both p and q in its subtree.

Real-Life Scenario

Finding the closest manager who oversees two specific employees in an org-chart tree.

Pattern Recognition

Clues that this is the right pattern:

- BST property tells you which side each value is on; you only need to walk down once.

Brute Force Approach

Treat as a general tree, search both subtrees. O(n).

Time complexity: O(n).

Space complexity: O(h).

Optimized Approach

Walk from the root. If both p and q are smaller than root, LCA is in the left subtree. If both are larger, LCA is in the right. Otherwise (one on each side, or one equals root), root is the LCA.

Time complexity: O(h).

Space complexity: O(1) iterative.

Dry Run

- BST: 6(2(0,4(3,5)), 8(7,9)). p=2, q=8.
- 2 < 6 < 8. Split. Return 6.
- BST: 6(...). p=2, q=4.
- Both < 6. Go left to 2.
- 2 == p. Return 2.

Code Implementation

Java:

```JAVA
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) {
                root = root.left;
            } else if (p.val > root.val && q.val > root.val) {
                root = root.right;
            } else {
                return root;
            }
        }
        return null;
    }
}
```

Python:

```PYTHON
class Solution:
    def lowestCommonAncestor(self, root, p, q):
```

```
        while root:
            if p.val < root.val and q.val < root.val:
                root = root.left
            elif p.val > root.val and q.val > root.val:
                root = root.right
            else:
                return root
        return None
```

Common Mistakes

- Using general-tree LCA logic — works but is O(n) instead of O(h).
- Forgetting that the root itself can be the LCA when one of p/q equals root.

Key Takeaways

- In a BST, you can decide direction by comparing values. Always exploit BST ordering when present.

---

31. Binary Tree Level Order Traversal

Problem Summary

Given the root of a binary tree, return the values of its nodes level by level (top to bottom, left to right).

Real-Life Scenario

Rendering a tree visualization layer by layer in a UI; printing a directory listing one folder depth at a time.

Pattern Recognition

Clues that this is the right pattern:

- "Level by level" means BFS with a queue.

Brute Force Approach

Compute depth of every node, sort by depth — works but is over-complicated.

Time complexity: O(n log n).

Space complexity: O(n).

Optimized Approach

Use a queue. Initially push root. While queue is not empty, snapshot its size as the count of nodes in the current level. Pop that many nodes, collect their values into a list, push their non-null children. Append the list to the result.

Time complexity: O(n).

Space complexity: O(n).

Dry Run

- Tree: 3(9, 20(15, 7)). queue = [3].
- Level: size=1. Pop 3, level=[3]. Push 9, 20. queue=[9,20]. result=[[3]].

- Level: size=2. Pop 9 (no children). Pop 20, push 15, 7. level=[9,20]. queue=[15,7]. result=[[3],[9,20]].
- Level: size=2. Pop 15, 7. level=[15,7]. queue=[]. result=[[3],[9,20],[15,7]].

Code Implementation

Java:

```JAVA
import java.util.*;

class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

Python:

```PYTHON
from collections import deque

class Solution:
    def levelOrder(self, root):
        result = []
        if not root:
            return result
        q = deque([root])
        while q:
            level = []
            for _ in range(len(q)):
                node = q.popleft()
                level.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            result.append(level)
        return result
```

Common Mistakes

- Forgetting to snapshot the queue size at the start of each level — children get mixed into the current level.
- Pushing null children onto the queue, leading to NullPointerException later.

Key Takeaways

- BFS with "size snapshot" is the standard pattern for level-aware traversal.

---

32. Validate Binary Search Tree

Problem Summary

Given the root of a binary tree, determine if it is a valid BST. Each node's value must be strictly greater than every value in its left subtree and strictly less than every value in its right subtree.

Real-Life Scenario

A migration tool ensures an imported BST file actually obeys BST ordering rules before swapping it into production.

Pattern Recognition

Clues that this is the right pattern:

- Pass down "allowed range" through recursion. Each recursive call narrows it.

Brute Force Approach

For each node check that ALL values in its left subtree are smaller and ALL in its right are bigger. $O(n^2)$ repeated subtree scans.

Time complexity: $O(n^2)$.

Space complexity: O(h).

Optimized Approach

Pass a (low, high) bound at every recursive call. The root has bounds (-inf, +inf). When recursing left, the new high becomes the current node's value. When recursing right, the new low becomes the current node's value.

A node is invalid the moment its value falls outside its inherited (low, high).

Time complexity: O(n).

Space complexity: O(h).

Dry Run

- Tree: 5(1, 4(3, 6)).
- check(5, -inf, +inf): OK. Recurse left on 1 with (-inf, 5).
- check(1, -inf, 5): OK. Recurse right (null), left (null). Return true.
- Recurse right on 4 with (5, +inf).
- check(4, 5, +inf): 4 is not > 5. Return false. Overall false.

Code Implementation

Java:

```JAVA
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, null, null);
    }
    private boolean validate(TreeNode node, Integer low, Integer high) {
        if (node == null) return true;
        if (low != null && node.val <= low) return false;
        if (high != null && node.val >= high) return false;
        return validate(node.left, low, node.val)
            && validate(node.right, node.val, high);
    }
}
```

Python:

```PYTHON
class Solution:
    def isValidBST(self, root):
        def go(node, low, high):
            if not node:
                return True
            if (low is not None and node.val <= low) or (high is not None and node.val >=
high):
                return False
            return go(node.left, low, node.val) and go(node.right, node.val, high)
        return go(root, None, None)
```

Common Mistakes

- Only checking node vs its immediate children — misses violations from deeper descendants.
- Using <= or >= where strict < and > are required.

Key Takeaways

- Pass bounds down the recursion to enforce a global property locally.
- Alternative: in-order traversal must produce a strictly increasing sequence.

---

33. Kth Smallest Element in a BST

Problem Summary

Given a BST and an integer k, return the k-th smallest value.

Real-Life Scenario

A leaderboard sorted in a BST returns "the player with rank K" without exporting the full ordered list.

Pattern Recognition

Clues that this is the right pattern:

- In-order traversal of a BST yields values in sorted order. Stop after k visits.

Brute Force Approach

Inorder traverse to a list, return list[k-1]. O(n) time and space.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Iterative inorder using a stack. Push left children, pop, count visits. When count == k, return the value.

Stops early — does not necessarily traverse the whole tree.

Time complexity: O(h + k).

Space complexity: O(h).

Dry Run

- BST: 3(1(_, 2), 4). k=1.
- Stack: push 3, then 1. Pop 1 (count=1, return 1).

Code Implementation

Java:

```java
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;
        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;
            }
            curr = stack.pop();
            if (--k == 0) return curr.val;
            curr = curr.right;
        }
        return -1;
    }
}
```

Python:

```python
class Solution:
    def kthSmallest(self, root, k):
        stack = []
        curr = root
        while curr or stack:
            while curr:
```

```
                stack.append(curr)
                curr = curr.left
            curr = stack.pop()
            k -= 1
            if k == 0:
                return curr.val
            curr = curr.right
        return -1
```

Common Mistakes

- Forgetting to go right after popping — you only "use up" left subtrees, but right ones are still pending.
- Using k as 0-indexed — the problem statement is 1-indexed.

Key Takeaways

- Inorder traversal of a BST is naturally sorted; great for "find the k-th something" queries.

---

34. Construct Binary Tree from Preorder and Inorder Traversal

Problem Summary

Given two integer arrays preorder and inorder representing a binary tree's preorder and inorder traversals, reconstruct the tree.

Real-Life Scenario

Reconstructing a parsed expression tree given the order in which a parser visited nodes.

Pattern Recognition

Clues that this is the right pattern:

- Preorder gives the root first; inorder splits left and right subtrees around the root.

Brute Force Approach

Same algorithm but use indexOf(root) inside inorder, which is O(n) each call. Total $O(n^2)$.

Time complexity: $O(n^2)$.

Space complexity: O(n).

Optimized Approach

Build a hash map { value -> inorder index } once.

Walk preorder using a moving pointer "preIndex." For each subtree, the next preorder value is the root. Find its position in inorder using the map. Values to the left of that position belong to the left subtree; to the right, the right subtree.

Recurse into the inorder ranges.

Time complexity: O(n).

Space complexity: O(n).

Dry Run

- preorder = [3,9,20,15,7], inorder = [9,3,15,20,7].
- Root = 3 (preorder[0]). inorder index of 3 is 1. Left = inorder[0..0] = [9]. Right = inorder[2..4] = [15,20,7].
- Recurse left: root = 9 (preorder[1]). inorder[0..0] has only 9. Leaf.
- Recurse right with inorder[2..4]: root = 20 (preorder[2]). inorder index of 20 is 3. Left = inorder[2..2] = [15]. Right = inorder[4..4] = [7].
- Recurse: 15 leaf, 7 leaf.
- Final tree: 3(9, 20(15, 7)).

Code Implementation

Java:

```JAVA
import java.util.HashMap;
import java.util.Map;

class Solution {
    private Map<Integer, Integer> idx;
    private int preIndex = 0;
    private int[] preorder;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        this.preorder = preorder;
        idx = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) idx.put(inorder[i], i);
        return build(0, inorder.length - 1);
    }
    private TreeNode build(int left, int right) {
        if (left > right) return null;
        int rootVal = preorder[preIndex++];
        TreeNode node = new TreeNode(rootVal);
        int inIdx = idx.get(rootVal);
        node.left = build(left, inIdx - 1);
        node.right = build(inIdx + 1, right);
        return node;
    }
}
```

Python:

```PYTHON
class Solution:
    def buildTree(self, preorder, inorder):
        idx = {v: i for i, v in enumerate(inorder)}
        self.pre = 0
        def build(l, r):
            if l > r:
                return None
            val = preorder[self.pre]
            self.pre += 1
            node = TreeNode(val)
```

```
            i = idx[val]
            node.left = build(l, i - 1)
            node.right = build(i + 1, r)
            return node
        return build(0, len(inorder) - 1)
```

Common Mistakes

- Building the left subtree AFTER the right — preIndex must always advance left-first because preorder is root, left, right.
- Searching inorder linearly each call — must be hashed for O(n) total.

Key Takeaways

- Preorder gives roots in order; inorder gives left/right split. Together they uniquely define the tree.

---

35. Binary Tree Maximum Path Sum

Problem Summary

Given the root of a binary tree, return the maximum path sum of any non-empty path. A path is a sequence of nodes connected by edges; it does not need to pass through root and can start and end anywhere.

Real-Life Scenario

Finding the most profitable consecutive route in a road network represented as a tree.

Pattern Recognition

Clues that this is the right pattern:

- Tree DP / recursion with two answers: "best path going through me" and "best one-sided path I can offer my parent."

Brute Force Approach

Try every node as the "peak" of the path and explore all paths going through it.

Time complexity: $O(n^2)$ at least.

Space complexity: O(h).

Optimized Approach

For each node, compute "max gain through this node going down on one side" = node.val + max(0, leftGain, rightGain). Negative gains are clamped to 0 because you can always choose not to extend in that direction.

While recursing, also update a global answer with node.val + leftGain + rightGain (the path that bends at this node).

Return only the one-side best to the parent — a path cannot fork upward.

Time complexity: O(n).

Space complexity: O(h).

Dry Run

- Tree: -10(9, 20(15, 7)).
- gain(9) = 9. gain(15) = 15. gain(7) = 7.
- gain(20) = 20 + max(15, 7) = 35. Update best with 20 + 15 + 7 = 42.
- gain(-10) = -10 + max(0, 9, 35) = 25. Update best with -10 + 9 + 35 = 34. Best stays 42.
- Return 42.

Code Implementation

Java:

```java
class Solution {
    private int best = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        gain(root);
        return best;
    }
    private int gain(TreeNode node) {
        if (node == null) return 0;
        int left = Math.max(0, gain(node.left));
        int right = Math.max(0, gain(node.right));
        best = Math.max(best, node.val + left + right);
        return node.val + Math.max(left, right);
    }
}
```

Python:

```python
class Solution:
    def maxPathSum(self, root):
        self.best = float('-inf')
        def gain(node):
            if not node:
                return 0
            l = max(0, gain(node.left))
            r = max(0, gain(node.right))
            self.best = max(self.best, node.val + l + r)
            return node.val + max(l, r)
        gain(root)
        return self.best
```

Common Mistakes

- Returning node.val + left + right to the parent — wrong, a path cannot turn upward and downward through the same node when going further up.
- Forgetting to clamp negative gains to 0.

Key Takeaways

- Tree problems often need TWO values: what to use locally (sum at the bend) and what to return upward (one-sided extension).
- Clamping negative contributions to 0 turns "must include this branch" into "include only if helpful."

---

36. Serialize and Deserialize Binary Tree

Problem Summary

Design an algorithm to serialize a binary tree to a string and deserialize a string back to a tree. The format is up to you, as long as it round-trips correctly.

Real-Life Scenario

Saving a parsed AST or DOM tree to disk so it can be reloaded later — exactly what compilers, IDEs, and browser caches do.

Pattern Recognition

Clues that this is the right pattern:

- Tree -> string -> tree using a chosen traversal order.

Brute Force Approach

Inefficient formats like JSON nested objects work but are bulky.

Time complexity: O(n).

Space complexity: O(n).

Optimized Approach

Use preorder traversal. Visit root, then serialize left subtree, then right. Use a special token like "#" for null children. Separate values with commas.

To deserialize, split by commas and consume tokens from the front using a queue. Recursive build mirrors the serialization.

Time complexity: O(n).

Space complexity: O(n).

Dry Run

- Tree: 1(2, 3(_, 4)).
- Serialize preorder with nulls: "1,2,#,#,3,#,4,#,#"
- Deserialize: read "1" -> root=1. Recurse left: read "2" -> 2. Recurse left of 2: read "#" -> null. Recurse right of 2: read "#" -> null. Recurse right of 1: read "3" -> 3. Recurse left of 3: read "#" -> null. Recurse right of 3: read "4" -> 4. Read "#" -> null. Read "#" -> null. Done.

Code Implementation

Java:

```
JAVA
import java.util.*;
```

```
public class Codec {
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        ser(root, sb);
        return sb.toString();
    }
    private void ser(TreeNode node, StringBuilder sb) {
        if (node == null) { sb.append("#,"); return; }
        sb.append(node.val).append(",");
        ser(node.left, sb);
        ser(node.right, sb);
    }
    public TreeNode deserialize(String data) {
        Deque<String> tokens = new ArrayDeque<>(Arrays.asList(data.split(",")));
        return des(tokens);
    }
    private TreeNode des(Deque<String> tokens) {
        String t = tokens.poll();
        if (t.equals("#")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(t));
        node.left = des(tokens);
        node.right = des(tokens);
        return node;
    }
}
```

Python:

```
PYTHON
from collections import deque

class Codec:
    def serialize(self, root):
        out = []
        def ser(node):
            if not node:
                out.append('#')
                return
            out.append(str(node.val))
            ser(node.left)
            ser(node.right)
        ser(root)
        return ','.join(out)

    def deserialize(self, data):
        tokens = deque(data.split(','))
        def des():
            t = tokens.popleft()
            if t == '#':
                return None
            node = TreeNode(int(t))
            node.left = des()
            node.right = des()
```

```
            return node
        return des()
```

Common Mistakes

- Using inorder for serialization — does not capture structure uniquely without nulls.
- Forgetting null markers — round trip becomes ambiguous.

Key Takeaways

- Preorder + null markers is the simplest, most reliable tree serialization.
- Mirror the structure: serialize root-left-right, deserialize the same way.

---

9. Tries (Prefix Trees)

Topic Introduction

What it is: A trie is a tree-like data structure where each node represents a single character. Words sharing a common prefix share the same path from the root. A flag at each node marks whether a complete word ends there.

Why it exists: Tries make prefix-based queries extremely fast. Searching for any word starting with "app" takes time proportional to the length of "app", not the size of the dictionary.

What problem it solves: Prefix matching, autocomplete, dictionary lookups, spell-checkers, and any string search where many strings share common starts.

Typical time complexity: Insert/search/startsWith all run in O(L), where L is the length of the word.

Typical space complexity: O(N * L) in the worst case, where N is the number of words and L is average length. Shared prefixes save memory.

Real-life analogy: A phone book organized by first letter, then second letter, then third — to find all names starting with "Sm", you only flip to the "S" section, then "m". You never scan unrelated names.

Real-world software engineering use cases:

- Search engine autocomplete suggestions.
- IDE code completion.
- IP routing tables (longest prefix matching).
- Spell-checkers and dictionary apps.
- T9 predictive text on older phones.

37. Implement Trie (Prefix Tree)

Problem Summary

Implement a trie supporting three operations: insert(word), search(word) — returns true if the word was inserted before — and startsWith(prefix) — returns true if any inserted word starts with the prefix.

Real-Life Scenario

You are building the autocomplete feature for a search bar. As the user types, you need to instantly tell which entries start with their input.

Pattern Recognition

Clues that this is the right pattern:

- Multiple words sharing prefixes.
- Need fast prefix lookups, not just full-word lookups.
- When you see "implement autocomplete" or "fast prefix search," think trie.

Brute Force Approach

Store all words in a list. To search a prefix, scan every word and check if it starts with the prefix.

Time complexity: O(N * L) per query, where N is the number of words and L is prefix length.

Space complexity: O(N * L).

Optimized Approach

Each TrieNode has a map of child characters (or a fixed-size array for 26 letters) and a boolean isEnd marking whether a word ends here.

Insert: walk down character by character, creating missing children, mark the last node isEnd = true.

Search: walk down characters; if any character is missing, return false. At the end, return isEnd.

StartsWith: same as search but skip the isEnd check at the end — just return true if you successfully walked all prefix characters.

Time complexity: O(L) per operation.

Space complexity: O(total characters across all words).

Dry Run

- Insert "apple": create nodes a -> p -> p -> l -> e, mark e as isEnd.
- Insert "app": walk a -> p -> p (already exists), mark second p as isEnd.
- Search "apple": walk a -> p -> p -> l -> e, last node isEnd = true. Return true.
- Search "app": walk a -> p -> p, last node isEnd = true. Return true.
- Search "ap": walk a -> p, last node isEnd = false. Return false.
- StartsWith "ap": walk a -> p successfully. Return true.

Code Implementation

Java:

```JAVA
class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            // Create child if it does not exist
```

```
            node.children.putIfAbsent(c, new TrieNode());
            node = node.children.get(c);
        }
        node.isEnd = true; // Mark end of word
    }

    public boolean search(String word) {
        TrieNode node = walk(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return walk(prefix) != null;
    }

    private TrieNode walk(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            node = node.children.get(c);
            if (node == null) return null; // Path broke
        }
        return node;
    }

    private static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        boolean isEnd = false;
    }
}
```

Python:

```
PYTHON
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.is_end
```

```
    def startsWith(self, prefix: str) -> bool:
        return self._walk(prefix) is not None

    def _walk(self, s: str):
        node = self.root
        for c in s:
            if c not in node.children:
                return None
            node = node.children[c]
        return node
```

Common Mistakes

- Forgetting to mark the final node as isEnd — search will then incorrectly return false for inserted words.
- Confusing search and startsWith — search needs the isEnd flag; startsWith does not.
- Using a single shared root reference and mutating it directly across operations.

Key Takeaways

- A trie trades memory for speed. Each operation is O(L), independent of dictionary size.
- Use a HashMap for variable alphabets, or a fixed-size array (size 26) for lowercase English letters — the array is faster.
- The isEnd flag is what separates "this is a real word" from "this is just a prefix".

---

38. Design Add and Search Words Data Structure

Problem Summary

Build a data structure that supports addWord(word) and search(word). Search may include the wildcard character "." which matches any single letter.

Real-Life Scenario

A regex-lite dictionary lookup: a user types "c.t" and you must return whether any of "cat", "cot", "cut" was added.

Pattern Recognition

Clues that this is the right pattern:

- Trie + recursive DFS to handle the wildcard branching.
- Whenever you see "search with wildcard" on words, think trie with DFS.

Brute Force Approach

Store all words in a list. For each search, compare against every stored word, treating "." as a match.

Time complexity: O(N * L) per search.

Space complexity: O(N * L).

Optimized Approach

Insert exactly like a normal trie.

Search recursively. At each character: if it is a letter, follow that one child. If it is ".", recurse into every child.

When all characters are consumed, return the current node.isEnd.

Time complexity: addWord is O(L). search is O(L) without dots; with dots, worst case O(26^L).

Space complexity: O(total characters).

Dry Run

- Add "bad", "dad", "mad".
- Search "pad": at root, no "p" child. Return false.
- Search "bad": follow b -> a -> d, isEnd true. Return true.
- Search ".ad": at root, "." -> try every child (b, d, m). For "b", recurse on "ad" -> b->a->d isEnd true. Return true.

Code Implementation

Java:

```JAVA
class WordDictionary {
    private TrieNode root;

    public WordDictionary() {
        root = new TrieNode();
    }

    public void addWord(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            node.children.putIfAbsent(c, new TrieNode());
            node = node.children.get(c);
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0);
    }

    private boolean dfs(TrieNode node, String word, int i) {
        if (node == null) return false;
        if (i == word.length()) return node.isEnd;

        char c = word.charAt(i);
        if (c == '.') {
            // Try every child
            for (TrieNode child : node.children.values()) {
                if (dfs(child, word, i + 1)) return true;
            }
            return false;
        }
        return dfs(node.children.get(c), word, i + 1);
    }
```

```
    private static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        boolean isEnd = false;
    }
}
```

Python:

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end = True

    def search(self, word: str) -> bool:
        return self._dfs(self.root, word, 0)

    def _dfs(self, node, word, i):
        if node is None:
            return False
        if i == len(word):
            return node.is_end
        c = word[i]
        if c == '.':
            for child in node.children.values():
                if self._dfs(child, word, i + 1):
                    return True
            return False
        return self._dfs(node.children.get(c), word, i + 1)
```

Common Mistakes

- Using a flat list — works but times out on the constraint sizes.
- For ".", iterating only over a fixed 26 letters even when the child does not exist (wastes work). Iterating over node.children directly is cleaner.
- Forgetting to check node.isEnd at the end of recursion — returns true for prefix matches that are not full words.

Key Takeaways

- Trie + DFS handles wildcard search elegantly.
- Wildcards turn one path into many — recursion is the natural tool.
- Always handle the "node is null" base case before reading any of its fields.

---

39. Word Search II

Problem Summary

Given an m x n board of letters and a list of words, return all words that can be formed by sequentially adjacent cells (horizontally or vertically), where each cell may not be reused within a word.

Real-Life Scenario

A Boggle-style game: scan a letter grid and report which dictionary words appear as connected paths.

Pattern Recognition

Clues that this is the right pattern:

- Multiple words to search + grid traversal = trie + backtracking.
- Without a trie, you would re-traverse the grid for every word; with one, you traverse once and prune dead ends.

Brute Force Approach

For each word in the list, run a DFS from every cell to see if that word can be formed. Slow when many words share prefixes.

Time complexity: O(W * M * N * 4^L), where W is number of words.

Space complexity: O(L) recursion.

Optimized Approach

Insert all words into a trie.

Run a DFS from every cell, walking the trie in parallel with the grid.

If the current letter is not a child of the current trie node, prune the branch immediately.

When trie node.isEnd is true, add the corresponding word to results and clear node.word to avoid duplicates.

Mark visited cells (e.g., set to "#") and restore them on backtrack.

Time complexity: O(M * N * 4^L) where L is the longest word length.

Space complexity: O(total characters in dictionary).

Dry Run

- Board = [[o,a,a,n],[e,t,a,e],[i,h,k,r],[i,f,l,v]]. Words = ["oath","pea","eat","rain"].
- Insert all words into trie. Trie root has children o, p, e, r.
- Start DFS at (0,0)="o": trie root has o -> recurse. Then "a" matches, then "t", then "h" -> isEnd true -> record "oath".
- DFS from cells starting with "p" finds nothing (no p in grid).
- DFS from "e" cells finds "eat". DFS from "r" finds nothing forming "rain".
- Result: ["oath","eat"].

Code Implementation

Java:

```java
class Solution {
    private List<String> result = new ArrayList<>();

    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = buildTrie(words);
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                dfs(board, r, c, root);
            }
        }
        return result;
    }

    private void dfs(char[][] board, int r, int c, TrieNode node) {
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return;
        char ch = board[r][c];
        TrieNode next = node.children[ch - 'a'];
        if (ch == '#' || next == null) return; // pruned

        if (next.word != null) {
            result.add(next.word);
            next.word = null; // prevent duplicates
        }

        board[r][c] = '#'; // mark visited
        dfs(board, r + 1, c, next);
        dfs(board, r - 1, c, next);
        dfs(board, r, c + 1, next);
        dfs(board, r, c - 1, next);
        board[r][c] = ch; // restore
    }

    private TrieNode buildTrie(String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode node = root;
            for (char c : w.toCharArray()) {
                if (node.children[c - 'a'] == null) {
                    node.children[c - 'a'] = new TrieNode();
                }
                node = node.children[c - 'a'];
            }
            node.word = w; // store full word at end node
        }
        return root;
    }

    private static class TrieNode {
        TrieNode[] children = new TrieNode[26];
```

```
        String word = null;
    }
}
```

Python:

```python
class Solution:
    def findWords(self, board, words):
        # Build trie
        root = {}
        for w in words:
            node = root
            for c in w:
                node = node.setdefault(c, {})
            node['#'] = w  # mark end with full word

        rows, cols = len(board), len(board[0])
        result = []

        def dfs(r, c, node):
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            ch = board[r][c]
            if ch not in node:
                return
            nxt = node[ch]
            if '#' in nxt:
                result.append(nxt['#'])
                del nxt['#']  # avoid duplicates

            board[r][c] = '*'  # mark visited
            for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                dfs(r + dr, c + dc, nxt)
            board[r][c] = ch  # restore

        for r in range(rows):
            for c in range(cols):
                dfs(r, c, root)
        return result
```

Common Mistakes

- Forgetting to mark cells visited — the same cell can then be reused inside a word.
- Forgetting to restore the cell on backtrack — destroys the board for other DFS starts.
- Adding the same word twice when it appears via different paths — clear the trie's word field after adding.
- Searching each word independently when they share prefixes — that is the brute force we are trying to avoid.

Key Takeaways

- Trie + backtracking is the canonical pattern for "find many words in a grid".

- Pruning is the real optimization: as soon as the current path cannot extend any dictionary word, stop.
- Storing the full word at the trie's end node avoids reconstructing the path during DFS.

---

10. Heap / Priority Queue

Topic Introduction

What it is: A heap is a binary tree where each parent is smaller (min-heap) or larger (max-heap) than its children. A priority queue is the abstract idea of "always give me the smallest/largest," and heaps are how we implement it efficiently.

Why it exists: You often need to repeatedly find or remove the smallest or largest element of a changing collection. Sorting every time is too slow; a heap does it in O(log n) per operation.

What problem it solves: Top-K queries, scheduling, finding running medians, merging sorted streams, Dijkstra's shortest path.

Typical time complexity: Insert and remove top: O(log n). Peek top: O(1). Building a heap from an array: O(n).

Typical space complexity: O(n).

Real-life analogy: A hospital emergency room. Patients arrive in any order, but the most critical one is always treated first. The "priority" is severity, and the queue automatically keeps it on top.

Real-world software engineering use cases:

- OS task schedulers — the highest-priority process runs next.
- Job queues in distributed systems (think: Kafka with priorities).
- Dijkstra's and A* shortest-path algorithms in maps and games.
- Streaming top-K — top trending hashtags, top-spending customers.
- Median maintenance over a moving window of values.

40. Find Median from Data Stream

Problem Summary

Design a data structure that supports two operations: addNum(num) — add a number to the stream — and findMedian() — return the median of all numbers seen so far.

Real-Life Scenario

A live analytics dashboard reports the median latency of incoming requests as they stream in.

Pattern Recognition

Clues that this is the right pattern:

- You need a "middle" element of a growing collection that keeps changing.
- Two heaps: a max-heap for the lower half, a min-heap for the upper half. The two tops give you the median.

Brute Force Approach

Store every number in a list. To find the median, sort and pick the middle.

Time complexity: addNum O(1), findMedian O(n log n).

Space complexity: O(n).

Optimized Approach

Maintain two heaps. low is a max-heap holding the smaller half. high is a min-heap holding the larger half.

Invariant: every element in low is <= every element in high, and the sizes differ by at most 1.

addNum: push to low first, then move low's top to high to keep order. If high becomes larger than low, move high's top back to low.

findMedian: if sizes differ, return low's top. If equal, return the average of the two tops.

Time complexity: addNum O(log n), findMedian O(1).

Space complexity: O(n).

Dry Run

- addNum(1): low=[1], high=[]. Median = 1.
- addNum(2): push 1 to low first, then move 1 to high. Push 2 to low. Move 2 to high. Re-balance: low=[1], high=[2]. Median = 1.5.
- addNum(3): push 3 to low, move 3 to high. low=[1], high=[2,3]. high too big, move 2 back. low=[2,1], high=[3]. Median = 2.

Code Implementation

Java:

```JAVA
class MedianFinder {
    // low is a max-heap (smaller half), high is a min-heap (larger half)
    private PriorityQueue<Integer> low = new PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue<Integer> high = new PriorityQueue<>();

    public void addNum(int num) {
        // Always push to low first, then move the largest of low to high
        low.offer(num);
        high.offer(low.poll());
        // Keep low.size() >= high.size() (low can have one extra)
        if (high.size() > low.size()) {
            low.offer(high.poll());
        }
    }

    public double findMedian() {
        if (low.size() > high.size()) {
            return low.peek();
        }
        return (low.peek() + high.peek()) / 2.0;
    }
}
```

Python:

```python
import heapq

class MedianFinder:
    def __init__(self):
        # Python's heapq is a min-heap, so we negate values for the max-heap
        self.low = []   # max-heap (stored as negatives)
        self.high = []  # min-heap

    def addNum(self, num: int) -> None:
        heapq.heappush(self.low, -num)
        # Move the largest of low to high
        heapq.heappush(self.high, -heapq.heappop(self.low))
        # Re-balance if high grew larger
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def findMedian(self) -> float:
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2.0
```

Common Mistakes

- Forgetting that Java's PriorityQueue is a min-heap by default — you must pass Collections.reverseOrder() for a max-heap.
- Pushing directly to whichever heap "feels right" without the cross-move, breaking the invariant that low's top <= high's top.
- Returning an integer median when the input contains an even count — should be a double.
- Letting one heap grow more than 1 larger than the other.

Key Takeaways

- Two heaps is the canonical pattern for "running median" or "kth smallest in a stream".
- Always push first, then move across, then rebalance — three crisp steps.
- In Java, use PriorityQueue with reverseOrder() for a max-heap. In Python, use negation tricks with heapq.

---

11. Backtracking

Topic Introduction

What it is: Backtracking is recursion that explores all possible decisions, but undoes a choice when it leads to a dead end. You build a partial solution, recurse to extend it, and "back up" if extension fails.

Why it exists: Many problems require enumerating combinations, permutations, or paths through a search space. A naive loop cannot express the branching; recursion with backtracking does.

What problem it solves: Combinations, permutations, subsets, N-Queens, Sudoku, word search in a grid, generate-all-X problems.

Typical time complexity: Often exponential. The shape of the search tree determines the bound — typically O(branching^depth).

Typical space complexity: O(depth) for the recursion stack, plus the partial solution.

Real-life analogy: Walking a maze by always trying the leftmost path. When you hit a dead end, you walk back to the last junction and try a different turn. You never give up until every branch has been tried.

Real-world software engineering use cases:

- Constraint solvers (Sudoku, crosswords, scheduling).
- Generating test inputs by enumerating combinations.
- Pathfinding in discrete game states (chess engines explore via backtracking).
- Compiler register allocation and SAT solvers.
- Permutation-based feature search in machine-learning experiments.

41. Combination Sum

Problem Summary

Given an array of distinct positive integers candidates and a target, return all unique combinations of candidates that sum to target. Each candidate may be used unlimited times.

Real-Life Scenario

Pricing an order: given coin denominations, return every distinct way to make exactly $X.

Pattern Recognition

Clues that this is the right pattern:

- "Find all combinations" or "all ways to" — backtracking signal.
- Reuse-allowed: when recursing, do not advance past the current candidate; recurse with the same index.

Brute Force Approach

Enumerate every possible multiset of candidates up to the target — typically by treating it as a knapsack-style search but without pruning. Same time as the optimized version actually; the optimization is in trimming branches early.

Time complexity: Exponential.

Space complexity: O(target).

Optimized Approach

Sort candidates so we can prune early once a candidate exceeds the remaining target.

Use DFS with parameters: start index, current path, remaining sum.

At each step: if remaining = 0, record a copy of the path. Else, for i from start to end, if candidates[i] > remaining, break (sorted prune). Otherwise, add candidate, recurse with same i (reuse allowed), backtrack.

Time complexity: O(N^(target/min)) loosely.

Space complexity: O(target/min) recursion depth.

Dry Run

- candidates = [2,3,6,7], target = 7. Sort -> already sorted.
- Start path=[], rem=7, i=0.
- Pick 2: path=[2], rem=5. Pick 2: [2,2], rem=3. Pick 2: [2,2,2], rem=1. Cannot pick 2 (rem<2). Backtrack.
- [2,2], pick 3: [2,2,3], rem=0 -> record. Backtrack.
- Eventually find [7] and [2,2,3]. Result: [[2,2,3],[7]].

Code Implementation

Java:

```JAVA
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(candidates); // for early pruning
        backtrack(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private void backtrack(int[] cand, int remain, int start, List<Integer> path,
List<List<Integer>> result) {
        if (remain == 0) {
            result.add(new ArrayList<>(path)); // copy snapshot
            return;
        }
        for (int i = start; i < cand.length; i++) {
            if (cand[i] > remain) break; // sorted -> all later are too big
            path.add(cand[i]);
            // i (not i+1) because we can reuse the same candidate
            backtrack(cand, remain - cand[i], i, path, result);
            path.remove(path.size() - 1); // undo
        }
    }
}
```

Python:

```PYTHON
class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []

        def backtrack(start, remain, path):
            if remain == 0:
                result.append(path[:])  # copy
                return
            for i in range(start, len(candidates)):
```

```
                if candidates[i] > remain:
                    break
                path.append(candidates[i])
                backtrack(i, remain - candidates[i], path)  # i, not i+1
                path.pop()

        backtrack(0, target, [])
        return result
```

Common Mistakes

- Recursing with i+1 instead of i — that disallows reuse, giving the wrong answer.
- Forgetting to copy the path when adding to results — every entry would point to the same mutating list.
- Not sorting and pruning — the solution still works but is slow.
- Not popping after the recursive call — the path keeps growing across branches.

Key Takeaways

- Backtracking template: choose, recurse, undo. Always undo.
- Snapshot the path with new ArrayList<>(path) or path[:] before storing.
- Pass start index forward to avoid producing duplicate combinations like [2,3] and [3,2].

---

42. Word Search

Problem Summary

Given an m x n board of letters and a word, return true if the word can be constructed from sequentially adjacent cells (horizontally or vertically) without reusing any cell.

Real-Life Scenario

A word-puzzle game where you check whether the player can connect a target word on the grid.

Pattern Recognition

Clues that this is the right pattern:

- Grid + path search where cells may not repeat = DFS with backtracking.
- Mark visited, recurse to neighbors, unmark on the way back.

Brute Force Approach

Same as the optimized DFS; there is no simpler approach. The only question is whether you backtrack correctly.

Time complexity: O(M * N * 4^L) where L is the word length.

Space complexity: O(L) recursion.

Optimized Approach

Try every cell as a starting point.

DFS from a cell: if the current cell does not match word[i], return false.

If i is the last index, return true.

Mark cell as visited (overwrite with "#"), recurse to four neighbors. If any neighbor returns true, propagate true.

Restore the cell before returning. Always.

Time complexity: O(M * N * 4^L).

Space complexity: O(L).

Dry Run

- Board=[[A,B,C,E],[S,F,C,S],[A,D,E,E]], word="ABCCED".
- Start at (0,0)="A" matches word[0].
- Mark (0,0)="#". Recurse to (0,1)="B" matches word[1]. Mark, recurse to (0,2)="C", word[2] match.
- From (0,2): (1,2)="C" matches word[3]. From (1,2): (2,2)="E" matches word[4]. From (2,2): (2,1)="D" matches word[5]. i=L-1 -> return true.

Code Implementation

Java:

```JAVA
class Solution {
    public boolean exist(char[][] board, String word) {
        int rows = board.length, cols = board[0].length;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (dfs(board, r, c, word, 0)) return true;
            }
        }
        return false;
    }

    private boolean dfs(char[][] board, int r, int c, String word, int i) {
        if (i == word.length()) return true; // matched all
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
        if (board[r][c] != word.charAt(i)) return false;

        char saved = board[r][c];
        board[r][c] = '#'; // mark visited

        boolean found = dfs(board, r + 1, c, word, i + 1)
                     || dfs(board, r - 1, c, word, i + 1)
                     || dfs(board, r, c + 1, word, i + 1)
                     || dfs(board, r, c - 1, word, i + 1);

        board[r][c] = saved; // restore (backtrack)
        return found;
    }
}
```

Python:

```PYTHON
class Solution:
```

```
    def exist(self, board, word):
        rows, cols = len(board), len(board[0])

        def dfs(r, c, i):
            if i == len(word):
                return True
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return False
            if board[r][c] != word[i]:
                return False

            saved = board[r][c]
            board[r][c] = '#'

            found = (dfs(r+1, c, i+1) or dfs(r-1, c, i+1)
                  or dfs(r, c+1, i+1) or dfs(r, c-1, i+1))

            board[r][c] = saved
            return found

        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True
        return False
```

Common Mistakes

- Not marking cells as visited — the path can revisit the same cell.
- Forgetting to restore the cell on the way back, corrupting the board for other starting points.
- Checking bounds after reading board[r][c] — that throws an out-of-bounds exception.
- Returning early from the inner DFS without restoring the cell.

Key Takeaways

- In-place visited marking saves memory vs a separate boolean grid.
- Always: bounds check first, then character match, then recurse, then restore.
- The same template extends to Word Search II — just walk a trie alongside the DFS.

---

12. Graphs

Topic Introduction

What it is: A graph is a set of nodes (vertices) connected by edges. Edges may be directed or undirected, weighted or unweighted. Trees, grids, and linked lists are all special cases of graphs.

Why it exists: Many real systems are network-shaped: roads, social connections, dependencies. Graph algorithms answer "is there a path?", "what is the shortest path?", "are there cycles?", "which components are connected?".

What problem it solves: Connectivity, shortest paths, cycle detection, topological ordering, network flow, clustering.

Typical time complexity: BFS and DFS run in O(V + E). Dijkstra is O((V+E) log V) with a heap. Union-Find operations are nearly O(1) amortized.

Typical space complexity: O(V + E) for the graph plus O(V) for visited tracking.

Real-life analogy: A city map. Intersections are nodes; streets are edges. To get from your house to a friend's house, you explore connected streets — that is graph traversal.

Real-world software engineering use cases:

- Social network friend suggestions (graph traversal).
- GPS route planning (Dijkstra, A*).
- Build systems detecting circular dependencies (cycle detection).
- Compilers ordering modules to compile (topological sort).
- Recommendation engines clustering similar users (connected components).

43. Number of Islands

Problem Summary

Given an m x n grid where "1" is land and "0" is water, count the number of islands. An island is land connected horizontally or vertically.

Real-Life Scenario

Image segmentation: count distinct shapes in a binary image.

Pattern Recognition

Clues that this is the right pattern:

- Grid + count connected regions = DFS or BFS from each unvisited land cell.
- Each launch of DFS/BFS exhausts one entire island; increment a counter per launch.

Brute Force Approach

No real brute force here — the standard DFS/BFS is the canonical solution. An alternative is union-find, which is more code but useful when the grid changes dynamically.

Time complexity: O(M * N).

Space complexity: O(M * N) for the visited stack/queue.

Optimized Approach

Loop through every cell. When you find a "1", increment count and DFS to mark all connected land as visited (overwrite with "0").

DFS recurses into the four neighbors; bounds check first, then check if it is "1".

Time complexity: O(M * N) — each cell is visited at most twice.

Space complexity: O(M * N) recursion depth in the worst case.

Dry Run

- Grid = [["1","1","0"],["1","0","0"],["0","0","1"]].
- (0,0)="1" -> count=1. DFS sinks (0,0),(0,1),(1,0). Grid -> [["0","0","0"],["0","0","0"],["0","0","1"]].
- (2,2)="1" -> count=2. DFS sinks (2,2). Result: 2.

Code Implementation

Java:

```JAVA
class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[0].length; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c); // sink the whole island
                }
            }
        }
        return count;
    }

    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
        if (grid[r][c] != '1') return;
        grid[r][c] = '0'; // mark visited by sinking
        dfs(grid, r + 1, c);
        dfs(grid, r - 1, c);
        dfs(grid, r, c + 1);
        dfs(grid, r, c - 1);
    }
}
```

Python:

```PYTHON
class Solution:
    def numIslands(self, grid):
        rows, cols = len(grid), len(grid[0])
        count = 0

        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            if grid[r][c] != '1':
                return
            grid[r][c] = '0'
            dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    count += 1
```

```
                    dfs(r, c)
        return count
```

Common Mistakes

- Not marking visited cells, leading to infinite recursion or double-counting.
- Using DFS without checking bounds first, causing index-out-of-bounds.
- On very large grids, deep recursion may stack-overflow — use BFS with a queue if so.

Key Takeaways

- "Count connected regions in a grid" -> launch DFS/BFS from each unvisited land cell.
- Sink-as-you-go (overwrite to "0") avoids needing a separate visited array.
- BFS and DFS both work; choose BFS for very deep grids to avoid stack overflow.

---

44. Clone Graph

Problem Summary

Given a reference to a node in a connected undirected graph, return a deep copy of the graph. Each node has a value and a list of neighbors.

Real-Life Scenario

You need to snapshot the current state of a network of objects so you can experiment without affecting the original.

Pattern Recognition

Clues that this is the right pattern:

- Graph traversal that builds a parallel structure as it walks.
- Map original-node -> cloned-node so you don't clone the same node twice.

Brute Force Approach

Naively recurse and create new nodes without a map. You will infinite-loop on cycles or create duplicate clones.

Time complexity: Unbounded due to cycles.

Space complexity: Unbounded.

Optimized Approach

Use a HashMap from original node to its clone.

DFS: if the original is already in the map, return its clone.

Otherwise, create a clone with the same value, store it in the map, then recurse on each neighbor and append the cloned neighbor to the new node's neighbors list.

Time complexity: O(V + E).

Space complexity: O(V) for the map and recursion.

Dry Run

- Graph: 1-2, 1-4, 2-3, 3-4. Start at node 1.
- Clone 1, map: {1->1'}. Recurse on neighbor 2.
- Clone 2, map: {1->1', 2->2'}. Recurse on neighbor 1: already in map, return 1'. Add 1' to 2'.neighbors.
- Recurse on neighbor 3. Clone 3, recurse on 2 (in map, return 2'), recurse on 4. Clone 4, recurse on 3 and 1 (both in map). Done.
- All clones link in the same shape as originals.

Code Implementation

Java:

```JAVA
/*
class Node {
    public int val;
    public List<Node> neighbors;
}
*/
class Solution {
    private Map<Node, Node> visited = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) return null;
        if (visited.containsKey(node)) return visited.get(node);

        Node clone = new Node(node.val);
        visited.put(node, clone); // store BEFORE recursing to handle cycles
        for (Node n : node.neighbors) {
            clone.neighbors.add(cloneGraph(n));
        }
        return clone;
    }
}
```

Python:

```PYTHON
# Definition for a Node:
# class Node:
#     def __init__(self, val=0, neighbors=None):
#         self.val = val
#         self.neighbors = neighbors if neighbors is not None else []

class Solution:
    def cloneGraph(self, node):
        if not node:
            return None
        visited = {}

        def dfs(n):
            if n in visited:
                return visited[n]
            clone = Node(n.val)
```

```
            visited[n] = clone  # store before recursion
            for nb in n.neighbors:
                clone.neighbors.append(dfs(nb))
            return clone

        return dfs(node)
```

Common Mistakes

- Storing the clone in the map AFTER recursing — cycles cause infinite recursion.
- Returning the original node's neighbor instead of the cloned neighbor.
- Forgetting to handle null input.

Key Takeaways

- "Deep clone with cycles" — always use a map keyed by the original node.
- Save before recursing; this is the universal trick for cyclic graph traversal.
- Same pattern works for serializing/deserializing graphs.

---

45. Pacific Atlantic Water Flow

Problem Summary

Given an m x n grid of heights, return all cells from which water can flow to both the Pacific (top and left edges) and the Atlantic (bottom and right edges). Water flows from higher to equal-or-lower neighbors.

Real-Life Scenario

Hydrology modeling on a digital elevation map: find ridges that drain to two different oceans.

Pattern Recognition

Clues that this is the right pattern:

- Reverse the direction. Instead of asking "can this cell reach the ocean?", ask "starting from the ocean edge, which cells can reach upward?". Run two BFS/DFS — one from each ocean — and intersect.

Brute Force Approach

For each cell, run a DFS to check if it can reach the Pacific, and another to check if it can reach the Atlantic. Slow because of repeated work.

Time complexity: O((M * N)^2).

Space complexity: O(M * N).

Optimized Approach

Create two boolean grids: pacific and atlantic.

DFS starting from every Pacific-edge cell, going to neighbors with height >= current, marking pacific[r][c] = true.

DFS starting from every Atlantic-edge cell similarly.

A cell qualifies iff pacific[r][c] && atlantic[r][c].

Time complexity: O(M * N).

Space complexity: O(M * N).

Dry Run

- Heights = small example with edges marked. From top row, DFS upward to higher cells -> mark pacific.
- From bottom row, DFS upward similarly -> mark atlantic.
- Intersect the two boolean grids -> result list.

Code Implementation

Java:

```
JAVA
class Solution {
    private int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};

    public List<List<Integer>> pacificAtlantic(int[][] heights) {
        int m = heights.length, n = heights[0].length;
        boolean[][] pac = new boolean[m][n];
        boolean[][] atl = new boolean[m][n];

        // Top row + bottom row
        for (int c = 0; c < n; c++) {
            dfs(heights, pac, 0, c);
            dfs(heights, atl, m - 1, c);
        }
        // Left column + right column
        for (int r = 0; r < m; r++) {
            dfs(heights, pac, r, 0);
            dfs(heights, atl, r, n - 1);
        }

        List<List<Integer>> result = new ArrayList<>();
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (pac[r][c] && atl[r][c]) {
                    result.add(Arrays.asList(r, c));
                }
            }
        }
        return result;
    }

    private void dfs(int[][] heights, boolean[][] visited, int r, int c) {
        visited[r][c] = true;
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr < 0 || nr >= heights.length || nc < 0 || nc >= heights[0].length)
continue;
            if (visited[nr][nc]) continue;
            // We are walking "uphill" (reverse flow), so neighbor must be >= current
            if (heights[nr][nc] < heights[r][c]) continue;
```

```
            dfs(heights, visited, nr, nc);
        }
    }
}
```

Python:

```
PYTHON
class Solution:
    def pacificAtlantic(self, heights):
        m, n = len(heights), len(heights[0])
        pac, atl = set(), set()

        def dfs(r, c, visited):
            visited.add((r, c))
            for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and (nr, nc) not in visited \
                   and heights[nr][nc] >= heights[r][c]:
                    dfs(nr, nc, visited)

        for c in range(n):
            dfs(0, c, pac)
            dfs(m - 1, c, atl)
        for r in range(m):
            dfs(r, 0, pac)
            dfs(r, n - 1, atl)

        return [[r, c] for r in range(m) for c in range(n) if (r, c) in pac and (r, c) in
atl]
```

Common Mistakes

- Walking downhill — remember we reverse the flow when starting from the ocean.
- Only running DFS from one edge per ocean — must cover the entire boundary.
- Using ">=" carelessly — water flows from higher to equal-or-lower; reversed, we step to neighbors that are equal or higher.

Key Takeaways

- "Reach the boundary" problems are often easier solved by starting at the boundary and going inward.
- Two parallel DFS/BFS + intersection is a common multi-source pattern.
- Marking visited per ocean is essential — the two searches are independent.

---

46. Course Schedule

Problem Summary

Given numCourses and prerequisites where prerequisites[i] = [a, b] means you must take b before a, return true if you can finish all courses (no cycle).

Real-Life Scenario

A build system must determine if it can compile all modules — equivalent to checking the dependency graph has no circular dependency.

Pattern Recognition

Clues that this is the right pattern:

- Detect cycle in a directed graph.
- Two standard ways: Kahn's algorithm (BFS topological sort) or DFS with three colors (white, gray, black).

Brute Force Approach

Try every order and check feasibility — exponential.

Time complexity: O(n!).

Space complexity: O(n).

Optimized Approach

Kahn's algorithm: build adjacency list and an in-degree array. Push all 0-indegree nodes to a queue. Pop one, decrement in-degree of each neighbor, push when in-degree hits 0.

If you process all numCourses nodes, no cycle. If fewer, a cycle exists.

Time complexity: O(V + E).

Space complexity: O(V + E).

Dry Run

- numCourses=2, prereqs=[[1,0]]. Adj: 0->[1]. Indegree: [0,1]. Queue=[0].
- Pop 0, decrement indegree of 1 -> 0. Push 1. Queue=[1].
- Pop 1. Processed 2 nodes -> return true.

Code Implementation

Java:

```
JAVA
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        int[] indegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());

        for (int[] p : prerequisites) {
            // edge: p[1] -> p[0]
            adj.get(p[1]).add(p[0]);
            indegree[p[0]]++;
        }

        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) queue.offer(i);
        }
```

```
        int processed = 0;
        while (!queue.isEmpty()) {
            int course = queue.poll();
            processed++;
            for (int next : adj.get(course)) {
                if (--indegree[next] == 0) queue.offer(next);
            }
        }

        return processed == numCourses;
    }
}
```

Python:

```
PYTHON
from collections import deque, defaultdict

class Solution:
    def canFinish(self, numCourses, prerequisites):
        adj = defaultdict(list)
        indegree = [0] * numCourses
        for a, b in prerequisites:
            adj[b].append(a)
            indegree[a] += 1

        queue = deque(i for i in range(numCourses) if indegree[i] == 0)
        processed = 0

        while queue:
            course = queue.popleft()
            processed += 1
            for nxt in adj[course]:
                indegree[nxt] -= 1
                if indegree[nxt] == 0:
                    queue.append(nxt)

        return processed == numCourses
```

Common Mistakes

- Mixing up the direction of the edge — the prerequisite points TO the dependent course.
- Forgetting to initialize the queue with all 0-indegree nodes (not just node 0).
- Using a visited set instead of in-degrees and missing the cycle detection logic.

Key Takeaways

- Cycle in directed graph = cannot finish all courses.
- Kahn's algorithm doubles as topological sort: the order nodes leave the queue is one valid order.
- DFS with three states (white/gray/black) is an equally valid alternative.

---

47. Number of Connected Components in an Undirected Graph

Problem Summary

Given n nodes labeled 0 to n-1 and a list of undirected edges, return the number of connected components.

Real-Life Scenario

A social network with n users and friendship edges — count the number of disconnected friend groups.

Pattern Recognition

Clues that this is the right pattern:

- Connected components = either DFS/BFS launches needed, or union-find unions executed.

Brute Force Approach

Build adjacency list, then run DFS from each unvisited node. Increment count per launch. (This IS the standard solution; "brute force" here just means without union-find.)

Time complexity: O(V + E).

Space complexity: O(V + E).

Optimized Approach

Union-Find (Disjoint Set Union, DSU): initialize each node as its own parent.

For each edge [a, b], union a and b. If their roots differ, decrement a "components" counter starting at n.

Use path compression and union by rank for nearly O(1) operations.

Time complexity: Nearly O(V + E * alpha(V)) — alpha is the inverse Ackermann, basically constant.

Space complexity: O(V).

Dry Run

- n=5, edges=[[0,1],[1,2],[3,4]]. parent=[0,1,2,3,4], components=5.
- Union(0,1): roots 0,1 differ. parent[1]=0. components=4.
- Union(1,2): root of 1 is 0, root of 2 is 2. parent[2]=0. components=3.
- Union(3,4): roots 3,4 differ. parent[4]=3. components=2. Return 2.

Code Implementation

Java:

```JAVA
class Solution {
    public int countComponents(int n, int[][] edges) {
        int[] parent = new int[n];
        int[] rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;

        int components = n;
        for (int[] e : edges) {
            if (union(parent, rank, e[0], e[1])) components--;
        }
        return components;
```

```
    }

    private int find(int[] parent, int x) {
        if (parent[x] != x) parent[x] = find(parent, parent[x]); // path compression
        return parent[x];
    }

    private boolean union(int[] parent, int[] rank, int a, int b) {
        int ra = find(parent, a), rb = find(parent, b);
        if (ra == rb) return false; // already connected
        // union by rank
        if (rank[ra] < rank[rb]) { parent[ra] = rb; }
        else if (rank[ra] > rank[rb]) { parent[rb] = ra; }
        else { parent[rb] = ra; rank[ra]++; }
        return true;
    }
}
```

Python:

```
PYTHON
class Solution:
    def countComponents(self, n: int, edges) -> int:
        parent = list(range(n))
        rank = [0] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]  # path compression
                x = parent[x]
            return x

        def union(a, b):
            ra, rb = find(a), find(b)
            if ra == rb:
                return False
            if rank[ra] < rank[rb]:
                parent[ra] = rb
            elif rank[ra] > rank[rb]:
                parent[rb] = ra
            else:
                parent[rb] = ra
                rank[ra] += 1
            return True

        components = n
        for a, b in edges:
            if union(a, b):
                components -= 1
        return components
```

Common Mistakes

- Forgetting path compression — find becomes O(n) on degenerate trees.
- Decrementing the counter on every union call instead of only when the roots actually merge.
- Initializing parent incorrectly (e.g., all zeros instead of identity).

Key Takeaways

- Union-Find is the go-to for "count groups" and "are these connected?" questions.
- Each successful merge reduces components by 1 — that is the trick.
- DFS is also fine and simpler to write; pick union-find when edges arrive dynamically.

---

48. Graph Valid Tree

Problem Summary

Given n nodes and a list of undirected edges, decide whether the graph forms a valid tree. A tree has exactly n-1 edges, is connected, and contains no cycles.

Real-Life Scenario

A network admin checks whether a wiring diagram is a tree (one path between any two devices, no loops).

Pattern Recognition

Clues that this is the right pattern:

- Tree property: connected + no cycle + exactly n-1 edges.
- Use union-find or DFS.

Brute Force Approach

Run DFS to check connectivity and detect cycles separately. The optimized union-find is shorter.

Time complexity: O(V + E).

Space complexity: O(V + E).

Optimized Approach

Quick prune: if edges.length != n - 1, return false.

Union-Find: for each edge, try to union. If both endpoints already share a root, a cycle exists -> return false.

Otherwise, after processing all edges, the graph has n-1 edges and no cycle, so it must be connected — return true.

Time complexity: Nearly O(V + E).

Space complexity: O(V).

Dry Run

- n=5, edges=[[0,1],[0,2],[0,3],[1,4]]. edges.length=4=n-1 (good).
- Union(0,1), Union(0,2), Union(0,3), Union(1,4). All four merges happen — no shared roots ever. Return true.
- Counter-example: n=4, edges=[[0,1],[1,2],[2,0]] has 3 edges; but n-1=3 and a cycle exists -> Union(2,0) finds same root -> false.

Code Implementation

Java:

```java
class Solution {
    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false;

        int[] parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;

        for (int[] e : edges) {
            int ra = find(parent, e[0]);
            int rb = find(parent, e[1]);
            if (ra == rb) return false; // cycle
            parent[ra] = rb;
        }
        return true; // n-1 edges and no cycle => connected tree
    }

    private int find(int[] parent, int x) {
        if (parent[x] != x) parent[x] = find(parent, parent[x]);
        return parent[x];
    }
}
```

Python:

```python
class Solution:
    def validTree(self, n: int, edges) -> bool:
        if len(edges) != n - 1:
            return False

        parent = list(range(n))

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for a, b in edges:
            ra, rb = find(a), find(b)
            if ra == rb:
                return False  # cycle
            parent[ra] = rb
        return True
```

Common Mistakes

- Skipping the n-1 edge check — without it you must run a separate connectivity check.
- Treating self-loops [a,a] as "harmless" — they create cycles.

- Forgetting that the graph is undirected; treat edges symmetrically.

Key Takeaways

- A valid tree has exactly n-1 edges, no cycle, and is connected — all three follow once any two are checked.
- Union-Find with the n-1 prune gives the shortest correct solution.
- This is a classic DSU warmup; remember the technique.

---

13. Advanced Graphs

Topic Introduction

What it is: Advanced graph problems combine traversal with extra structure — ordering constraints, weighted edges, or reconstruction from indirect data. Topological sort is the centerpiece.

Why it exists: Real systems often need ordering (build dependencies, course planning) or shortest path under weights (maps). These call for more than plain BFS/DFS.

What problem it solves: Order tasks under dependencies, infer order from clues, find shortest weighted paths.

Typical time complexity: Topological sort: O(V + E). Dijkstra: O((V+E) log V).

Typical space complexity: O(V + E).

Real-life analogy: Cooking a multi-course meal: some dishes require others to be ready first. You build the schedule by repeatedly picking dishes whose prerequisites are done.

Real-world software engineering use cases:

- Build systems and package managers (npm, Maven).
- Spreadsheet recalculation order.
- Course scheduling apps.
- Decomposing tasks in project-management tools.

49. Alien Dictionary

Problem Summary

Given a list of words sorted lexicographically by an unknown alphabet, return any valid ordering of the alien letters. If no valid order exists, return "".

Real-Life Scenario

You receive a sorted dictionary in an unknown writing system and want to deduce the alphabet from word order alone.

Pattern Recognition

Clues that this is the right pattern:

- Pairs of adjacent words give constraints between two letters.
- These constraints form a directed graph; the letter order is its topological sort.

Brute Force Approach

Try every permutation of letters and check whether each adjacent word pair respects it — factorial explosion.

Time complexity: O(L!).

Space complexity: O(L).

Optimized Approach

For every adjacent pair (w1, w2), find the first index where they differ. That gives constraint w1[i] before w2[i]. Edge case: if w1 is longer than w2 and w2 is a prefix of w1, return "" immediately (invalid).

Build a directed graph and compute in-degrees over all distinct letters appearing in any word.

Run Kahn's topological sort. If the result includes every distinct letter, return it; else a cycle exists -> return "".

Time complexity: O(C) where C is the total characters across all words.

Space complexity: O(unique letters + edges).

Dry Run

- Words = ["wrt","wrf","er","ett","rftt"].
- Compare ("wrt","wrf"): t before f.
- Compare ("wrf","er"): w before e.
- Compare ("er","ett"): r before t.
- Compare ("ett","rftt"): e before r.
- Edges: t->f, w->e, r->t, e->r. Topo sort starts at w (in-degree 0). Then e, then r, then t, then f. Output: "wertf".

Code Implementation

Java:

```JAVA
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> adj = new HashMap<>();
        Map<Character, Integer> indegree = new HashMap<>();
        for (String w : words) {
            for (char c : w.toCharArray()) {
                indegree.putIfAbsent(c, 0);
                adj.putIfAbsent(c, new HashSet<>());
            }
        }

        // Build edges from adjacent pairs
        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i + 1];
            // Invalid case: prefix issue
            if (w1.length() > w2.length() && w1.startsWith(w2)) return "";
            for (int j = 0; j < Math.min(w1.length(), w2.length()); j++) {
                char a = w1.charAt(j), b = w2.charAt(j);
                if (a != b) {
                    if (!adj.get(a).contains(b)) {
```

```
                        adj.get(a).add(b);
                        indegree.put(b, indegree.get(b) + 1);
                    }
                    break; // only the first differing char gives info
                }
            }
        }

        Queue<Character> queue = new ArrayDeque<>();
        for (char c : indegree.keySet()) {
            if (indegree.get(c) == 0) queue.offer(c);
        }

        StringBuilder sb = new StringBuilder();
        while (!queue.isEmpty()) {
            char c = queue.poll();
            sb.append(c);
            for (char next : adj.get(c)) {
                indegree.put(next, indegree.get(next) - 1);
                if (indegree.get(next) == 0) queue.offer(next);
            }
        }

        if (sb.length() < indegree.size()) return ""; // cycle
        return sb.toString();
    }
}
```

Python:

```
PYTHON
from collections import defaultdict, deque

class Solution:
    def alienOrder(self, words):
        adj = {c: set() for w in words for c in w}
        indegree = {c: 0 for c in adj}

        for i in range(len(words) - 1):
            w1, w2 = words[i], words[i + 1]
            if len(w1) > len(w2) and w1.startswith(w2):
                return ""
            for a, b in zip(w1, w2):
                if a != b:
                    if b not in adj[a]:
                        adj[a].add(b)
                        indegree[b] += 1
                    break

        queue = deque([c for c in indegree if indegree[c] == 0])
        result = []
        while queue:
            c = queue.popleft()
```

```
            result.append(c)
            for nb in adj[c]:
                indegree[nb] -= 1
                if indegree[nb] == 0:
                    queue.append(nb)

        return "".join(result) if len(result) == len(indegree) else ""
```

Common Mistakes

- Forgetting to break after the first differing character — adding spurious edges.
- Not handling the prefix-too-long edge case ("abc" before "ab" is invalid).
- Adding duplicate edges and inflating in-degree.
- Forgetting to seed every distinct character, even ones with no constraints.

Key Takeaways

- "Order from clues" — topological sort.
- Each adjacent pair gives at most one new constraint (the first differing letter).
- Always validate the result length against the number of distinct letters to catch cycles.

---

14. Dynamic Programming (1-D)

Topic Introduction

What it is: Dynamic Programming (DP) solves a problem by breaking it into overlapping subproblems and storing each subproblem's answer so it is computed only once. 1-D DP means the state is described by a single index.

Why it exists: Plain recursion often recomputes the same subproblems exponentially many times. DP stores answers in a table or a few variables, turning exponential time into linear or polynomial.

What problem it solves: Counting paths, picking optimum subsets, parsing strings, sequence questions where each step depends on a few previous steps.

Typical time complexity: Usually O(n) for 1-D problems with constant transitions, or O(n * k) when each step looks at k options.

Typical space complexity: O(n) for a table; often optimizable to O(1) by keeping only the last few values.

Real-life analogy: Climbing stairs by remembering how many ways you got to each step. By the time you reach step n, the answer is just step (n-1) plus step (n-2) — no need to recount from scratch.

Real-world software engineering use cases:

- Resource allocation (knapsack variants).
- Edit-distance for spell-checkers and DNA sequence alignment.
- Compiler register allocation and code generation.
- Game scoring and AI evaluation functions.
- Financial planning calculators (compound choices over time).

50. Climbing Stairs

Problem Summary

You are climbing a staircase with n steps. Each move goes up 1 or 2 steps. Return how many distinct ways you can reach the top.

Real-Life Scenario

A simple game where each level lets you advance 1 or 2 levels — count winning sequences.

Pattern Recognition

Clues that this is the right pattern:

- "How many ways to reach state n?" Often Fibonacci-shaped.
- ways(n) = ways(n-1) + ways(n-2): the last move was either 1 step or 2 steps.

Brute Force Approach

Recursion: climb(n) = climb(n-1) + climb(n-2). Without memoization, recomputes the same subproblems.

Time complexity: O(2^n).

Space complexity: O(n) recursion.

Optimized Approach

Bottom-up: dp[0]=1, dp[1]=1, dp[i] = dp[i-1] + dp[i-2].

Space-optimize: only the last two values matter — use two variables.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- n=4. a=1 (ways for 0), b=1 (ways for 1).
- i=2: c=a+b=2. Shift: a=1, b=2.
- i=3: c=3. Shift: a=2, b=3.
- i=4: c=5. Return 5.

Code Implementation

Java:

```JAVA
class Solution {
    public int climbStairs(int n) {
        if (n <= 1) return 1;
        int a = 1, b = 1;
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
}
```

Python:

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        if n <= 1:
            return 1
        a, b = 1, 1
        for _ in range(2, n + 1):
            a, b = b, a + b
        return b
```

Common Mistakes

- Counting sequences as if 1+2 and 2+1 are the same — they are different ways.
- Off-by-one with base cases (climbStairs(0)=1, climbStairs(1)=1).
- Solving with O(n) memoized recursion when O(1) iteration is just as easy.

Key Takeaways

- Identify the recurrence first, then optimize.
- Many counting DPs collapse to two variables once you notice only the last two states matter.
- This is your gateway DP problem — internalize the pattern.

---

51. House Robber

Problem Summary

Houses are arranged in a line, each with a non-negative amount of money. You cannot rob two adjacent houses. Return the maximum amount you can rob.

Real-Life Scenario

A scheduling problem: pick non-adjacent time slots that maximize total reward.

Pattern Recognition

Clues that this is the right pattern:

- "Pick non-adjacent items, maximize sum" — classic 1-D DP.
- At each house: rob it (and add money + dp[i-2]) or skip it (dp[i-1]).

Brute Force Approach

Try every subset of non-adjacent houses — exponential.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

dp[i] = max(dp[i-1], dp[i-2] + nums[i]).

Space-optimize: keep prev1 (dp[i-1]) and prev2 (dp[i-2]).

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[2,7,9,3,1]. prev2=0, prev1=0.
- i=0 (2): cur=max(0,0+2)=2. prev2=0, prev1=2.
- i=1 (7): cur=max(2,0+7)=7. prev2=2, prev1=7.
- i=2 (9): cur=max(7,2+9)=11. prev2=7, prev1=11.
- i=3 (3): cur=max(11,7+3)=11. prev2=11, prev1=11.
- i=4 (1): cur=max(11,11+1)=12. Return 12.

Code Implementation

Java:

```JAVA
class Solution {
    public int rob(int[] nums) {
        int prev2 = 0; // dp[i-2]
        int prev1 = 0; // dp[i-1]
        for (int num : nums) {
            int cur = Math.max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

Python:

```PYTHON
class Solution:
    def rob(self, nums) -> int:
        prev2, prev1 = 0, 0
        for num in nums:
            cur = max(prev1, prev2 + num)
            prev2, prev1 = prev1, cur
        return prev1
```

Common Mistakes

- Greedy "pick every other house" — fails when small houses surround large ones.
- Forgetting to compare with prev1 (skip option) and just always adding nums[i].
- Mismanaging variable updates — using cur after overwriting prev1.

Key Takeaways

- At each step you have two choices: take or skip — pick the better.
- Two rolling variables suffice — no need for an array.
- This pattern reappears as a building block in many other DP problems.

---

52. House Robber II

Problem Summary

Same as House Robber, but houses are arranged in a circle. The first and last houses are now adjacent. Return the maximum money you can rob.

Real-Life Scenario

Same scheduling idea, but the calendar wraps — slot 23:00 is adjacent to slot 00:00.

Pattern Recognition

Clues that this is the right pattern:

- Circular constraint -> split into two linear problems and take the better.

Brute Force Approach

Same exponential subset enumeration with the extra adjacency.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

Either rob the first house (then skip the last) or skip the first (then last is allowed).

Run the linear House Robber on nums[0..n-2] and on nums[1..n-1]; return the max.

Edge case: n == 1, return nums[0].

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[2,3,2]. n=3.
- Linear on [2,3] -> 3.
- Linear on [3,2] -> 3.
- Return max(3,3) = 3 (cannot rob both 2s because they are now adjacent).

Code Implementation

Java:

```JAVA
class Solution {
    public int rob(int[] nums) {
        if (nums.length == 1) return nums[0];
        return Math.max(
            robLinear(nums, 0, nums.length - 2), // skip last
            robLinear(nums, 1, nums.length - 1)  // skip first
        );
    }

    private int robLinear(int[] nums, int lo, int hi) {
        int prev2 = 0, prev1 = 0;
        for (int i = lo; i <= hi; i++) {
            int cur = Math.max(prev1, prev2 + nums[i]);
            prev2 = prev1;
```

```
            prev1 = cur;
        }
        return prev1;
    }
}
```

Python:

```
PYTHON
class Solution:
    def rob(self, nums) -> int:
        if len(nums) == 1:
            return nums[0]

        def linear(arr):
            prev2, prev1 = 0, 0
            for num in arr:
                cur = max(prev1, prev2 + num)
                prev2, prev1 = prev1, cur
            return prev1

        return max(linear(nums[:-1]), linear(nums[1:]))
```

Common Mistakes

- Forgetting the n=1 edge case (slicing nums[:-1] gives an empty array).
- Trying to handle circular adjacency with one combined DP — much harder than two linear runs.

Key Takeaways

- Circular DP: solve two linear cases and take the maximum.
- Reuse working code by extracting the linear solver into its own function.

---

53. Longest Palindromic Substring

Problem Summary

Given a string s, return the longest palindromic substring (a substring that reads the same forwards and backwards).

Real-Life Scenario

A text-search engine highlighting palindromes for a wordplay app.

Pattern Recognition

Clues that this is the right pattern:

- Expand around center: every palindrome has a center (a character or a gap between two characters).
- For each of the 2n-1 centers, expand outward while characters match.

Brute Force Approach

Check every substring (O(n^2) of them) and verify each is a palindrome (O(n)). Total O(n^3).

Time complexity: O(n^3).

Space complexity: O(1).

Optimized Approach

For each index i, expand twice: once with center at i (odd length), once between i and i+1 (even length).

Track the longest palindrome found.

Time complexity: O(n^2) — there are 2n-1 centers, each expansion at most O(n).

Space complexity: O(1).

Dry Run

- s="babad". i=0 -> "b", "" (gap). Best="b".
- i=1: odd center "b" expands to "bab" (length 3). Best="bab".
- i=2: odd center "a" expands to "aba" (length 3). Tie.
- i=3,4: shorter. Return "bab" or "aba" — both are valid.

Code Implementation

Java:

```JAVA
class Solution {
    public String longestPalindrome(String s) {
        int start = 0, maxLen = 1;
        for (int i = 0; i < s.length(); i++) {
            int len1 = expand(s, i, i);     // odd
            int len2 = expand(s, i, i + 1); // even
            int len = Math.max(len1, len2);
            if (len > maxLen) {
                maxLen = len;
                start = i - (len - 1) / 2; // recover left boundary
            }
        }
        return s.substring(start, start + maxLen);
    }

    private int expand(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1; // length of palindrome found
    }
}
```

Python:

```PYTHON
class Solution:
    def longestPalindrome(self, s: str) -> str:
        def expand(l, r):
            while l >= 0 and r < len(s) and s[l] == s[r]:
```

```
                l -= 1
                r += 1
            return l + 1, r - 1  # inclusive bounds

        start, end = 0, 0
        for i in range(len(s)):
            l1, r1 = expand(i, i)
            l2, r2 = expand(i, i + 1)
            if r1 - l1 > end - start:
                start, end = l1, r1
            if r2 - l2 > end - start:
                start, end = l2, r2
        return s[start:end + 1]
```

Common Mistakes

- Only expanding for odd-length palindromes — missing even-length ones like "abba".
- Recomputing length incorrectly after the loop exits — the loop ends when characters do not match, so subtract 1.
- Off-by-one when computing the start index from a center and length.

Key Takeaways

- 2n-1 centers covers every possible palindrome.
- Manacher's algorithm achieves O(n) but is complex; expand-around-center is the interview default.
- Always handle both odd and even cases.

---

54. Palindromic Substrings

Problem Summary

Return the number of palindromic substrings in s. Different start/end indices count as different substrings even if the content is identical.

Real-Life Scenario

Same wordplay app — now counting all palindromes for a stats panel.

Pattern Recognition

Clues that this is the right pattern:

- Same expand-around-center as Longest Palindromic Substring; just count instead of tracking the longest.

Brute Force Approach

Check every substring for palindrome property — O(n^3).

Time complexity: O(n^3).

Space complexity: O(1).

Optimized Approach

Each successful expansion step adds one palindrome.

For each center (odd and even), increment count while characters match.

Time complexity: O(n^2).

Space complexity: O(1).

Dry Run

- s="aaa". i=0: odd center "a" -> count=1. Even center "" -> 0.
- i=1: odd center "a" -> 1. Expand to "aaa" -> +1. Even center "aa" -> 1.
- i=2: odd "a" -> 1. Even "" -> 0.
- Total = 6.

Code Implementation

Java:

```JAVA
class Solution {
    public int countSubstrings(String s) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            count += expand(s, i, i);     // odd
            count += expand(s, i, i + 1); // even
        }
        return count;
    }

    private int expand(String s, int left, int right) {
        int count = 0;
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            count++;
            left--;
            right++;
        }
        return count;
    }
}
```

Python:

```PYTHON
class Solution:
    def countSubstrings(self, s: str) -> int:
        def expand(l, r):
            count = 0
            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1
                l -= 1
                r += 1
            return count

        return sum(expand(i, i) + expand(i, i + 1) for i in range(len(s)))
```

Common Mistakes

- Counting unique strings instead of substring positions.
- Forgetting even-length centers.

Key Takeaways

- Same expand-around-center technique, different reduction (count vs length).
- Each successful match in the loop is one more palindrome.

---

55. Decode Ways

Problem Summary

A message is encoded with mapping A=1..Z=26. Given a string of digits, return the number of ways to decode it.

Real-Life Scenario

A legacy cipher where users send numeric codes — count valid interpretations.

Pattern Recognition

Clues that this is the right pattern:

- At each position you may take 1 digit (if not "0") or 2 digits (if it forms 10..26).
- dp[i] = number of ways to decode the first i characters; transitions from dp[i-1] and dp[i-2].

Brute Force Approach

Recursive enumeration without memo — exponential.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

dp[0]=1 (empty string has one decoding). dp[1]=1 if s[0]!='0', else 0.

For i >= 2: if s[i-1] != '0', add dp[i-1]. Then check if s[i-2..i-1] is between 10 and 26; if so, add dp[i-2].

Space-optimize to two variables.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- s="226". dp[0]=1. dp[1]=1 ('2' valid).
- i=2: '2' valid -> dp[2] += dp[1] = 1. "22" in [10..26] -> dp[2] += dp[0] = 2.
- i=3: '6' valid -> dp[3] += dp[2] = 2. "26" in [10..26] -> dp[3] += dp[1] = 3. Return 3.

Code Implementation

Java:

```
JAVA
class Solution {
    public int numDecodings(String s) {
```

```
        int n = s.length();
        if (n == 0 || s.charAt(0) == '0') return 0;

        int prev2 = 1; // dp[i-2]
        int prev1 = 1; // dp[i-1]

        for (int i = 2; i <= n; i++) {
            int cur = 0;
            // single digit
            if (s.charAt(i - 1) != '0') cur += prev1;
            // two digits
            int two = Integer.parseInt(s.substring(i - 2, i));
            if (two >= 10 && two <= 26) cur += prev2;

            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

Python:

```
PYTHON
class Solution:
    def numDecodings(self, s: str) -> int:
        if not s or s[0] == '0':
            return 0

        prev2, prev1 = 1, 1
        for i in range(2, len(s) + 1):
            cur = 0
            if s[i-1] != '0':
                cur += prev1
            two = int(s[i-2:i])
            if 10 <= two <= 26:
                cur += prev2
            prev2, prev1 = prev1, cur
        return prev1
```

Common Mistakes

- Treating "0" as decodable on its own.
- Treating "06" as a valid two-digit decoding (it is not — must be in 10..26).
- Wrong base case: dp[0] = 1 is the conventional empty-string base.

Key Takeaways

- Edge cases dominate this problem — read "0" rules carefully.
- Two transitions per step (1-digit, 2-digit) -> two-variable DP.

---

56. Coin Change

Problem Summary

Given coin denominations and a target amount, return the fewest number of coins to make the amount, or -1 if impossible.

Real-Life Scenario

A vending machine paying out change with as few coins as possible.

Pattern Recognition

Clues that this is the right pattern:

- Unbounded knapsack — coins can be reused.
- dp[a] = minimum coins to make amount a. dp[a] = min over coins c of dp[a-c] + 1.

Brute Force Approach

Recurse: try each coin, take the minimum +1. Without memo, recomputes amounts repeatedly.

Time complexity: O(coins^amount).

Space complexity: O(amount).

Optimized Approach

Bottom-up: dp array of size amount+1, initialized to a sentinel (amount+1 acts as infinity).

dp[0]=0. For a from 1 to amount: for each coin c <= a, dp[a] = min(dp[a], dp[a-c] + 1).

Return dp[amount] if it is < sentinel, else -1.

Time complexity: O(amount * coins).

Space complexity: O(amount).

Dry Run

- coins=[1,2,5], amount=11. dp[0]=0.
- dp[1]=dp[0]+1=1. dp[2]=min(dp[1]+1, dp[0]+1)=1.
- dp[3]=min(dp[2]+1, dp[1]+1)=2. dp[4]=2. dp[5]=min(dp[4]+1,dp[3]+1,dp[0]+1)=1.
- ... dp[11]=3 (5+5+1). Return 3.

Code Implementation

Java:

```JAVA
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1); // sentinel = "infinity"
        dp[0] = 0;

        for (int a = 1; a <= amount; a++) {
            for (int c : coins) {
                if (c <= a) {
                    dp[a] = Math.min(dp[a], dp[a - c] + 1);
                }
```

```
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

Python:

```
PYTHON
class Solution:
    def coinChange(self, coins, amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for a in range(1, amount + 1):
            for c in coins:
                if c <= a:
                    dp[a] = min(dp[a], dp[a - c] + 1)
        return dp[amount] if dp[amount] <= amount else -1
```

Common Mistakes

- Greedy ("always pick the biggest coin") — works for some coin sets but not in general.
- Returning 0 instead of -1 when no combination works.
- Using Integer.MAX_VALUE as sentinel and overflowing when adding 1.

Key Takeaways

- Unbounded knapsack: outer loop over amount, inner loop over coins. Each coin reusable.
- Use amount+1 as a safe sentinel — adding 1 will not overflow.
- Greedy fails on coins like [1,3,4] amount=6 (greedy picks 4+1+1=3 coins, optimal is 3+3=2).

---

57. Maximum Product Subarray

Problem Summary

Given an integer array, return the largest product of any contiguous non-empty subarray.

Real-Life Scenario

Compounded daily gains/losses on an investment — find the best contiguous streak.

Pattern Recognition

Clues that this is the right pattern:

- Maximum product is tricky because two negatives multiply to a large positive.
- Track both maxSoFar and minSoFar at each index. The current min may become the future max if multiplied by a negative.

Brute Force Approach

For every pair (i, j), compute the product. O(n^2).

Time complexity: O(n^2).

Space complexity: O(1).

Optimized Approach

Iterate, keeping curMax and curMin ending at current index.

If nums[i] is negative, swap curMax and curMin first.

curMax = max(nums[i], curMax * nums[i]). curMin = min(nums[i], curMin * nums[i]).

Update global max with curMax.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[2,3,-2,4]. curMax=2, curMin=2, ans=2.
- i=1 (3): curMax=max(3,6)=6, curMin=min(3,6)=3. ans=6.
- i=2 (-2): swap -> curMax=3, curMin=6. curMax=max(-2,-6)=-2, curMin=min(-2,-12)=-12. ans=6.
- i=3 (4): curMax=max(4,-8)=4, curMin=min(4,-48)=-48. ans=6. Return 6.

Code Implementation

Java:

```JAVA
class Solution {
    public int maxProduct(int[] nums) {
        int curMax = nums[0];
        int curMin = nums[0];
        int ans = nums[0];

        for (int i = 1; i < nums.length; i++) {
            int n = nums[i];
            if (n < 0) {
                int tmp = curMax;
                curMax = curMin;
                curMin = tmp;
            }
            curMax = Math.max(n, curMax * n);
            curMin = Math.min(n, curMin * n);
            ans = Math.max(ans, curMax);
        }
        return ans;
    }
}
```

Python:

```PYTHON
class Solution:
    def maxProduct(self, nums) -> int:
        cur_max = cur_min = ans = nums[0]
        for n in nums[1:]:
            if n < 0:
                cur_max, cur_min = cur_min, cur_max
```

```
            cur_max = max(n, cur_max * n)
            cur_min = min(n, cur_min * n)
            ans = max(ans, cur_max)
        return ans
```

Common Mistakes

- Tracking only the running max — fails when a negative multiplies a large negative.
- Forgetting to compare n alone with the products — sometimes restarting fresh wins.
- Initializing ans to 0 — fails on all-negative arrays.

Key Takeaways

- "Max product" needs both running max and running min.
- Always reset by comparing the current element alone — handles zeros.
- Different from Maximum Subarray (sum) — products have sign-flip issues.

---

58. Word Break

Problem Summary

Given a string s and a dictionary of words wordDict, return true if s can be segmented into one or more words from wordDict.

Real-Life Scenario

A search engine deciding whether to interpret a query string as a sequence of meaningful words.

Pattern Recognition

Clues that this is the right pattern:

- Substring decision DP. dp[i] = "is s[0..i-1] segmentable?".
- For each i, check every j < i: if dp[j] is true and s[j..i-1] is in the dictionary, set dp[i]=true.

Brute Force Approach

Recursively try every prefix in the dictionary; recurse on the suffix. Without memo, explodes.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

Use a HashSet for O(1) word lookup.

dp[0] = true (empty string is segmentable).

For i = 1..n: for each j = 0..i-1, if dp[j] && set.contains(s.substring(j, i)), dp[i] = true. Break.

Time complexity: O(n^2 * L) where L is the substring construction cost.

Space complexity: O(n + dict).

Dry Run

- s="leetcode", dict={"leet","code"}. dp[0]=true.

- i=4: j=0, dp[0]=true and "leet" in dict -> dp[4]=true.
- i=8: j=4, dp[4]=true and "code" in dict -> dp[8]=true. Return true.

Code Implementation

Java:

```JAVA
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> set = new HashSet<>(wordDict);
        int n = s.length();
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && set.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }
}
```

Python:

```PYTHON
class Solution:
    def wordBreak(self, s: str, wordDict) -> bool:
        words = set(wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True

        for i in range(1, n + 1):
            for j in range(i):
                if dp[j] and s[j:i] in words:
                    dp[i] = True
                    break
        return dp[n]
```

Common Mistakes

- Not putting the dictionary in a set — O(L * D) per lookup ruins performance.
- Forgetting to break inner loop after a true assignment — minor speedup but cleaner.
- Mixing index conventions between dp[i] meaning "first i chars" and substring(j, i).

Key Takeaways

- Decision DPs over substrings often follow this two-pointer template.
- A HashSet for constant-time dictionary checks is standard.

- dp[0]=true is the empty-string base case — easy to forget.

---

59. Longest Increasing Subsequence

Problem Summary

Given an integer array nums, return the length of the longest strictly increasing subsequence.

Real-Life Scenario

Telemetry analysis: longest streak of strictly increasing values across non-contiguous points.

Pattern Recognition

Clues that this is the right pattern:

- Subsequence (not substring) — pick indices i1<i2<...<ik with strictly increasing values.
- Two solutions: O(n^2) DP, or O(n log n) with patience-sort idea.

Brute Force Approach

For each subset, check if it is increasing — exponential.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

O(n^2): dp[i] = LIS ending at index i. dp[i] = 1 + max(dp[j]) for j < i with nums[j] < nums[i]; default 1.

O(n log n): maintain a "tails" array. For each num, find the leftmost element in tails >= num and replace it; if num is larger than all, append. Length of tails is the LIS length (the array itself is not necessarily a valid LIS).

Time complexity: O(n log n).

Space complexity: O(n).

Dry Run

- nums=[10,9,2,5,3,7,101,18]. tails=[].
- 10 -> tails=[10]. 9: replace 10 -> [9]. 2: replace 9 -> [2]. 5: append -> [2,5].
- 3: replace 5 -> [2,3]. 7: append -> [2,3,7]. 101: append -> [2,3,7,101].
- 18: replace 101 -> [2,3,7,18]. Length 4 — return 4.

Code Implementation

Java:

```
JAVA
class Solution {
    public int lengthOfLIS(int[] nums) {
        // O(n log n) patience sorting approach
        int[] tails = new int[nums.length];
        int size = 0;
        for (int num : nums) {
            int lo = 0, hi = size;
            while (lo < hi) {
```

```
                int mid = (lo + hi) / 2;
                if (tails[mid] < num) lo = mid + 1;
                else hi = mid;
            }
            tails[lo] = num;
            if (lo == size) size++;
        }
        return size;
    }
}
```

Python:

```
PYTHON
from bisect import bisect_left

class Solution:
    def lengthOfLIS(self, nums) -> int:
        tails = []
        for num in nums:
            i = bisect_left(tails, num)
            if i == len(tails):
                tails.append(num)
            else:
                tails[i] = num
        return len(tails)
```

Common Mistakes

- Returning the tails array as the LIS — it is not necessarily a valid increasing subsequence; only its length is correct.
- Using bisect_right instead of bisect_left — gives non-decreasing instead of strictly increasing.
- Off-by-one in the manual binary search.

Key Takeaways

- O(n log n) LIS uses binary search to maintain a sorted "best-so-far" array.
- The O(n^2) DP is easier to understand and adequate for small inputs.
- Subsequence != substring — items don't need to be contiguous.

---

15. Dynamic Programming (2-D)

Topic Introduction

What it is: 2-D DP describes a state with two indices, usually two pointers into two sequences or a position on a grid. The DP table is a 2-D matrix.

Why it exists: When a problem compares two sequences (strings, arrays) or moves on a grid, one index is not enough to capture the state. A second dimension is required.

What problem it solves: Edit distance, LCS, grid paths, matching problems, knapsack with weight + index dimensions.

Typical time complexity: Usually O(m * n).

Typical space complexity: O(m * n); often optimizable to O(min(m, n)) by keeping only the previous row.

Real-life analogy: Filling a crossword grid one square at a time. Each square depends on the squares above and to its left — by the time you reach the bottom-right, the answer is built up from neighbors.

Real-world software engineering use cases:

- Diff tools (edit distance) for comparing files or DNA sequences.
- Spell-checkers (suggest corrections by edit distance).
- Game scoring on grid-based boards.
- Image-processing tasks like seam carving.

60. Unique Paths

Problem Summary

A robot starts at the top-left of an m x n grid and wants to reach the bottom-right, moving only right or down. Return the number of unique paths.

Real-Life Scenario

A delivery drone in a Manhattan-style grid counts how many shortest routes connect two points.

Pattern Recognition

Clues that this is the right pattern:

- Grid path counting with restricted moves -> 2-D DP.
- paths(r, c) = paths(r-1, c) + paths(r, c-1).

Brute Force Approach

Recursive enumeration of all paths — exponential.

Time complexity: O(2^(m+n)).

Space complexity: O(m + n).

Optimized Approach

Bottom-up: dp[r][c] = dp[r-1][c] + dp[r][c-1].

Initialize first row and first column to 1 (only one way to reach them).

Space-optimize to a single 1-D row updated in place.

Time complexity: O(m * n).

Space complexity: O(n) with row optimization.

Dry Run

- m=3, n=2. row=[1,1].
- r=1: row[0] stays 1. row[1]=row[0]+row[1]=1+1=2. row=[1,2].
- r=2: row[0]=1. row[1]=1+2=3. row=[1,3]. Return row[1]=3.

Code Implementation

Java:

```java
class Solution {
    public int uniquePaths(int m, int n) {
        int[] row = new int[n];
        Arrays.fill(row, 1);
        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                row[c] = row[c] + row[c - 1];
            }
        }
        return row[n - 1];
    }
}
```

Python:

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        row = [1] * n
        for _ in range(1, m):
            for c in range(1, n):
                row[c] += row[c - 1]
        return row[-1]
```

Common Mistakes

- Forgetting to initialize the first row and column to 1.
- Indexing into dp[r-1][c-1] when dp[r][c] = dp[r-1][c] + dp[r][c-1] is the right recurrence.
- Allocating the full 2-D matrix when the rolling 1-D array is shorter and faster.

Key Takeaways

- Closed-form alternative: C(m+n-2, m-1). Useful to mention in interviews even if you implement DP.
- Rolling 1-D arrays are a common space optimization for 2-D DP.

---

61. Longest Common Subsequence

Problem Summary

Given two strings text1 and text2, return the length of their longest common subsequence (a sequence appearing in both, not necessarily contiguous).

Real-Life Scenario

A diff tool aligning two versions of a file finds the LCS to highlight unchanged regions.

Pattern Recognition

Clues that this is the right pattern:

- Two strings + "common" / "matching" -> 2-D DP indexed by both lengths.
- If characters match, dp[i][j] = dp[i-1][j-1] + 1; else dp[i][j] = max(dp[i-1][j], dp[i][j-1]).

Brute Force Approach

Recursive comparison without memo — exponential.

Time complexity: O(2^(m+n)).

Space complexity: O(m + n).

Optimized Approach

dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1].

Build the (m+1) x (n+1) table from top-left to bottom-right.

Return dp[m][n].

Time complexity: O(m * n).

Space complexity: O(m * n).

Dry Run

- text1="abcde", text2="ace". Build 6x4 table.
- i=1 j=1 (a vs a) match -> dp[1][1]=1.
- i=2 j=2 (b vs c) no -> dp[2][2]=max(dp[1][2],dp[2][1])=1.
- i=3 j=2 (c vs c) match -> dp[3][2]=dp[2][1]+1=2.
- i=5 j=3 (e vs e) match -> dp[5][3]=dp[4][2]+1=3. Return 3.

Code Implementation

Java:

```JAVA
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

Python:

```PYTHON
```

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j] = dp[i-1][j-1] + 1
                else:
                    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
        return dp[m][n]
```

Common Mistakes

- Confusing subsequence with substring — substring requires contiguity, subsequence does not.
- Off-by-one between string indices (0-based) and DP indices (1-based to leave room for empty-string row/col).
- Returning dp[m-1][n-1] instead of dp[m][n].

Key Takeaways

- Two-string DP almost always uses an (m+1) x (n+1) table with dp[0][*]=dp[*][0]=0.
- The two transitions — match-then-shrink-both vs no-match-take-best-of-shrinking-one — show up in many string DP problems.
- Edit distance, shortest common supersequence, and others share this skeleton.

---

16. Greedy

Topic Introduction

What it is: Greedy algorithms make the locally optimal choice at each step, hoping it leads to a global optimum. Greedy works only when the problem has a structure that guarantees this — usually proven by an exchange argument.

Why it exists: When greedy works, it is dramatically simpler and faster than DP — often O(n) instead of O(n^2).

What problem it solves: Interval scheduling, currency change in canonical systems, Huffman coding, MST (Kruskal/Prim), sometimes maximum subarray-style problems.

Typical time complexity: Usually O(n) or O(n log n) (when sorting is needed).

Typical space complexity: O(1) or O(n).

Real-life analogy: Tipping at a restaurant: the simple rule "round up to the nearest dollar" is greedy. It is fast and good enough — though sometimes a more careful calculation would be more optimal.

Real-world software engineering use cases:

- Activity scheduling and meeting-room allocation.
- Huffman coding for compression.
- Network routing protocols.

- Caching policies (e.g., LRU is essentially a greedy heuristic).

62. Maximum Subarray

Problem Summary

Given an integer array, return the largest sum of any contiguous non-empty subarray.

Real-Life Scenario

Daily profit/loss series: find the most profitable contiguous streak.

Pattern Recognition

Clues that this is the right pattern:

- Kadane's algorithm: at each index, the best subarray ending here is either just nums[i] alone, or nums[i] extending the best subarray ending at i-1.
- cur = max(num, cur + num); best = max(best, cur).

Brute Force Approach

Sum every contiguous subarray — O(n^2) with prefix sums, O(n^3) without.

Time complexity: O(n^2).

Space complexity: O(1).

Optimized Approach

Maintain cur (best sum ending here) and best (overall best).

For each num: if cur is negative, drop it and start fresh at num.

Otherwise, extend: cur += num. Update best.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[-2,1,-3,4,-1,2,1,-5,4]. cur=-2, best=-2.
- 1: cur=max(1,-1)=1. best=1.
- -3: cur=max(-3,-2)=-2. best=1.
- 4: cur=max(4,2)=4. best=4.
- -1: cur=3. best=4. 2: cur=5. best=5. 1: cur=6. best=6.
- -5: cur=1. best=6. 4: cur=5. best=6. Return 6.

Code Implementation

Java:

```JAVA
class Solution {
    public int maxSubArray(int[] nums) {
        int cur = nums[0];
        int best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);
```

```
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Python:

```
PYTHON
class Solution:
    def maxSubArray(self, nums) -> int:
        cur = best = nums[0]
        for num in nums[1:]:
            cur = max(num, cur + num)
            best = max(best, cur)
        return best
```

Common Mistakes

- Initializing best to 0 — fails on all-negative arrays (the answer must be a non-empty subarray).
- Resetting cur to 0 instead of nums[i] when cur becomes negative.
- Confusing this with Maximum Product Subarray — products need both running max and min.

Key Takeaways

- Kadane's is the canonical "maximum contiguous sum" algorithm.
- The decision is binary: extend or restart. The smaller of the two losses wins.
- O(1) space — beautifully simple.

---

63. Jump Game

Problem Summary

Given a non-negative integer array, where each element represents your maximum jump length from that position, return true if you can reach the last index.

Real-Life Scenario

A tile-jumping puzzle: each tile lists the max distance you can jump from it. Can you reach the end?

Pattern Recognition

Clues that this is the right pattern:

- Greedy reachability. Track farthest = the farthest index reachable so far.
- For each i from 0..n-1: if i > farthest, return false. Else update farthest = max(farthest, i + nums[i]).

Brute Force Approach

BFS / DFS over reachable indices — works but may explode.

Time complexity: O(n^2) in the worst case.

Space complexity: O(n).

Optimized Approach

One linear pass with a single integer "farthest".

Stop early if farthest >= last index.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[2,3,1,1,4]. farthest=0.
- i=0: 0<=0, farthest=max(0,2)=2.
- i=1: 1<=2, farthest=max(2,4)=4. 4>=4 -> return true.
- Counter-example: nums=[3,2,1,0,4]. farthest grows to 3 by i=2; at i=3, farthest still 3, but 0+i=3 stays. At i=4, 4>3 -> return false.

Code Implementation

Java:

```java
class Solution {
    public boolean canJump(int[] nums) {
        int farthest = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i > farthest) return false; // unreachable
            farthest = Math.max(farthest, i + nums[i]);
            if (farthest >= nums.length - 1) return true;
        }
        return true;
    }
}
```

Python:

```python
class Solution:
    def canJump(self, nums) -> bool:
        farthest = 0
        for i, num in enumerate(nums):
            if i > farthest:
                return False
            farthest = max(farthest, i + num)
            if farthest >= len(nums) - 1:
                return True
        return True
```

Common Mistakes

- Trying every jump length recursively without memo — explodes.
- Using "i + nums[i] >= len-1" as the only check — must also verify i is reachable.
- Returning false too early before processing the whole array (or until we are sure we cannot continue).

Key Takeaways

- Greedy reachability replaces DP/BFS for this problem.
- Check "current position reachable" before advancing.
- Same idea generalizes to Jump Game II (minimum jumps).

---

17. Intervals

Topic Introduction

What it is: An interval is a pair [start, end] representing a range. Interval problems ask about overlap, merging, scheduling, or counting non-overlaps.

Why it exists: Time-based scheduling and 1-D range queries (calendars, meeting rooms, network packets) appear constantly in real systems.

What problem it solves: Detect overlaps, merge ranges, schedule resources, find a maximum set of non-overlapping intervals.

Typical time complexity: Most interval problems are O(n log n) due to a sort, then O(n) sweep.

Typical space complexity: O(n).

Real-life analogy: Booking meeting rooms in a calendar app. Each meeting is an interval; you decide whether two meetings can share a room (no overlap) or need separate rooms (overlap).

Real-world software engineering use cases:

- Calendar apps detecting conflicts.
- Network packet scheduling.
- Database transaction concurrency control.
- CPU job scheduling.
- Ad slot allocation across time.

64. Insert Interval

Problem Summary

Given a list of non-overlapping intervals sorted by start time, insert a new interval and merge any overlaps. Return the resulting list.

Real-Life Scenario

Adding a new meeting to an already-sorted calendar and merging it with any meetings it overlaps.

Pattern Recognition

Clues that this is the right pattern:

- Three phases: copy intervals strictly before the new one, merge overlapping ones, copy intervals strictly after.

Brute Force Approach

Append the new interval, sort the whole list, and re-merge — O(n log n).

Time complexity: O(n log n).

Space complexity: O(n).

Optimized Approach

Walk through intervals once.

While intervals[i].end < newInterval.start, append intervals[i].

While intervals[i].start <= newInterval.end, merge: newInterval.start = min(...); newInterval.end = max(...).

Append newInterval, then append the rest.

Time complexity: O(n).

Space complexity: O(n).

Dry Run

- intervals=[[1,3],[6,9]], newInterval=[2,5]. result=[].
- i=0: end=3, newStart=2 -> overlap. Merge: newInterval=[1,5]. i=1.
- i=1: start=6 > newEnd=5. Push newInterval [1,5]. Then push [6,9].
- result=[[1,5],[6,9]].

Code Implementation

Java:

```JAVA
class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> result = new ArrayList<>();
        int i = 0, n = intervals.length;

        // Phase 1: intervals strictly before newInterval
        while (i < n && intervals[i][1] < newInterval[0]) {
            result.add(intervals[i++]);
        }
        // Phase 2: merge overlapping
        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
            newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
            i++;
        }
        result.add(newInterval);
        // Phase 3: intervals strictly after
        while (i < n) {
            result.add(intervals[i++]);
        }
        return result.toArray(new int[0][]);
    }
}
```

Python:

```PYTHON
```

```
class Solution:
    def insert(self, intervals, newInterval):
        result = []
        i, n = 0, len(intervals)

        while i < n and intervals[i][1] < newInterval[0]:
            result.append(intervals[i])
            i += 1
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1
        result.append(newInterval)
        while i < n:
            result.append(intervals[i])
            i += 1

        return result
```

Common Mistakes

- Treating <= and < inconsistently for the boundary cases.
- Forgetting to add the merged newInterval after the merge phase.
- Mutating intervals[i] when you only meant to mutate newInterval.

Key Takeaways

- Sorted input is the gift — exploit it with a single pass and three phases.
- Mutate the newInterval as you absorb overlaps; do not allocate a new one each time.

---

65. Merge Intervals

Problem Summary

Given an array of intervals, merge all overlapping intervals and return them.

Real-Life Scenario

A calendar wants to consolidate overlapping busy blocks into a single "busy" period.

Pattern Recognition

Clues that this is the right pattern:

- Sort by start. Walk through; if current overlaps with the previous, merge; else push as new.

Brute Force Approach

Repeatedly scan for any overlapping pair and merge — O(n^2) in the worst case.

Time complexity: O(n^2).

Space complexity: O(n).

Optimized Approach

Sort by interval start.

Initialize result with the first interval.

For each subsequent interval, compare its start with the last in result. If start <= last.end, extend last.end. Otherwise, append.

Time complexity: O(n log n).

Space complexity: O(n).

Dry Run

- intervals=[[1,3],[2,6],[8,10],[15,18]]. Sorted already.
- res=[[1,3]]. [2,6]: 2<=3 -> extend last to [1,6]. res=[[1,6]].
- [8,10]: 8>6 -> append. res=[[1,6],[8,10]].
- [15,18]: 15>10 -> append. res=[[1,6],[8,10],[15,18]].

Code Implementation

Java:

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> result = new ArrayList<>();

        for (int[] cur : intervals) {
            if (!result.isEmpty() && cur[0] <= result.get(result.size() - 1)[1]) {
                // overlap -> extend last
                int[] last = result.get(result.size() - 1);
                last[1] = Math.max(last[1], cur[1]);
            } else {
                result.add(cur);
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Python:

```python
class Solution:
    def merge(self, intervals):
        intervals.sort(key=lambda x: x[0])
        result = []
        for cur in intervals:
            if result and cur[0] <= result[-1][1]:
                result[-1][1] = max(result[-1][1], cur[1])
            else:
                result.append(cur)
        return result
```

Common Mistakes

- Forgetting to sort first.
- Using strict < instead of <= for overlap (e.g., [1,2] and [2,3] should merge).
- Allocating new intervals on every merge instead of mutating the last in place.

Key Takeaways

- Sort + linear merge is the universal pattern for interval consolidation.
- When extending, take max of ends — the second interval may end earlier than the first.

---

66. Non-Overlapping Intervals

Problem Summary

Given an array of intervals, return the minimum number to remove so the rest are non-overlapping.

Real-Life Scenario

Selecting the maximum set of non-conflicting meetings — what we keep is the answer; what we drop is the count we return.

Pattern Recognition

Clues that this is the right pattern:

- Greedy: sort by end time. Keep an interval if it starts at or after the last kept end. Otherwise, it must be removed.

Brute Force Approach

Try every subset and pick the largest non-overlapping one — exponential.

Time complexity: O(2^n).

Space complexity: O(n).

Optimized Approach

Sort by end time.

Greedily pick the first one. For each next, if its start >= last kept end, keep it; else, count++ (must drop).

Time complexity: O(n log n).

Space complexity: O(1) extra.

Dry Run

- intervals=[[1,2],[2,3],[3,4],[1,3]]. Sort by end: [[1,2],[2,3],[1,3],[3,4]].
- lastEnd=2 (kept [1,2]).
- [2,3]: start 2 >= 2 -> keep. lastEnd=3.
- [1,3]: start 1 < 3 -> remove. count=1.
- [3,4]: start 3 >= 3 -> keep. lastEnd=4. Return 1.

Code Implementation

Java:

```
JAVA
```

```
class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
        int count = 0;
        int lastEnd = Integer.MIN_VALUE;

        for (int[] iv : intervals) {
            if (iv[0] >= lastEnd) {
                lastEnd = iv[1]; // keep
            } else {
                count++; // remove
            }
        }
        return count;
    }
}
```

Python:

```
PYTHON
class Solution:
    def eraseOverlapIntervals(self, intervals) -> int:
        intervals.sort(key=lambda x: x[1])
        count = 0
        last_end = float('-inf')
        for s, e in intervals:
            if s >= last_end:
                last_end = e
            else:
                count += 1
        return count
```

Common Mistakes

- Sorting by start instead of end — sorting by end is what makes the greedy optimal.
- Counting kept intervals instead of removed ones — return value is the removed count.
- Using > instead of >= for the boundary (touching ends do not overlap).

Key Takeaways

- Classic interval-scheduling greedy: sort by end, pick the earliest-ending compatible.
- Touching endpoints (end of one == start of next) do not count as overlap.

---

67. Meeting Rooms

Problem Summary

Given an array of meeting time intervals, determine if a person could attend all meetings (i.e., no two overlap).

Real-Life Scenario

A booking system checks whether a single attendee's schedule has any conflicts.

Pattern Recognition

Clues that this is the right pattern:

- Sort by start time. If any meeting starts before the previous one ends, conflict.

Brute Force Approach

Check every pair — O(n^2).

Time complexity: O(n^2).

Space complexity: O(1).

Optimized Approach

Sort by start time.

For i from 1: if intervals[i].start < intervals[i-1].end, return false.

If the loop finishes, return true.

Time complexity: O(n log n).

Space complexity: O(1).

Dry Run

- intervals=[[0,30],[5,10],[15,20]]. Sort: same.
- i=1: 5 < 30 -> conflict. Return false.

Code Implementation

Java:

```java
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] < intervals[i - 1][1]) return false;
        }
        return true;
    }
}
```

Python:

```python
class Solution:
    def canAttendMeetings(self, intervals) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(1, len(intervals)):
            if intervals[i][0] < intervals[i-1][1]:
                return False
        return True
```

Common Mistakes

- Using <= when the correct check is < (touching meetings are fine, you can finish at 10 and start at 10).
- Forgetting to sort first.

Key Takeaways

- A simple sort + adjacent check answers it.
- Foundation for Meeting Rooms II — generalizes to "how many rooms do we need".

---

68. Meeting Rooms II

Problem Summary

Given meeting intervals, return the minimum number of meeting rooms required.

Real-Life Scenario

A scheduling system that books rooms — what is the peak number of simultaneous meetings?

Pattern Recognition

Clues that this is the right pattern:

- Two clean approaches: (a) Min-heap of end times — push starts, pop ends that finished. (b) Sweep line — separate starts and ends, two pointers, count overlaps.

Brute Force Approach

Check, for every minute on a timeline, how many meetings cover it. O(n * T) — slow when times are large.

Time complexity: O(n * T).

Space complexity: O(T).

Optimized Approach

Heap approach: sort by start. Push first end into a min-heap. For each next meeting, if its start >= heap.peek(), pop (room freed). Always push current end. Heap size at the end is the answer.

Sweep approach: sort starts and ends separately. Walk both arrays with pointers; if next start < current end, rooms++. Else, advance ends pointer (room freed).

Time complexity: O(n log n).

Space complexity: O(n).

Dry Run

- intervals=[[0,30],[5,10],[15,20]]. Sort by start: same.
- Heap=[30] after first. [5,10]: 5<30 -> push 10. Heap=[10,30].
- [15,20]: 15>=10 (peek) -> pop. Push 20. Heap=[20,30]. Size 2.

Code Implementation

Java:

```
JAVA
class Solution {
```

```
    public int minMeetingRooms(int[][] intervals) {
        if (intervals.length == 0) return 0;
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        PriorityQueue<Integer> heap = new PriorityQueue<>();
        for (int[] iv : intervals) {
            // If the earliest-ending meeting has finished, reuse that room
            if (!heap.isEmpty() && iv[0] >= heap.peek()) {
                heap.poll();
            }
            heap.offer(iv[1]);
        }
        return heap.size();
    }
}
```

Python:

```
PYTHON
import heapq

class Solution:
    def minMeetingRooms(self, intervals) -> int:
        if not intervals:
            return 0
        intervals.sort(key=lambda x: x[0])

        heap = []
        for s, e in intervals:
            if heap and s >= heap[0]:
                heapq.heappop(heap)
            heapq.heappush(heap, e)
        return len(heap)
```

Common Mistakes

- Using a max-heap or just tracking an integer count — the heap of end times gives you the exact freed room.
- Comparing with > instead of >= (touching ends free the room).
- Forgetting to push the current end after a pop — the new meeting still occupies a room.

Key Takeaways

- Min-heap of end times = elegant rooms-needed calculator.
- Heap size is the running count of simultaneous meetings.
- Sweep line is an equivalent alternative if heaps are unavailable.

---

18. Math & Geometry

Topic Introduction

What it is: Math & Geometry problems involve grids, rotations, spirals, and in-place matrix transformations. They test your ability to manipulate 2-D data carefully without extra memory.

Why it exists: Real software constantly transforms 2-D arrays — image rotation, screen redraws, matrix manipulations in machine learning, board-game state changes.

What problem it solves: In-place rotations, traversals in non-row-major order, marking states without extra storage.

Typical time complexity: Usually O(M * N).

Typical space complexity: O(1) when in-place; O(M * N) otherwise.

Real-life analogy: Rearranging photos in a printed grid: you can rearrange them without buying a second album, but you have to be careful which photo you move first.

Real-world software engineering use cases:

- Image processing: rotation, transposition.
- 2-D game state updates (chess, Sudoku).
- Matrix multiplication and linear algebra primitives.
- Console UI redraws.

69. Rotate Image

Problem Summary

Rotate an n x n 2-D matrix 90 degrees clockwise in place.

Real-Life Scenario

A photo viewer rotates a square image without allocating a second image buffer.

Pattern Recognition

Clues that this is the right pattern:

- Two-step trick: transpose the matrix (swap [i][j] with [j][i]), then reverse each row.

Brute Force Approach

Allocate a new n x n matrix; place each old element at its rotated position. Uses O(n^2) extra memory.

Time complexity: O(n^2).

Space complexity: O(n^2).

Optimized Approach

Transpose: for i < j, swap matrix[i][j] with matrix[j][i].

Reverse each row.

In-place; O(1) extra memory.

Time complexity: O(n^2).

Space complexity: O(1).

Dry Run

- matrix=[[1,2,3],[4,5,6],[7,8,9]].
- Transpose -> [[1,4,7],[2,5,8],[3,6,9]].
- Reverse each row -> [[7,4,1],[8,5,2],[9,6,3]].

Code Implementation

Java:

```JAVA
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        // Transpose in place (swap above the diagonal)
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int tmp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = tmp;
            }
        }
        // Reverse each row
        for (int i = 0; i < n; i++) {
            for (int l = 0, r = n - 1; l < r; l++, r--) {
                int tmp = matrix[i][l];
                matrix[i][l] = matrix[i][r];
                matrix[i][r] = tmp;
            }
        }
    }
}
```

Python:

```PYTHON
class Solution:
    def rotate(self, matrix) -> None:
        n = len(matrix)
        # Transpose
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        # Reverse each row
        for row in matrix:
            row.reverse()
```

Common Mistakes

- Looping j from 0 in the transpose — that swaps each pair twice, undoing the transpose.
- Using a temporary matrix when in-place is required.
- Mixing up "transpose then reverse rows" with "reverse rows then transpose" — both produce a rotation but in opposite directions.

Key Takeaways

- Two-pass transpose + reverse is the cleanest in-place rotation.
- For 90-degree counter-clockwise: transpose then reverse columns instead.

---

70. Spiral Matrix

Problem Summary

Given an m x n matrix, return all its elements in spiral order (clockwise from the top-left).

Real-Life Scenario

A pixel spiral pattern in an animation, or visiting cells of a board game spirally.

Pattern Recognition

Clues that this is the right pattern:

- Maintain four boundaries: top, bottom, left, right.
- Walk top-left to top-right, then top-right to bottom-right, then back across, then up. Shrink boundaries each turn.

Brute Force Approach

Direction simulation with a visited matrix and four direction vectors. Works but more code.

Time complexity: O(M * N).

Space complexity: O(M * N) for visited.

Optimized Approach

Initialize top=0, bottom=m-1, left=0, right=n-1.

Loop while top <= bottom and left <= right.

Traverse left -> right on row top; top++.

Traverse top -> bottom on column right; right--.

If top <= bottom: traverse right -> left on row bottom; bottom--.

If left <= right: traverse bottom -> top on column left; left++.

Time complexity: O(M * N).

Space complexity: O(1) extra (excluding the result list).

Dry Run

- matrix=[[1,2,3],[4,5,6],[7,8,9]]. top=0,bottom=2,left=0,right=2.
- Top row: 1,2,3. top=1.
- Right col: 6,9. right=1.
- Bottom row: 8,7. bottom=1.
- Left col: 4. left=1.
- Top row: 5. top=2. Loop ends. Result: [1,2,3,6,9,8,7,4,5].

Code Implementation

Java:

JAVA
```java
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        int top = 0, bottom = matrix.length - 1;
        int left = 0, right = matrix[0].length - 1;

        while (top <= bottom && left <= right) {
            for (int c = left; c <= right; c++) result.add(matrix[top][c]);
            top++;
            for (int r = top; r <= bottom; r++) result.add(matrix[r][right]);
            right--;
            if (top <= bottom) {
                for (int c = right; c >= left; c--) result.add(matrix[bottom][c]);
                bottom--;
            }
            if (left <= right) {
                for (int r = bottom; r >= top; r--) result.add(matrix[r][left]);
                left++;
            }
        }
        return result;
    }
}
```

Python:

PYTHON
```python
class Solution:
    def spiralOrder(self, matrix):
        result = []
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1

        while top <= bottom and left <= right:
            for c in range(left, right + 1):
                result.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1):
                result.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1):
                    result.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1):
                    result.append(matrix[r][left])
                left += 1
        return result
```

Common Mistakes

- Skipping the inner if-checks for non-square matrices — leads to double-visiting cells.
- Reversing the order of bottom-row or left-column traversals.
- Off-by-one with boundaries (using < n vs <= right).

Key Takeaways

- Four shrinking boundaries is the cleanest way to traverse a spiral.
- Always re-check the loop condition between the third and fourth segment to handle skinny matrices.

---

71. Set Matrix Zeroes

Problem Summary

Given an m x n matrix, if any element is 0, set its entire row and column to 0. Do it in place.

Real-Life Scenario

A spreadsheet zeroing out rows and columns of corrupted cells without copying the whole sheet.

Pattern Recognition

Clues that this is the right pattern:

- O(m+n) extra space is easy: track rows and columns to zero in two sets.
- O(1) trick: use the first row and first column as the markers themselves.

Brute Force Approach

Copy the matrix; in the copy, mark zeros for whole rows and columns. Uses O(m*n) extra.

Time complexity: O(m * n).

Space complexity: O(m * n).

Optimized Approach

Track whether the first row and first column themselves contain a 0 (booleans firstRowZero, firstColZero).

For each cell (i, j) with i>0 and j>0, if matrix[i][j]==0, set matrix[i][0]=0 and matrix[0][j]=0.

Second pass: for each cell (i, j) with i>0 and j>0, if matrix[i][0]==0 or matrix[0][j]==0, set matrix[i][j]=0.

Finally, zero out the first row and first column based on the saved flags.

Time complexity: O(m * n).

Space complexity: O(1).

Dry Run

- matrix=[[1,1,1],[1,0,1],[1,1,1]]. firstRowZero=false, firstColZero=false.
- Find (1,1)=0. Set matrix[1][0]=0 and matrix[0][1]=0.
- Second pass: (1,0)=0 -> entire row 1 to 0. (0,1)=0 -> entire col 1 to 0.
- Result: [[1,0,1],[0,0,0],[1,0,1]].

Code Implementation

Java:

```java
class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        boolean firstRowZero = false, firstColZero = false;
        for (int j = 0; j < n; j++) if (matrix[0][j] == 0) firstRowZero = true;
        for (int i = 0; i < m; i++) if (matrix[i][0] == 0) firstColZero = true;

        // Use first row and first column as markers
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        // Zero cells based on markers
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
            }
        }
        // Finally handle the first row and column
        if (firstRowZero) for (int j = 0; j < n; j++) matrix[0][j] = 0;
        if (firstColZero) for (int i = 0; i < m; i++) matrix[i][0] = 0;
    }
}
```

Python:

```python
class Solution:
    def setZeroes(self, matrix) -> None:
        m, n = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][j] == 0 for j in range(n))
        first_col_zero = any(matrix[i][0] == 0 for i in range(m))

        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        if first_row_zero:
            for j in range(n): matrix[0][j] = 0
        if first_col_zero:
```

```
            for i in range(m): matrix[i][0] = 0
```

Common Mistakes

- Forgetting to record whether the first row or column originally contained a 0.
- Zeroing the first row/col before using it as markers — destroys the markers.
- Iterating over (i, j) starting from 0 in the second pass — overwrites markers prematurely.

Key Takeaways

- Using existing storage as state (the first row/col) is a powerful in-place trick.
- Order of operations matters: read markers, then write, then handle the row/col last.

---

19. Bit Manipulation

Topic Introduction

What it is: Bit manipulation works directly with the binary representation of integers using AND, OR, XOR, NOT, and shifts.

Why it exists: Bit operations are extremely fast and let you encode flags, perform set operations on small universes, and solve problems with hidden structure (e.g., XOR cancels duplicates).

What problem it solves: Counting set bits, finding unique numbers among duplicates, encoding multiple flags, fast multiplication/division by powers of 2.

Typical time complexity: Each bit op is O(1). Loops over bits are O(32) or O(64) — effectively constant.

Typical space complexity: O(1).

Real-life analogy: A row of light switches (32 of them). Each switch is a bit, on or off. Toggling, querying, and combining switches is instant if you operate on the whole row at once.

Real-world software engineering use cases:

- Bitsets / bitmaps for compact storage (e.g., Bloom filters).
- Permission flags in OS file systems.
- Network protocol headers.
- Fast hashing and crypto primitives.
- Game state compression.

72. Number of 1 Bits

Problem Summary

Given an unsigned integer, return the number of 1 bits in its binary representation (also called the Hamming weight).

Real-Life Scenario

A network protocol counts set flags in a 32-bit field.

Pattern Recognition

Clues that this is the right pattern:

- Brian Kernighan trick: n & (n - 1) clears the lowest set bit. Iterate until n == 0.

Brute Force Approach

Loop 32 times: check each bit with (n >> i) & 1.

Time complexity: O(32).

Space complexity: O(1).

Optimized Approach

count = 0. While n != 0: n &= (n - 1) clears the lowest 1; count++.

Faster when most bits are 0.

Time complexity: O(k) where k is the number of set bits.

Space complexity: O(1).

Dry Run

- n = 0b1011 (11). count=0.
- n & (n-1) = 0b1011 & 0b1010 = 0b1010. count=1.
- n=0b1010. n&(n-1)=0b1010 & 0b1001=0b1000. count=2.
- n=0b1000. n&(n-1)=0. count=3. Stop. Return 3.

Code Implementation

Java:

```JAVA
public class Solution {
    public int hammingWeight(int n) {
        int count = 0;
        while (n != 0) {
            n &= (n - 1); // clear the lowest 1 bit
            count++;
        }
        return count;
    }
}
```

Python:

```PYTHON
class Solution:
    def hammingWeight(self, n: int) -> int:
        count = 0
        while n:
            n &= (n - 1)
            count += 1
        return count
```

Common Mistakes

- Using signed-right-shift on a negative integer in Java (>>) — use >>> for unsigned shift.
- Confusing AND (&) with logical AND (&&).

Key Takeaways

- n & (n - 1) is a famous bit trick. Memorize it.
- Loop runs once per set bit — beats the fixed 32-iteration version on sparse inputs.

---

73. Counting Bits

Problem Summary

Given an integer n, return an array ans of length n+1 where ans[i] is the number of 1 bits in i.

Real-Life Scenario

Pre-compute Hamming weights for 0..n to use later in a tight loop.

Pattern Recognition

Clues that this is the right pattern:

- DP based on the bit representation: ans[i] = ans[i >> 1] + (i & 1).
- i >> 1 drops the last bit; (i & 1) checks if that last bit was 1.

Brute Force Approach

Compute Hamming weight from scratch for each i — O(n log n).

Time complexity: O(n log n).

Space complexity: O(n).

Optimized Approach

Allocate ans[0..n], ans[0] = 0.

For i = 1..n: ans[i] = ans[i >> 1] + (i & 1).

Each entry uses a previously computed value -> O(1) per step.

Time complexity: O(n).

Space complexity: O(n) for the output.

Dry Run

- n=5. ans=[0,?,?,?,?,?].
- i=1: ans[1]=ans[0]+(1&1)=0+1=1.
- i=2: ans[2]=ans[1]+(2&1)=1+0=1.
- i=3: ans[3]=ans[1]+(3&1)=1+1=2.
- i=4: ans[4]=ans[2]+(4&1)=1+0=1.
- i=5: ans[5]=ans[2]+(5&1)=1+1=2. Result: [0,1,1,2,1,2].

Code Implementation

Java:

```
JAVA
class Solution {
```

```
    public int[] countBits(int n) {
        int[] ans = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            ans[i] = ans[i >> 1] + (i & 1);
        }
        return ans;
    }
}
```

Python:

```
PYTHON
class Solution:
    def countBits(self, n: int):
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans
```

Common Mistakes

- Using the slow per-element Hamming weight when the DP recurrence runs in O(1) per i.
- Mismatching the offset by 1 (i.e., i+1 instead of i).

Key Takeaways

- Bit-wise DP turns a O(n log n) into O(n).
- Pattern: dropping the last bit reduces a number to a previously computed state.

---

74. Reverse Bits

Problem Summary

Reverse the bits of a 32-bit unsigned integer.

Real-Life Scenario

A networking module receives data in big-endian and must reverse the bit order to little-endian.

Pattern Recognition

Clues that this is the right pattern:

- For each of the 32 bits, take the lowest bit of n and shift it onto the result, then shift n right.

Brute Force Approach

Convert to a binary string, reverse, parse back. Works but allocates strings.

Time complexity: O(32).

Space complexity: O(32).

Optimized Approach

result = 0. For 32 iterations: result = (result << 1) | (n & 1); n >>>= 1.

In-place arithmetic only.

Time complexity: O(32).

Space complexity: O(1).

Dry Run

- n = 0b00000010 (8 bits, simplified). Reversed should be 0b01000000.
- Iter 1: result=0<<1|0=0. n=0b00000001.
- Iter 2: result=0<<1|1=1. n=0b00000000.
- Iter 3-8: shift left, OR 0. result=0b01000000. Done.

Code Implementation

Java:

```JAVA
public class Solution {
    public int reverseBits(int n) {
        int result = 0;
        for (int i = 0; i < 32; i++) {
            result = (result << 1) | (n & 1);
            n >>>= 1; // unsigned right shift
        }
        return result;
    }
}
```

Python:

```PYTHON
class Solution:
    def reverseBits(self, n: int) -> int:
        result = 0
        for _ in range(32):
            result = (result << 1) | (n & 1)
            n >>= 1
        return result
```

Common Mistakes

- Using >> instead of >>> in Java — sign-extends and creates infinite loops on negatives.
- Forgetting to shift the result LEFT before OR-ing in the new bit.
- Ending up with extra bits beyond 32.

Key Takeaways

- Build the reversed number by always pushing onto the most-significant side.
- Java's unsigned right shift (>>>) is the correct operator here.

---

75. Missing Number

Problem Summary

Given an array nums containing n distinct numbers in [0, n], return the missing one.

Real-Life Scenario

A registration system numbered 0..n issued badges — one badge is missing; find which number.

Pattern Recognition

Clues that this is the right pattern:

- Two clean approaches. Sum: expected = n*(n+1)/2; missing = expected - sum(nums). XOR: x = 0..n XOR all nums; equal pairs cancel, missing remains.

Brute Force Approach

Sort and scan for the gap — O(n log n).

Time complexity: O(n log n).

Space complexity: O(1).

Optimized Approach

Sum approach: compute expected sum n*(n+1)/2 and subtract actual sum.

XOR approach: XOR all numbers from 0 to n with all elements of nums. Pairs cancel; the missing one survives.

Time complexity: O(n).

Space complexity: O(1).

Dry Run

- nums=[3,0,1]. n=3. Expected=3*4/2=6. Actual=4. Missing=2.
- XOR: 0^1^2^3 ^ 3^0^1 = 2.

Code Implementation

Java:

```JAVA
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int xor = n; // start with n so range is 0..n inclusive
        for (int i = 0; i < n; i++) {
            xor ^= i ^ nums[i];
        }
        return xor;
    }
}
```

Python:

```PYTHON
class Solution:
    def missingNumber(self, nums) -> int:
        xor = len(nums)
        for i, num in enumerate(nums):
```

```
            xor ^= i ^ num
        return xor
```

Common Mistakes

- For very large n, the sum approach can overflow in 32-bit integers — use long or XOR.
- Forgetting the off-by-one — range is 0..n, not 0..n-1.

Key Takeaways

- XOR cancels duplicates — perfect for "find the unpaired" problems.
- Sum is intuitive and equally fast; pick whichever is clearer for you.

---

76. Sum of Two Integers

Problem Summary

Compute the sum of two integers a and b without using the + or - operators.

Real-Life Scenario

A hardware-style exercise modeling an ALU adder using only logical gates.

Pattern Recognition

Clues that this is the right pattern:

- a XOR b is sum without carry. (a AND b) << 1 is the carry. Repeat until carry == 0.

Brute Force Approach

Increment a one step at a time using ++ — O(b). Slow.

Time complexity: O(|b|).

Space complexity: O(1).

Optimized Approach

Loop: while b != 0, sum = a XOR b, carry = (a AND b) << 1, then a = sum, b = carry.

Final answer is a.

Time complexity: O(32) — at most 32 iterations for 32-bit ints.

Space complexity: O(1).

Dry Run

- a=2 (0b10), b=3 (0b11).
- Iter 1: sum=2^3=0b01=1. carry=(2&3)<<1=0b10<<1=0b100=4. a=1, b=4.
- Iter 2: sum=1^4=0b101=5. carry=(1&4)<<1=0. a=5, b=0. Stop. Return 5.

Code Implementation

Java:

```
JAVA
class Solution {
    public int getSum(int a, int b) {
```

```
        while (b != 0) {
            int carry = (a & b) << 1;
            a = a ^ b;
            b = carry;
        }
        return a;
    }
}
```

Python:

```PYTHON
class Solution:
    def getSum(self, a: int, b: int) -> int:
        # Python ints are arbitrary precision; mask to 32 bits
        MASK = 0xFFFFFFFF
        MAX_INT = 0x7FFFFFFF
        while b != 0:
            carry = ((a & b) << 1) & MASK
            a = (a ^ b) & MASK
            b = carry
        # Convert back to signed
        return a if a <= MAX_INT else ~(a ^ MASK)
```

Common Mistakes

- In Python, forgetting to mask to 32 bits — the loop never terminates because Python ints grow without bound.
- Reading a after assigning sum to it but before saving carry — order of statements matters.
- Using + or - anywhere — defeats the exercise.

Key Takeaways

- XOR + carry mimics binary addition.
- Loop terminates because every iteration moves carries strictly leftward.
- Python needs explicit 32-bit masking for the bit trick to behave like a fixed-width adder.

Part B: Pattern Recognition

Most Blind 75 problems are not about clever code — they are about recognizing which family of techniques a problem belongs to. This section gives you a decision tree, a keyword-to-pattern map, and a comparison of the most common patterns.

Decision Tree: Which Pattern Should I Reach For?

Read the problem statement and ask the questions below in order. The first "yes" usually points to the right pattern.

1. Is the input a sorted array, or can be sorted, and you are looking for a value or a range?

- Yes -> Binary Search.
- Variations: searching for a boundary, searching in a rotated sorted array, searching for kth element.

2. Are you looking for a contiguous subarray or substring with some property?

- Fixed window size -> Sliding Window (fixed).
- Variable window with a constraint (e.g., "longest with at most K of X") -> Sliding Window (variable).
- Two indices that move toward each other on a sorted/symmetric structure -> Two Pointers.

3. Are you looking for pairs, triplets, or sums of indices in a sorted array?

- Two Pointers from opposite ends — works when pairing toward a target sum.
- Three Sum and similar -> sort + outer loop + two pointers.

4. Are you frequently asking "have I seen this value/element/key before?"

- Yes -> HashMap or HashSet. Trades space for O(1) lookup.
- Examples: Two Sum, Group Anagrams, Contains Duplicate, Longest Consecutive Sequence.

5. Does the problem involve a tree?

- Need to visit every node? DFS (recursive) or BFS (queue).
- Need level information? BFS by levels.
- BST search/insert/delete -> use the BST property to skip half the tree.
- Need a root-to-leaf path or aggregate? DFS with accumulators.

6. Does the problem involve a graph (or grid that behaves like a graph)?

- Connectivity / count regions -> DFS or BFS from each unvisited node.
- Shortest path in unweighted graph -> BFS.
- Shortest path in weighted graph -> Dijkstra (heap).
- Detect cycle in directed graph -> Topological sort (Kahn or DFS coloring).
- Group questions on dynamic edges -> Union-Find.

7. Does the problem ask for top K, kth largest, or running median?

- Top K / kth -> Heap (size K). Min-heap for top K largest, max-heap for top K smallest.
- Running median -> Two heaps (max-heap of lower half, min-heap of upper half).

8. Does the problem ask for "all combinations / permutations / paths / arrangements"?

- Backtracking. Build a partial solution; recurse to extend; undo on the way back.
- Reuse-allowed: recurse with same index. No reuse: recurse with index + 1.

9. Does the problem involve overlapping subproblems and a single optimal answer?

- Yes -> Dynamic Programming.
- 1-D state (one sequence, one position) -> 1-D DP.
- Two sequences, or grid -> 2-D DP.
- Test: can you write recursion that calls itself with similar arguments? Add a memo and you have DP.

10. Are you scheduling, merging, or comparing ranges?

- Sort by start or end, then walk linearly with one or two pointers.
- Examples: Merge Intervals, Insert Interval, Meeting Rooms II (heap of end times).

Keyword-to-Pattern Quick Map

Skim the problem for these clue words; they often reveal the intended pattern.

Keyword or clue in problem	Likely pattern
"sorted" or "sorted array"	Binary Search or Two Pointers
"contiguous", "subarray", "substring"	Sliding Window or Prefix Sum
"distinct", "unique", "duplicate"	HashSet / HashMap
"k closest", "top k", "kth largest"	Heap
"all combinations", "all permutations", "all subsets"	Backtracking
"shortest path" (unweighted)	BFS
"shortest path" (weighted)	Dijkstra
"connected components", "groups"	DFS/BFS or Union-Find
"prerequisites", "ordering", "build before"	Topological Sort
"prefix", "autocomplete", "dictionary"	Trie
"max sum", "min sum", "ways to..."	Dynamic Programming
"in-place", "O(1) space"	Two Pointers, in-place mutation, or bit tricks

Keyword or clue in problem	Likely pattern
"merge", "overlap", "schedule"	Intervals (sort + sweep)
"undo", "previous N", "next greater"	Stack

Pattern Comparison Table

When two patterns might both apply, this table helps you choose.

Pattern	Best for	Typical time	Typical space
Two Pointers	Pairs/triples in sorted arrays	O(n)	O(1)
Sliding Window	Contiguous subarrays with a constraint	O(n)	O(k) for window state
Binary Search	Sorted data or monotonic decisions	O(log n)	O(1)
HashMap / HashSet	"Have I seen this?" lookups	O(n) total	O(n)
Stack	Last-in-first-out, matching, monotonic	O(n)	O(n)
Heap / Priority Queue	Top-K, running median, scheduling	O(n log k)	O(k)
DFS	Exhaustive exploration, backtracking	O(V + E)	O(h) recursion
BFS	Shortest path in unweighted graphs/levels	O(V + E)	O(V)
Backtracking	Generate all combinations/permutations	Exponential	O(depth)
DP (1-D)	Sequence problems with overlapping subproblems	O(n) or O(n*k)	O(n) -> O(1)
DP (2-D)	Two sequences or grids	O(m*n)	O(m*n) -> O(min(m,n))
Union-Find	Dynamic connectivity, grouping	Nearly O(1) amortized	O(n)
Trie	Prefix queries on many strings	O(L) per op	O(total chars)
Greedy	Locally optimal -> globally optimal	O(n) or O(n log n)	O(1)

Common Pattern Mistakes

- Reaching for DP when greedy works. Test whether a single locally optimal step is provably best before writing a DP table.
- Reaching for backtracking when DP works. If you only need a count or a single best answer (not a list of solutions), DP is usually faster.
- Reaching for BFS when DFS is enough (or vice versa). Both work for "is there a path?" but BFS is needed for "shortest path in unweighted graphs".

- Forgetting that two pointers needs a sorted (or symmetric) input.
- Using a sliding window on a problem that does not have a contiguous structure — always verify the result is a contiguous slice.

Part C: Complexity Cheat Sheet

Big-O notation describes how an algorithm scales as input size grows. You do not need to be a math wizard — interview-grade Big-O is about recognizing a few standard shapes and being able to argue why your code matches one of them.

What Big-O Means in Plain English

Big-O is the answer to: "If I double the input size, what happens to the running time?" It ignores constant factors and small terms because, at scale, they stop mattering.

- O(1) — running time does not depend on input size at all. Doubling the input does nothing.
- O(log n) — running time grows by a small constant when the input doubles. Halving each step.
- O(n) — running time doubles when input doubles.
- O(n log n) — running time slightly more than doubles. Sorting lives here.
- O(n^2) — running time quadruples when input doubles. Two nested loops.
- O(2^n) — exponential. Each new input element doubles the work. Avoid for n > ~25.
- O(n!) — factorial. Each new element multiplies the work by n. Avoid for n > ~10.

Common Operations: Time and Space

Operation	Average	Worst	Notes
Array access by index	O(1)	O(1)	Direct memory offset
Array search (unsorted)	O(n)	O(n)	Must scan every element
Array insert at end (dynamic)	O(1) amortized	O(n)	Occasional resize
Array insert at index	O(n)	O(n)	Must shift elements
HashMap get/put	O(1)	O(n)	Worst case is rare collisions
HashSet contains	O(1)	O(n)	Same backing structure
Sorted array binary search	O(log n)	O(log n)	Halving each step
Linked list traversal	O(n)	O(n)	No random access
Linked list insert at head	O(1)	O(1)	Pointer rewire only
Stack/Queue push/pop	O(1)	O(1)	Constant-time on both ends
Heap insert / extract-top	O(log n)	O(log n)	Bubble up/down
Heap peek-top	O(1)	O(1)	Always at the root
BST search/insert (balanced)	O(log n)	O(n)	Worst when skewed
Trie insert/search	O(L)	O(L)	L = word length
Union-Find union/find	Nearly O(1)	O(log n)	With path compression

Operation	Average	Worst	Notes
BFS / DFS on graph	O(V + E)	O(V + E)	Visit each vertex/edge once
Dijkstra (heap)	O((V+E) log V)	Same	Cannot handle negative edges
Sort (Java/Python default)	O(n log n)	O(n log n)	TimSort variant
Two-pointer / sliding window scan	O(n)	O(n)	Each pointer moves O(n)

How Fast is Each Class of Algorithm?

Rough rule of thumb: assume your machine can perform around 10^8 simple operations per second. These numbers help you decide if your draft solution will pass within typical time limits.

Complexity	n=10	n=1,000	n=100,000	n=1,000,000
O(1)	<1 micro-s	<1 micro-s	<1 micro-s	<1 micro-s
O(log n)	<1 micro-s	<1 micro-s	<1 micro-s	<1 micro-s
O(n)	<1 micro-s	<1 micro-s	~1 ms	~10 ms
O(n log n)	<1 micro-s	~0.01 ms	~17 ms	~200 ms
O(n^2)	<1 micro-s	~10 ms	~100 s	~3 hours
O(n^3)	<1 micro-s	~10 s	~30 years	never
O(2^n)	~10 micro-s	never	never	never
O(n!)	~36 ms	never	never	never

Rule of thumb during interviews: target O(n log n) or better when n >= 10^5; O(n^2) is fine when n <= 10^3; exponential and factorial are fine only when n <= 20.

How to Estimate Big-O of Your Code

1. Find the loops. A single pass over n elements is O(n). Nested loops over n elements are O(n^2). Loops where the index doubles or halves each step are O(log n).
2. Find the recursive calls. Each recursive call adds a layer. If recursion divides the input in half each time, depth is O(log n) (e.g., binary search). If it splits in half but processes everything (e.g., merge sort), it is O(n log n).
3. Account for the work inside the loop or recursion. Calling something that is itself O(n) inside a loop multiplies your complexity.
4. Add up the parts but drop lower-order terms and constants. O(n + n log n) is just O(n log n). O(3 * n) is just O(n).

Counting Space Complexity

- Auxiliary space (extra memory you allocate) is what counts. The input itself is usually free.

- Recursion uses stack space proportional to the maximum depth.
- A HashMap of n elements is O(n).
- In-place algorithms have O(1) auxiliary space (besides recursion).
- Always state space separately from time. Interviewers often ask "can you do that with O(1) extra space?"

Part D: Interview Strategy

Knowing the algorithms is half the battle. The other half is communicating clearly, managing time, and recovering when you get stuck. This section gives you a structured framework for working through any coding interview.

The UMPIRE Framework

Use this six-step framework for every problem. It gives you something to do at every moment so you never freeze.

U — Understand the Problem

- Read the prompt carefully. Restate it in your own words to confirm understanding.
- Ask about the input size: how big is n? Are values bounded?
- Ask about constraints: can the input be empty? Can it contain negatives? Duplicates? Can the array be unsorted?
- Ask about output format: a count, an actual list, a boolean, or in-place mutation?
- Walk through one or two simple examples by hand. This confirms your understanding and exposes edge cases.

M — Match a Pattern

- Run the input through the keyword map from Part B. Sorted? Contiguous? Sees-this-before? Top-K?
- Name the pattern out loud: "This looks like a sliding-window problem because we want a contiguous subarray with a constraint."
- If multiple patterns might apply, mention them and explain which one you will try first.

P — Plan

- Sketch the brute-force solution first. This proves you understand the problem and gives you a baseline complexity.
- Then describe an optimization. Use phrases like "we can avoid recomputing X by..." or "we can replace this O(n) lookup with a HashMap."
- State the final time and space complexity before writing code.
- If the interviewer pushes back, defend or revise your plan.

I — Implement

- Write code top-to-bottom in a clear style. Use meaningful variable names like "left", "right", "freq", not "i", "j", "x".
- Comment the tricky lines briefly. Comments are also documentation of your thinking.
- If you realize a bug while typing, stop and say it out loud — interviewers reward self-correction.
- Do not optimize prematurely. Get a correct version first, then talk about improvements.

R — Review

- Walk through your code with the example inputs you discussed at the start.

- Pay special attention to off-by-one errors, empty inputs, single-element inputs, and the largest inputs.
- Trace variable values out loud.
- Fix any bugs before the interviewer points them out.

E — Evaluate

- Restate the final time and space complexity.
- Mention any improvement that did not fit in the time you had.
- Talk briefly about extensions: "If the input were streamed, I would change this to..."

What to Say While You Are Thinking

Silence is the enemy. Use these phrases as a script when you need a moment to think.

When you are...	Say something like...
Reading the problem	"Let me restate the problem in my own words..."
Asking clarifications	"Just to confirm — can the input contain duplicates? What is the maximum size of n?"
Considering approaches	"There are a couple of ways to attack this. The brute force would be... A faster approach would be..."
About to start coding	"I am going to start with the optimized approach. The plan is..."
Stuck on a detail	"Let me think out loud here — I want to make sure I update this pointer correctly."
Spotting your own bug	"Wait — I just realized this would not handle the empty array case. Let me fix that."
Finishing up	"That is my full solution. Time complexity is..., space complexity is..."

When You Get Stuck

Everyone gets stuck. The interviewer is watching how you handle it, not whether it happens.

5. Pause and re-read the problem. You may have missed a constraint.
6. Walk through a small example by hand. Watch what your brain does — that is often the algorithm.
7. Try a related, simpler version of the problem first. Solve "valid parentheses with one type of bracket" before "with three types."
8. Step back and ask: "Is there a data structure that would make this lookup constant time?" or "Is there a way to avoid recomputing this?"
9. Talk through your stuck-ness. The interviewer can hint, but only if they hear what you are stuck on.
10. If truly blocked, fall back to brute force and ask if you can submit that and discuss optimizations.

Debugging Logically Under Pressure

- Add print statements (or describe them out loud) to track variable values at each step.
- Run the smallest test case in your head, line by line. Most bugs reveal themselves in 4-5 lines of trace.
- Check loop boundaries: are they inclusive or exclusive? Do you go to n-1 or n?
- Check pointer updates: do you increment when you should? Decrement? Both?
- For trees and graphs, draw the structure and trace the algorithm on the drawing.
- For DP, check base cases first. Most DP bugs are at the boundaries of the table.

Common Bug Categories and How to Catch Them

Bug type	Symptom	Quick check
Off-by-one	Crashes on first or last element	Test with n=1 and n=2
Empty input	NullPointerException or wrong answer	Test with empty array/string
Integer overflow	Mysterious negative numbers	Use long for sums; in Python, just be aware of mod operations
Mutation while iterating	ConcurrentModificationException or skipped elements	Iterate over a copy, or use index-based loop
Wrong base case	Stack overflow or wrong answer at depth 1	Trace the recursion at the smallest input
Stale pointer	Modified one variable but used another	Check variable update order
Boundary inequality	Off-by-one bug in subtle way	Test on touching/equal boundaries (e.g., interval [a, a])

Time Management in a 45-Minute Interview

- First 5 minutes: understand the problem, ask clarifying questions, walk through examples.
- Next 5 minutes: state brute force, propose optimization, agree on approach with the interviewer.
- Next 20-25 minutes: implement.
- Last 10 minutes: walk through your code with examples, fix bugs, discuss complexity, mention extensions.
- If you have not started coding by minute 15, you are spending too long on planning. Start the brute force.

Final Tips

- Practice talking out loud while you solve problems at home. Communication is a skill that does not develop silently.
- Re-do problems you got wrong a week later. Spaced repetition cements patterns.
- After each mock interview, write a one-page debrief: what worked, what to fix.
- Sleep before the interview. A rested brain spots patterns; a tired one misses them.
- Be kind to yourself. Even strong candidates fail occasional interviews. The pattern is what matters over many tries.

Good luck. You have done the work. Now show up and solve problems the way you have practiced.